



Tema

PROYECTO SEGUNDO PARCIAL

ALMEIDA, MERINO, PAEZ

Tutor

Ing. Eduardo Mauricio Campaña Ortega

MIS.MDU.CCNA.CCIA.

PhD. (c) Ingeniería de Software

PhD. (c) Seguridad Información

Fecha

17/12/2025

PROYECTO SOAP JAVA

PARTE PRÁCTICA	8
INSTALACIÓN DE HERRAMIENTAS	8
Diagrama de Casos de Uso del Módulo de Crédito del Banco BanQuito	8
Diagrama de Casos de Uso específico que verifica si el cliente es sujeto de crédito	9
Diagrama de Arquitectura	9
Modelo Entidad Relación BanQuito	10
CREACIÓN DE BD	10
APLICACIÓN SERVIDOR BANQUITO	18
CREACIÓN DEL PROYECTO SERVIDOR	18
CREACIÓN DE MODELOS	23
CREACIÓN DE CONTROLADORES	27
SERVIDOR COMERCIALIZADORA	46
CREACIÓN DEL PROYECTO COMERCIALIZADORA	46
CREACIÓN DE MODELOS	48
CREACIÓN DE CONTROLADORES	53
APLICACIÓN CLIENTE	81
CREACIÓN DEL PROYECTO CLIENTE ESCRITORIO	81
CREACIÓN DEL MODELO	81
CREACIÓN DEL CONTROLADOR	83
CREACIÓN DE LA VISTA	87
EJECUCIÓN	99
APLICACIÓN CLIENTE	102
CREACIÓN DEL PROYECTO CONSOLA	102
CODIFICACIÓN DE CLASES	107
CREACIÓN DEL CONTROLADOR	108
}EJECUCIÓN	113
APLICACIÓN CLIENTE	114
CREACIÓN DEL PROYECTO WEB	114
CREACIÓN CLASES CONTROLADOR	127
CLASE SERVICIO BANQUITOSERVICE	133

EJECUCION	134
APLICACIÓN CLIENTE	137
CREACIÓN DEL PROYECTO MÓVIL	138
VISTA	146
CONCLUSIONES	154
RECOMENDACIONES	154

ÍNDICE DE IMÁGENES

Figura 1 Creación de la carpeta raíz	8
Figura 2 Estructura inicial del proyecto	8
Figura 3 Pantalla principal SQL (Símbolo del Sistema)	9
Figura 4 Creación de base de datos	9
Figura 5 Cambiar a banquito en mysql	9
Figura 6 Uso de scripts para crear tablas y datos en cada una de ellas dentro de la base de datos	9
Figura 7 Tablas de la base de datos	32
Figura 8 Modelo de la Base de Datos	33
Figura 9 Nuevo proyecto	34
Figura 10 Aplicación tipo web	34
Figura 11 Asignación de nombre y ubicación	35
Figura 12 Configuración del servidor	35
Figura 13 Configuración de frameworks	36
Figura 14 Página principal del proyecto	36
Figura 15 Creación de los paquetes	37
Figura 16 Denominación de los paquetes	37
Figura 17 Verificación de la creación de paquetes	38
Figura 18 Estructura de los paquetes	38
Figura 19 Creación de la clase AccesoDB	39
Figura 20 Configuración de la clase	39
Figura 21 Codificación de la clase	40
Figura 22 Creación de la clase de pruebas para la conexión	43
Figura 23 Codificación clase de prueba	43
Figura 24 Error de comprobación del Driver	44
Figura 25 Bibliotecas del proyecto	45
Figura 26 Adición de JAR's	45
Figura 27 Selección del conector	46
Figura 28 Conector integrado en el proyecto	46

Figura 29 Resultado de la prueba de conexión a la base	46
Figura 30 Creación de la clase movimiento	47
Figura 31 Creación de la clase	47
Figura 32 Creación de la prueba de la clase Movimiento	58
Figura 33 Prueba de ejecución de los movimientos	59
Figura 34 Creación de la prueba de la clase Depósito	62
Figura 35 Confirmación de la realización del depósito	62
Figura 36 Verificación en mysql cmd	62
Figura 37 Creación del nuevo servicio web	64
Figura 38 Configuración de nombre y ubicación	64
Figura 39 Codificación de los servicios web	65
Figura 40 Implementación de etiquetas	65
Figura 41 Codificación del servicio web	66
Figura 42 Adición de tags	66
Figura 43 Despliegue de la aplicación	66
Figura 44 Inicio el servidor	67
Figura 45 Verificación de los servicios web creados	67
Figura 46 Prueba de servicios	67
Figura 47 Tester del IDE	68
Figura 48 Búsqueda	68
Figura 49 Resultado mostrado en el motor de búsqueda	69
Figura 50 Página principal de descarga	69
Figura 51 Pagina de descarga	70
Figura 52 Mensaje de confirmación de descarga	70
Figura 53 Pantalla inicial de instalación y configuración	70
Figura 54 Configuración de ruta	71
Figura 55 Instalación de componentes	71
Figura 56 Pantalla final de instalación	72
Figura 57 Pantalla principal del aplicativo	72
Figura 58 Pantalla inicial	72
Figura 59 Creación de nuevo proyecto	73
Figura 60 Estructura tras cargar el proyecto	73

Figura 61 Lectura del WSDL del servicio	73
Figura 62 Petición del servicio	74
Figura 63 Respuesta del servicio incorrecto	74
Figura 64 Respuesta del servicio correcto	74
Figura 65 Verificación del movimiento	75
Figura 66 Creación del proyecto cliente	75
Figura 67 Creación del cliente del servicio web	76
Figura 68 Ubicación del proyecto servidor	76
Figura 69 Estructura inicial del proyecto	77
Figura 70 Creación de la clase	77
Figura 71 Llamada a los métodos del servicio web	78
Figura 72 Generación de la capa de pruebas	91
Figura 73 Clases de pruebas	91
Figura 74 Resultado de la prueba del servicio movimientos	96
Figura 75 Resultados de la prueba de servicio depósito	96
Figura 76 Creación del controlador	96
Figura 77 Creación de las vistas	99
Figura 78 Prueba a la interfaz gráfica principal	99
Figura 79 Prueba a la interfaz gráfica de depósito	99
Figura 80 Prueba a la interfaz gráfica de consulta	100
Figura 81 Ejecución de depósito	140
Figura 82 Ejecución de la consulta de movimientos	140
Figura 83 Creación Dotnet Application	141
Figura 84 Creación proyecto estructura general	141
Figura 85 Creación RestFul	142
Figura 86 Asignar servicios al cliente	142
Figura 87 Verificar directorios de creación	143
Figura 88 90 Servicios generados en el cliente	143
Figura 89 Creación paquete ec.edu.monster.controlador	143
Figura 90 Creación paquete ec.edu.monster.servicio	144
Figura 91 Creación paquete ec.edu.monster.db	144
Figura 92 Creación paquete ec.edu.monster.modelo	145

Figura 93 Creación paquete ec.edu.monster.pruebas	145
Figura 94 Estructura proyecto cliente	145
Figura 95 Creación clase Telefono	146
Figura 96 Codificación clase Telefono	146
Figura 97 Creación clase TelefonoController	147
Figura 98 Codificación clase TelefonoController	147
Figura 99 Creación clase WSTelefono	150
Figura 100 Codificación clase WSTelefonoClient	150
Figura 101 Ejecución clase WSTelefonoClient	157
Figura 102 Creación directorio 02.CLIENTE WEB	158
Figura 103 Creación directorio WSTelefono	158
Figura 104 Creación estructura proyecto Web	158
Figura 105 Ejecución Vista Principal	178
Figura 106 Ejecución Vista Consultar Movimientos	179
Figura 107 Ejecución Vista Realizar Depósitos	179
Figura 108 Configuración Nuevo Proyecto	180
Figura 109 Estructura proyecto	180
Figura 110 Creación nueva clase en paquete modelo	180
Figura 111 Codificación clase Movimiento	180
Figura 112 Creación clase TelefonoService	182
Figura 113 Creacion SoapClient	185
Figura 114 Creación archivo DepositosScreen, MenuScreen & MovimientosScreen	186
Figura 115 Ejecución Pantalla Principal	196
Figura 116 Ejecución Pantalla Depósitos	196
Figura 117 Ejecución Pantalla Consultas	197

ÍNDICE DE TABLAS

Tabla 1 Script para la creación de objetos de la base de datos	11
--	----

Tabla 2 Clase Acceso Base de Datos	42
Tabla 3 Clase de prueba de conexión	45
Tabla 4 Codificación clase Movimiento	49
Tabla 5 Codificación Clase TelefonoService	54
Tabla 6 Codificación prueba de la clase movimiento	61
Tabla 7 Codificación prueba depósito	64
Tabla 8 Clase llamada a servicios web	80
Tabla 9 Clase llamada a servicios web	93
Tabla 10 Clase de prueba movimientos	95
Tabla 11 Clase de prueba depósito	96
Tabla 12 Codificación controlador	98
Tabla 13 ConsultaView	102
Tabla 14 MainView	116
Tabla 15 DepositoView	127
Tabla 16 Codificación controlador	149
Tabla 17 Codificación Clase WSTelefonoClient	152
Tabla 18 Codificación Vista Principal	160
Tabla 19 Codificación consultarMovimientos	161
Tabla 20 Codificación deposito	164
Tabla 21 Codificación movimientos	167
Tabla 22 Codificación resultado	170
Tabla 23 DepositoServlet	172
Tabla 24 IndexServlet	174
Tabla 25 MovimientosServlet	175
Tabla 26 TelefonoService codigo	177
Tabla 27 codificación clase Movimiento	181
Tabla 28 Codificación clase TelefonoService	183
Tabla 29 Codificación clase TelefonoController	186
Tabla 30 Codificación archivo DepositosScreen	187
Tabla 31 Codificación archivo MenuScreen	190
Tabla 32 Codificación MovimientosScreen	193

PARTE PRÁCTICA

Para la realización de la parte práctica de este proyecto, es necesario realizar los siguientes pasos detallados a continuación y adaptarlo a las necesidades de cada individuo.

INSTALACIÓN DE HERRAMIENTAS

Las herramientas a utilizar son las siguientes, PowerDesigner como schema para el modelamiento de diagramas y las bases de datos, SQL Server como gestor de base de datos, Google Cloud Platform para el despliegue de la base de datos en la nube, Apache Netbeans 23 como IDE de desarrollo, Laravel para la realización del cliente web, Dotnet para los clientes de escritorio/console y finalmente Android Studio para la realización del cliente móvil.

Diagrama de Casos de Uso del Módulo de Crédito del Banco BanQuito

Partimos creando el diagrama de casos de uso general para el módulo de BanQuito en Power Designer para este caso se ha definido de la siguiente manera

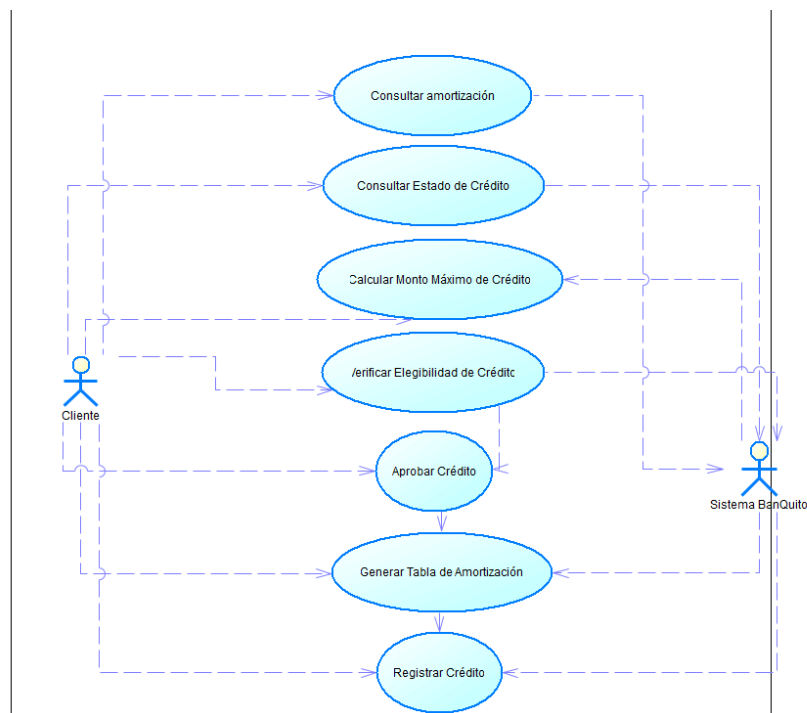


Figura 1 Creación del caso de uso general

Diagrama de Casos de Uso específico que verifica si el cliente es sujeto de crédito

Diagramamos el caso de uso específico que nos ayudara a comprobar si un cliente es sujeto de crédito.

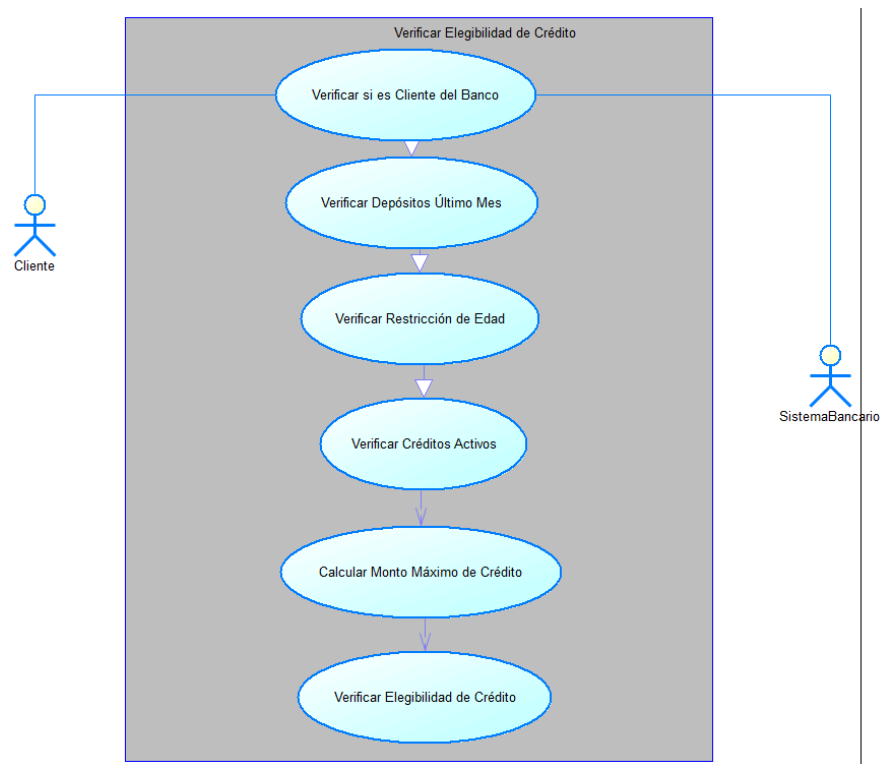


Figura 2 Caso de uso específico PowerDesigner

Diagrama de Arquitectura

Se procede con la diagramación de la arquitectura del sistema esta incluye 2 servidores uno para el banco y otro para la comercializadora además de 2 bases de datos y 4 clientes

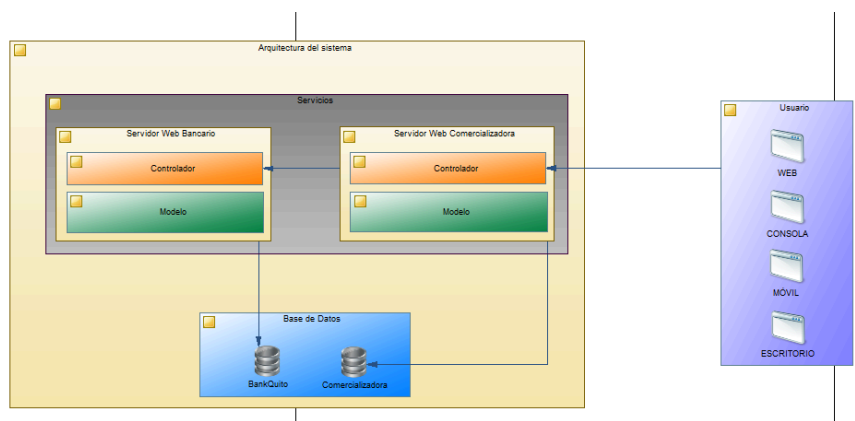


Figura 3 Arquitectura general del sistema

Modelo Entidad Relación BanQuito

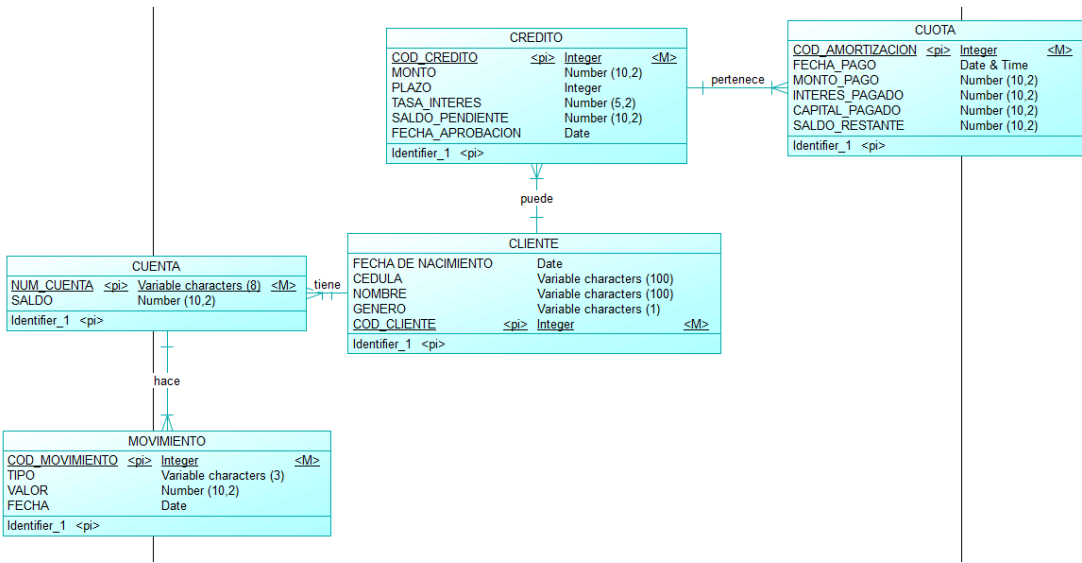


Figura 4 Modelo Entidad Relación en PowerDesigner

CREACIÓN Y DESPLIEGUE DE LA BASE DE DATOS EN LA NUBE

Una vez realizados los pasos anteriores, se procede con la creación de una base de datos con sus tablas y datos, para ello es necesario realizar lo siguiente.

Iniciamos sesión con nuestra cuenta de google



Figura 3 Web de Google Cloud

Solicitamos la prueba gratis de google cloud con credito de 300\$

Te damos la bienvenida, JOHNNY SEBASTIAN LOACHAMIN CHICAIZA

Prueba Google Cloud con el crédito gratis de \$300

- ✓ Acceso a productos y servicios de Google Cloud
- ✓ 90 días para invertir tus créditos
- ✓ Sin facturación durante la prueba

[Probar gratis](#)

Figura 4 Opción de crédito gratis en la nube

Una vez le damos a probar gratis tenemos acceso a los productos ofrecidos por Google Cloud

Productos

 Crea una VM Compute Engine	 Crea una base de datos Cloud SQL	 Ejecuta apps alojadas en contenedores Cloud Run	 Crea un bucket de almacenamiento Cloud Storage
 Crea una app alojada en contenedores Kubernetes Engine	 Accede a servicios APIs	 Crea una red VPC	 Compila una app web o para dispositivos... Firebase

[Ver todos los productos](#) [Ver todas las APIs](#) [Comparar con AWS y Azure](#)

Instructivos interactivos

 Crea un sitio web personalizado con... 15 min	 Implementa una aplicación web alojad... 10 min	 Crea e implementa un servicio de Cloud Run 10 min	 Crea una instancia de MySQL con Cloud SQL 15 min
 Crea un bucket de Cloud Storage 5 min	 Implementa un sitio web de WordPress co... 15 min	 Crea un bucket de Cloud Storage con un... 10 min	

Figura 5 Productos ofrecidos por Google Cloud

Entre las opciones seleccionamos Cloud SQL este producto nos permitirá subir nuestra base de datos. El paso siguiente es crear una instancia dentro de esta seleccionamos un motor de base de datos, en nuestro caso MySQL.

Elige tu motor de base de datos

MySQL

Versiones: 8.4, 8.0, 5.7, 5.6

[Elegir MySQL](#)

PostgreSQL

Versiones: 17, 16, 15, 14, 13, 12, 11, 10, 9.6

[Elegir PostgreSQL](#)

SQL Server

Versiones: 2022, 2019, 2017

[Elegir SQL Server](#)

¿Quieres obtener más información sobre los motores de base de datos de Cloud SQL?

[Más información](#)

Presentamos AlloyDB para PostgreSQL

PostgreSQL más lo mejor de la nube: procesamiento y almacenamiento con escalamiento horizontal, almacenamiento en caché inteligente y administración potenciada por IA y AA. Elige AlloyDB para cargas de trabajo transaccionales 4 veces más rápidas, consultas analíticas y vectoriales más eficientes y ANS del 99.99% líder en la industria, con mantenimiento incluido.

[ELEGIR ALLOYDB](#)[MÁS INFORMACIÓN](#)

Figura 6 Motor de base de datos

Habilitamos la API para poder tener la mayoría de funciones de Cloud SQL

Posteriormente elegimos una ediciones de cloud SQL en nuestro caso es Enterprise

← Crea una instancia de MySQL

Elige una edición de Cloud SQL

Una edición de Cloud SQL determina las características fundamentales de tu instancia.

Elige la mejor opción según tu presupuesto y necesidades de rendimiento. [Más información](#)☐ Enterprise Plus

- ANS de disponibilidad del 99.99%
- Tiempo de inactividad por mantenimiento planificado en menos de un segundo
- Escalación vertical de instancias con tiempo de inactividad casi nulo
- Máquinas con rendimiento optimizado
- Hasta 35 días para la ventana de recuperación de un momento determinado
- Capacidad de procesamiento hasta 3 veces mayor con caché de datos
- Recuperación ante desastres avanzada con cambio fácil

☒ Enterprise

- ANS de disponibilidad del 99.95%
- Menos de 60 segundos de tiempo de inactividad por mantenimiento planificado
- Máquinas de uso general
- Hasta 7 días para la ventana de recuperación de un momento determinado

Resumen

Edición de Cloud SQL	Enterprise
Región	us-central1 (Iowa)
Versión de la base de datos	MySQL 8.0
CPU virtuales	2 CPU virtual(es)
RAM	8 GB
Caché de datos	Inhabilitada
Almacenamiento	10 GB
Conexiones	IP pública
Copia de seguridad	Automatizada
Disponibilidad	Zona única
Recuperación de un momento determinado	Habilitada
Capacidad de procesamiento de la red (MB/s)	500 de 500
IOPS	Lectura: 300 de 15,000 Escritura: 300 de 15,000
Capacidad de procesamiento del disco (MB/s)	Lectura: 4.8 de 240.0 Escritura: 4.8 de 240.0

Elige un ajuste predeterminado para esta edición. Los ajustes predeterminados se pueden personalizar más adelante según sea necesario.

Se aplicó la configuración "Zona de pruebas"

Estos elementos representan únicamente los recursos de

Figura 7 Edición de Cloud SQL

Elegimos la información de nuestra instancia

Versión de la base de datos *
MySQL 8.0

▼

▼

MOSTRAR VERSIONES SECUNDARIAS

ID de instancia *
banquito-dba

Usa letras minúsculas, números y guiones. Comienza con una letra.

Contraseña

GENERAR

Establece una contraseña para el usuario raíz. [Más información](#)

☒

Sin contraseña

Sin una contraseña raíz, esta instancia permitirá que todos puedan conectarse con los privilegios de administrador completos. Establece una contraseña raíz para asegurarte de que solo los usuarios autorizados tengan estos privilegios.

▼

POLÍTICA DE CONTRASEÑAS

Elige la región y la disponibilidad zonal

Para obtener un mejor rendimiento, mantén tus datos cerca de los servicios que los necesitan. La región es permanente, mientras que la zona se puede cambiar en cualquier momento.

Región
us-central1 (Iowa)

Disponibilidad zonal
☒ Zona única
In case of outage, no failover. Not recommended for production.

Figura 8 Información de la instancia

Además de la región debemos agregar una nueva red a través de la cual podremos conectarnos desde nuestra dirección a la nube.

Conexiones

Elige cómo quieres que se conecte la fuente a esta instancia y, luego, define las redes que estarán autorizadas para conectarse. [Más información](#)

Puedes usar el proxy de Cloud SQL para aumentar la seguridad con cualquiera de las opciones. [Más información](#)

Asignación de IP de la instancia
☐ IP privada
Asigna una dirección IP de VPC interna alojada en Google. Se requieren API y permisos adicionales. No se puede inhabilitar una vez que se habilitó. [Más información](#)
☒ IP pública
Asigna una dirección IP externa a la que se puede acceder a través de Internet. Se requiere el uso de una red autorizada o el proxy de Cloud SQL para conectarse a esta instancia. [Más información](#)

Redes autorizadas
Puedes especificar rangos de CIDR para permitir que las direcciones IP de esos rangos accedan a tu instancia. [Más información](#)

^ Nueva red

Nombre

Usar [Notación CIDR](#)

Red *

Ejemplo: 199.27.25.0/24

Figura 9 Conexiones

Agregamos redes con su ip pública que nos permitirán la conexión.

Redes autorizadas
Puedes especificar rangos de CIDR para permitir que las direcciones IP de esos rangos accedan a tu instancia. [Más información](#)

▼ ESPE (192.188.58.15/32)	🗑️
▼ Casa (186.69.112.45/32)	🗑️
AGREGAR UNA RED	

Figura 10 Agregar redes

Una vez cumplido con todo seleccionamos crear instancia

^ OCULTAR OPCIONES DE CONFIGURACIÓN

[CREAR INSTANCIA](#) [CANCELAR](#)

Figura 11 Crear instancia.

Hecho esto en nuestra página de inicio Google Cloud vamos a ver la instancia que hemos creado

banquito-mysql	Auditoria no habilitada No hay política de contraseña Permite conexiones directas sin encriptar	Enterprise	MySQL 8.0	34.69.193.248
----------------	--	------------	-----------	---------------

Figura 12 Crear instancia.

Si la seleccionamos podemos ver todas sus opciones incluyendo la parte de conexiones donde vamos a encontrar la ip pública por medio de la cual vamos a subir la base de datos.

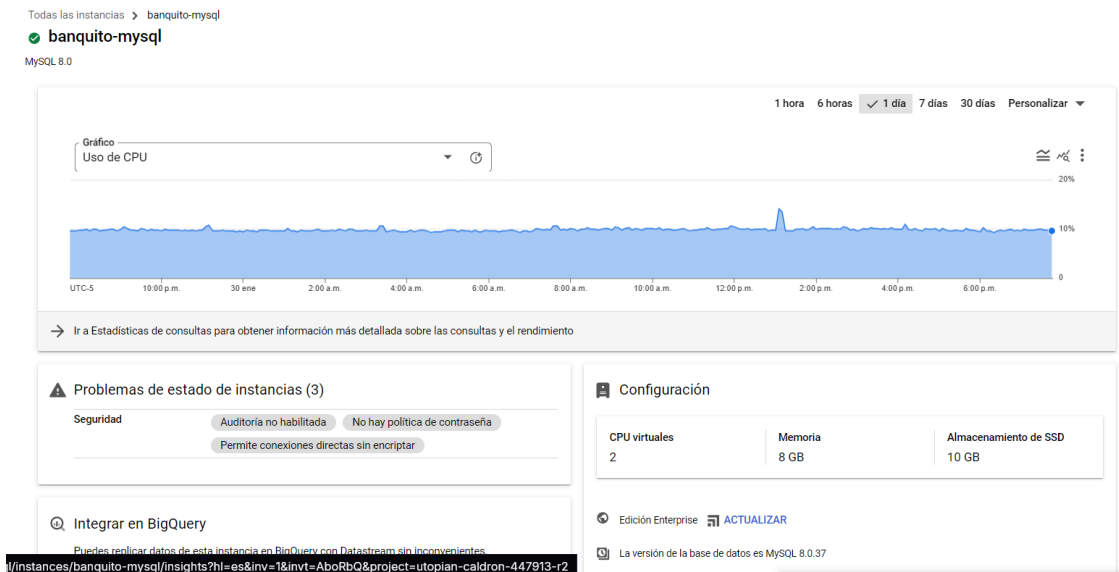


Figura 13 Pantalla Principal.

Opción de conexiones

Todas las instancias > banquito-mysql

banquito-mysql

MySQL 8.0

RESUMEN REDES SEGURIDAD PRUEBAS DE CONECTIVIDAD

Redes	
Nombre de la conexión	utopian-caldron-447913-r2:us-central1:banquito-mysql
Conectividad de IP privada	Inhabilitado
Conectividad de IP pública	Habilitado
Dirección IP pública	34.69.193.248
Seguridad	
Redes autorizadas	3 networks
	ESPE : 192.188.58.23/32 Casa-Erick : 186.69.75.215/32 Casa-Johnny : 186.69.112.45/32
Autorización de servicios de Google Cloud	Inhabilitado
Autorización de App Engine	Habilitado
Encriptación SSL / TLS	
Solo permitir conexiones SSL	Inhabilitado
Exigir certificados de cliente de confianza	Inhabilitado
Certificado del servidor	Vence en 26 ene 2035 20:07:28.

Figura 14 Conexiones.

El siguiente paso es crear los schemas de base de datos para esto abrimos nuestro MySQL Workbench en Windows, o cualquier gestor de base de datos que tengamos, nos conectamos a MySQL con nuestras credenciales

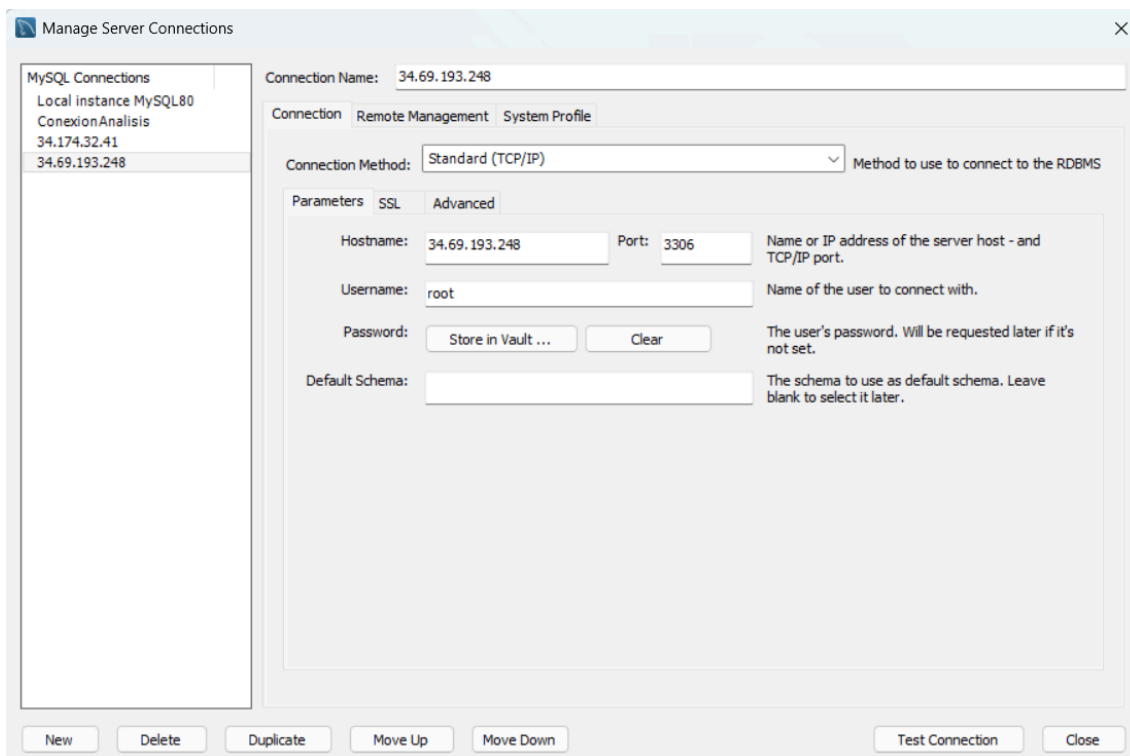


Figura 15 Configuración de conexión.

Utilizamos nuestra IP pública obtenida y la introducimos en hostname junto con el puerto por defecto que es 3306 además del usuario por defecto que es root.

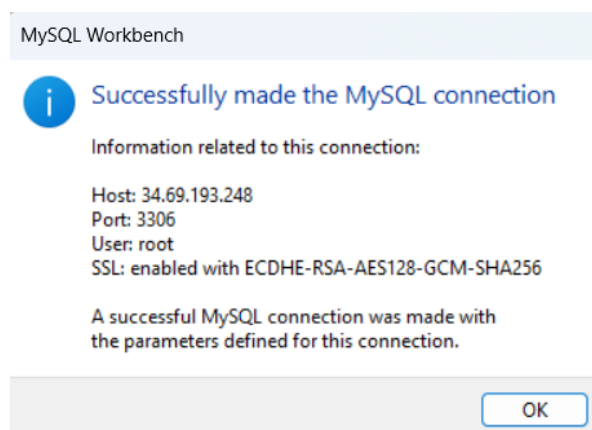


Figura 16 Test de conexión.

Una vez establecida la conexión procedemos a crear los schemas para banquito y la comercializadora mediante la introducción de sus respectivos scripts.

```
1  -- Crear la base de datos si no existe y seleccionarla
2  • CREATE DATABASE IF NOT EXISTS banquito;
3  • USE banquito;
4
5  -- =====
6  --          TABLA CLIENTE
7  -- =====
8  • DROP TABLE IF EXISTS `cliente`;
9  • CREATE TABLE `cliente` (
10     `COD_CLIENTE` INT(11) NOT NULL AUTO_INCREMENT,
11     `COD_CREDITO` INT(11) DEFAULT NULL,
12     `CEDULA` VARCHAR(100) DEFAULT NULL,
13     `NOMBRE` VARCHAR(100) DEFAULT NULL,
14     `GENERO` VARCHAR(100) DEFAULT NULL,
15     `FECHA_NACIMIENTO` DATETIME DEFAULT NULL,
16     PRIMARY KEY (`COD_CLIENTE`)
17 ) ENGINE=InnoDB AUTO_INCREMENT=30 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
18
19 -- Insertar datos en CLIENTE (sin borrar los existentes)
20 • INSERT INTO `cliente` (`CEDULA`, `NOMBRE`, `GENERO`, `FECHA_NACIMIENTO`)
```

Figura 17 Creación de schemas..

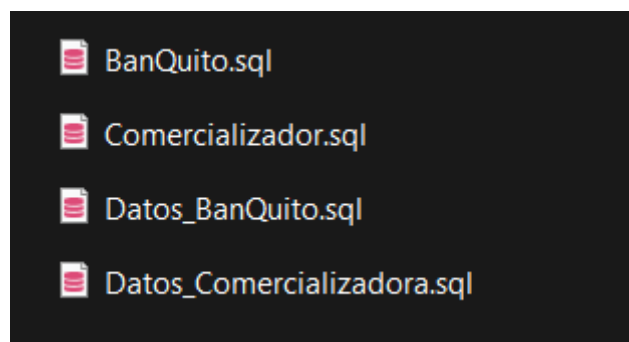


Figura 18 Scripts de creación.

Tabla 1 Script para la creación de objetos de la base de datos

```
-- Crear la base de datos si no existe y seleccionarla

CREATE DATABASE IF NOT EXISTS banquito;

USE banquito;

-- =====

--          TABLA CLIENTE

-- =====

DROP TABLE IF EXISTS `cliente`;

CREATE TABLE `cliente` (

  `COD_CLIENTE` INT(11) NOT NULL AUTO_INCREMENT,

  `COD_CREDITO` INT(11) DEFAULT NULL,

  `CEDULA` VARCHAR(100) DEFAULT NULL,

  `NOMBRE` VARCHAR(100) DEFAULT NULL,

  `GENERO` VARCHAR(100) DEFAULT NULL,

  `FECHA_NACIMIENTO` DATETIME DEFAULT NULL,

  PRIMARY KEY (`COD_CLIENTE`)

) ENGINE=InnoDB AUTO_INCREMENT=30 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
```

```

-- Insertar datos en CLIENTE (sin borrar Los existentes)

INSERT INTO `cliente` (`CEDULA`, `NOMBRE`, `GENERO`, `FECHA_NACIMIENTO`)

VALUES

('1723456789', 'Juan Perez', 'Masculino', '1985-02-15'),

('1729876543', 'Maria Lopez', 'Femenino', '1990-06-10'),

('1721234567', 'Carlos Gomez', 'Masculino', '1978-12-22'),

('1728765432', 'Ana Torres', 'Femenino', '1982-09-08');

-- =====

--          TABLA CREDITO

-- =====

DROP TABLE IF EXISTS `credito`;

CREATE TABLE `credito` (

  `COD_CREDITO` INT(11) NOT NULL AUTO_INCREMENT,

  `COD_CLIENTE` INT(11) NOT NULL,

  `SALDO` DECIMAL(10,2) DEFAULT NULL,

  `FECHA` DATETIME DEFAULT NULL,

```

```

`NUM_CUOTAS` INT(11) DEFAULT NULL,

PRIMARY KEY (`COD_CREDITO`),

FOREIGN KEY (`COD_CLIENTE`) REFERENCES `cliente` (`COD_CLIENTE`) ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;

-- Insertar datos en CREDITO

INSERT INTO `credito` (`COD_CLIENTE`, `SALDO`, `FECHA`, `NUM_CUOTAS`)

VALUES

(26, 5000.00, '2024-01-01', 12),

(27, 3000.00, '2024-02-01', 6),

(28, 7000.00, '2023-12-15', 24),

(29, 2500.00, '2024-03-10', 10);

-- =====

--          TABLA CUOTA

-- =====

DROP TABLE IF EXISTS `cuota`;

CREATE TABLE `cuota` (

`COD_CUOTA` INT(11) NOT NULL AUTO_INCREMENT,

`COD_CREDITO` INT(11) NOT NULL,

```

```

`NUMERO_CUOTA` INT(11) NOT NULL,

`VALOR_CUOTA` DECIMAL(10,2) NOT NULL,

`INTERES_PAGADO` DECIMAL(10,2) NOT NULL,

`CAPITAL_PAGADO` DECIMAL(10,2) NOT NULL,

`SALDO_RESTANTE` DECIMAL(10,2) NOT NULL,

`FECHA` DATETIME DEFAULT NULL,

PRIMARY KEY (`COD_CUOTA`),

FOREIGN KEY (`COD_CREDITO`) REFERENCES `credito` (`COD_CREDITO`) ON DELETE CASCADE ON UPDATE CASCADE

) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;

-- Insertar datos en CUOTA

INSERT INTO `cuota` (`COD_CREDITO`, `NUMERO_CUOTA`, `VALOR_CUOTA`, `INTERES_PAGADO`, `CAPITAL_PAGADO`, `SALDO_RESTANTE`,
`FECHA`)

VALUES

(1, 1, 500.00, 50.00, 450.00, 4500.00, '2024-02-01'),

(1, 2, 500.00, 45.00, 455.00, 4000.00, '2024-03-01'),

(2, 1, 600.00, 30.00, 570.00, 2400.00, '2024-03-01'),

(2, 2, 600.00, 25.00, 575.00, 1800.00, '2024-04-01'),

(3, 1, 300.00, 40.00, 260.00, 6740.00, '2024-01-15'),

```

```

(3, 2, 300.00, 38.00, 262.00, 6480.00, '2024-02-15');

-- =====

--          TABLA CUENTA

-- =====

DROP TABLE IF EXISTS `cuenta`;

CREATE TABLE `cuenta` (
  `NUM_CUENTA` VARCHAR(100) NOT NULL,
  `COD_CLIENTE` INT(11) NOT NULL,
  `SALDO` DECIMAL(10,2) DEFAULT NULL,
  PRIMARY KEY (`NUM_CUENTA`),
  FOREIGN KEY (`COD_CLIENTE`) REFERENCES `cliente` (`COD_CLIENTE`) ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;

-- Insertar datos en CUENTA

INSERT INTO `cuenta` (`NUM_CUENTA`, `COD_CLIENTE`, `SALDO`)
VALUES
('0001', 26, 1500.00),
('0002', 27, 2000.00),
('0003', 28, 1000.00),

```

```

('0004', 29, 500.00);

-- =====
--          TABLA MOVIMIENTO
-- =====

DROP TABLE IF EXISTS `movimiento`;

CREATE TABLE `movimiento` (
  `COD_MOVIMIENTO` INT(11) NOT NULL AUTO_INCREMENT,
  `NUM_CUENTA` VARCHAR(100) NOT NULL,
  `TIPO` VARCHAR(8) DEFAULT NULL,
  `VALOR` DECIMAL(10,2) DEFAULT NULL,
  `FECHA` DATETIME DEFAULT NULL,
  PRIMARY KEY (`COD_MOVIMIENTO`),
  FOREIGN KEY (`NUM_CUENTA`) REFERENCES `cuenta` (`NUM_CUENTA`) ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB AUTO_INCREMENT=50 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;

-- Insertar datos en MOVIMIENTO

INSERT INTO `movimiento` (`NUM_CUENTA`, `TIPO`, `VALOR`, `FECHA`)
VALUES
('0001', 'DEPOSITO', 200.00, '2025-01-21 19:31:26'),

```



```

('0001', 'RETIRO', 50.00, '2025-01-23 19:31:26'),
('0002', 'DEPOSITO', 500.00, '2025-01-19 19:31:26'),
('0003', 'DEPOSITO', 300.00, '2025-01-16 19:31:26');

-- Insertar datos en CLIENTE

INSERT INTO `cliente` (`CEDULA`, `NOMBRE`, `GENERO`, `FECHA_NACIMIENTO`)
VALUES
('1726543210', 'Pedro Alvarado', 'Masculino', '1988-04-25'),
('1727890123', 'Lucia Fernández', 'Femenino', '1992-11-30'),
('1725678901', 'Ricardo Montoya', 'Masculino', '1980-07-14'),
('1723456788', 'Sofia Herrera', 'Femenino', '1995-05-20'),
('1726789012', 'Diego Castro', 'Masculino', '1983-10-02');

-- Insertar datos en CUENTA

INSERT INTO `cuenta` (`NUM_CUENTA`, `COD_CLIENTE`, `SALDO`)
VALUES
('0010', 30, 1800.00),
('0011', 31, 2200.00),
('0012', 32, 1950.00),
('0013', 33, 1670.00),

```

```

('0014', 34, 2100.00);

-- Insertar movimientos de DEPOSITO

INSERT INTO `movimiento` (`NUM_CUENTA`, `TIPO`, `VALOR`, `FECHA`)

VALUES

('0010', 'DEPOSITO', 400.00, '2025-01-29 10:00:00'),
('0011', 'DEPOSITO', 600.00, '2025-01-29 11:00:00'),
('0012', 'DEPOSITO', 300.00, '2025-01-29 12:00:00'),
('0013', 'DEPOSITO', 500.00, '2025-01-29 13:00:00'),
('0014', 'DEPOSITO', 700.00, '2025-01-29 14:00:00');

```

Una vez realizado el proceso anterior, se debe dirigir a opción de estructura y verificar la creación de todas las tablas.

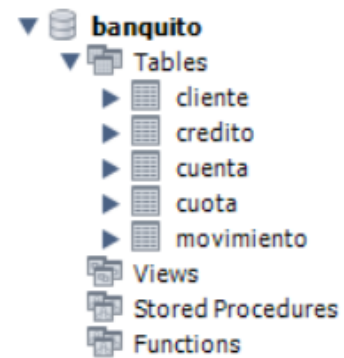


Figura 19 Tablas de la base de datos banquito

Tabla 2 Script para la creación de objetos de la base de datos comercializadora

```
-- Crear la base de datos si no existe y seleccionarla
CREATE DATABASE IF NOT EXISTS comercializadora;
USE comercializadora;

-- =====
--          TABLA CLIENTE
-- =====

DROP TABLE IF EXISTS `cliente`;

CREATE TABLE `cliente` (
  `CEDULA` VARCHAR(10) NOT NULL PRIMARY KEY,
  `NOMBRE` VARCHAR(100) NOT NULL,
  `EMAIL` VARCHAR(100) NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;

-- =====
--          TABLA FACTURA
-- =====

DROP TABLE IF EXISTS `factura`;

CREATE TABLE `factura` (
  `ID_FACTURA` INT(11) NOT NULL AUTO_INCREMENT,
```

```

`FECHA` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP(),
`CLIENTE_CEDULA` VARCHAR(10) NOT NULL,
`FORMA_PAGO` VARCHAR(50) NOT NULL,
`TOTAL` DECIMAL(10,2) NOT NULL,
`DESCUENTO` DECIMAL(10,2) NOT NULL DEFAULT 0.00,
PRIMARY KEY (`ID_FACTURA`),
FOREIGN KEY (`CLIENTE_CEDULA`) REFERENCES `cliente` (`CEDULA`) ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;

-- =====
--          TABLA TELEFONO
-- =====

DROP TABLE IF EXISTS `telefono`;

CREATE TABLE `telefono` (
  `ID_TELEFONO` INT(11) NOT NULL AUTO_INCREMENT,
  `MODELO` VARCHAR(100) NOT NULL,
  `MARCA` VARCHAR(50) NOT NULL,
  `PRECIO` DECIMAL(10,2) NOT NULL,
  `DESCRIPCION` VARCHAR(255) DEFAULT NULL,
  `IMAGEN_URL` VARCHAR(255) DEFAULT NULL,

```

```

PRIMARY KEY (`ID_TELEFONO`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;

-- Insertar datos en TELEFONO

INSERT INTO `telefono` (`MODELO`, `MARCA`, `PRECIO`, `DESCRIPCION`, `IMAGEN_URL`)
VALUES
('iPhone 13 Pro', 'Apple', 999.99, 'Pantalla Super Retina XDR de 6.1 pulgadas', 'https://example.com/iphone13.jpg'),
('Galaxy S21', 'Samsung', 799.99, 'Cámara de 108 MP y pantalla Dynamic AMOLED', 'https://example.com/galaxys21.jpg'),
('Pixel 6 Pro', 'Google', 899.99, 'Chip Google Tensor, Android puro', 'https://example.com/pixel6.jpg'),
('Xiaomi Mi 11', 'Xiaomi', 749.99, 'Snapdragon 888, carga rápida de 55W', 'https://example.com/mi11.jpg'),
('OnePlus 9 Pro', 'OnePlus', 829.99, '120Hz Fluid Display, Hasselblad Camera', 'https://example.com/oneplus9.jpg'),
('Sony Xperia 1 III', 'Sony', 1099.99, 'Pantalla OLED 4K, Snapdragon 888', 'https://example.com/xperia1.jpg'),
('Huawei P50 Pro', 'Huawei', 1199.99, 'Cámara cuádruple Leica, HarmonyOS', 'https://example.com/p50pro.jpg'),
('Motorola Edge+', 'Motorola', 699.99, 'Pantalla OLED curva, batería de 5000mAh', 'https://example.com/edgeplus.jpg');

-- =====

-- TABLA DETALLE FACTURA

-- =====

DROP TABLE IF EXISTS `detalle_factura`;

CREATE TABLE `detalle_factura` (
  `ID_DETALLE` INT(11) NOT NULL AUTO_INCREMENT,

```

```

`ID_FACTURA` INT(11) NOT NULL,
`ID_TELEFONO` INT(11) NOT NULL,
`CANTIDAD` INT(11) NOT NULL,
`PRECIO_UNITARIO` DECIMAL(10,2) NOT NULL,
`SUBTOTAL` DECIMAL(10,2) NOT NULL,
PRIMARY KEY (`ID_DETALLE`),
FOREIGN KEY (`ID_FACTURA`) REFERENCES `factura` (`ID_FACTURA`) ON DELETE CASCADE ON UPDATE CASCADE,
FOREIGN KEY (`ID_TELEFONO`) REFERENCES `telefono` (`ID_TELEFONO`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;

-- =====
--          TABLA CARRITO
-- =====

DROP TABLE IF EXISTS `carrito`;

CREATE TABLE `carrito` (
  `ID_CARRITO` INT(11) NOT NULL AUTO_INCREMENT,
  `CLIENTE_CEDULA` VARCHAR(10) NOT NULL,
  `FECHA_CREACION` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP(),
  PRIMARY KEY (`ID_CARRITO`),

```

```

    FOREIGN KEY (`CLIENTE_CEDULA`) REFERENCES `cliente` (`CEDULA`) ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;

-- =====

--          TABLA CARRITO PRODUCTO

-- =====

DROP TABLE IF EXISTS `carrito_producto`;

CREATE TABLE `carrito_producto` (
    `ID_CARRITO_PRODUCTO` INT(11) NOT NULL AUTO_INCREMENT,
    `ID_CARRITO` INT(11) NOT NULL,
    `ID_TELEFONO` INT(11) NOT NULL,
    `CANTIDAD` INT(11) NOT NULL,
    `PRECIO_UNITARIO` DECIMAL(10,2) NOT NULL,
    `SUBTOTAL` DECIMAL(10,2) GENERATED ALWAYS AS (`CANTIDAD` * `PRECIO_UNITARIO`) VIRTUAL,
    PRIMARY KEY (`ID_CARRITO_PRODUCTO`),
    FOREIGN KEY (`ID_CARRITO`) REFERENCES `carrito` (`ID_CARRITO`) ON DELETE CASCADE,
    FOREIGN KEY (`ID_TELEFONO`) REFERENCES `telefono` (`ID_TELEFONO`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;

```

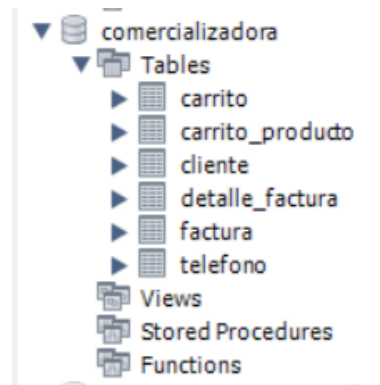


Figura 20 Tablas de la base de datos Comercializadora

APLICACIÓN SERVIDOR BANQUITO

CREACIÓN DEL PROYECTO SERVIDOR

El siguiente paso del proyecto es crear el servidor de la aplicación, recordando que este debe ser necesariamente web, además de que debe ser creado y guardado en la carpeta 01. SERVIDOR.

Se crea entonces un proyecto seleccionando la opción File > New Project.

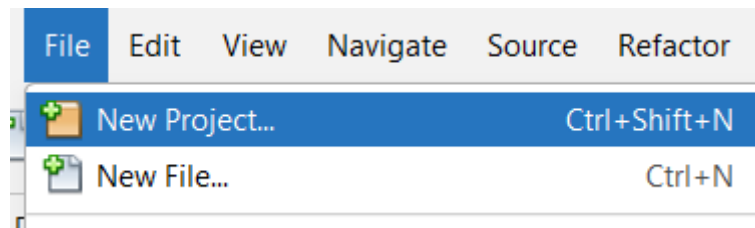


Figura 22 Nuevo proyecto

Posteriormente, seleccionar la opción Java with Maven > Web Application.

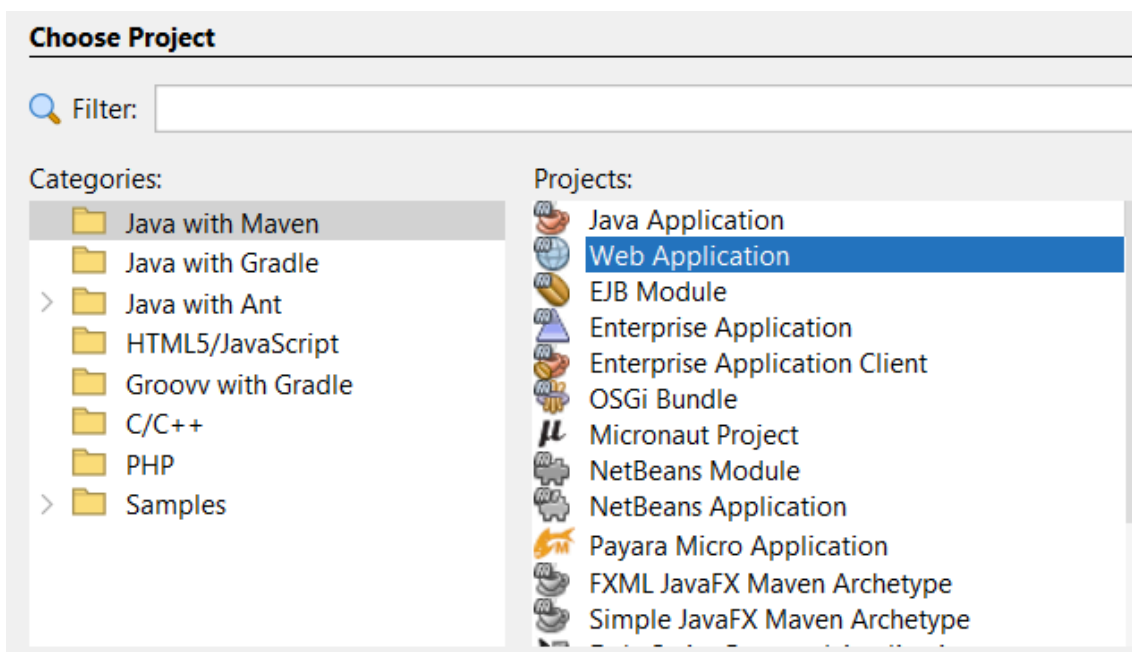


Figura 23 Aplicación tipo web

Se procede a configurar el nombre y la ubicación de la aplicación servidor.

New Web Application

Steps

1. Choose Project
- 2. Name and Location**
3. Settings

Name and Location

Project Name:

Project Location:

Project Folder:

Artifact Id:

Group Id:

Version:

Package: (Optional)

< Back **Next >** Finish Cancel Help

Figura 24 Asignación de nombre y ubicación

Luego ajustar las configuraciones del servidor.

New Web Application

Steps

1. Choose Project
2. Name and Location
- 3. Settings**

Settings

Server:

Java EE Version:

< Back Next > **Finish** Cancel Help

Figura 25 Configuración del servidor

Se verifica que todo salió bien cuando en el IDE se muestra el siguiente archivo.

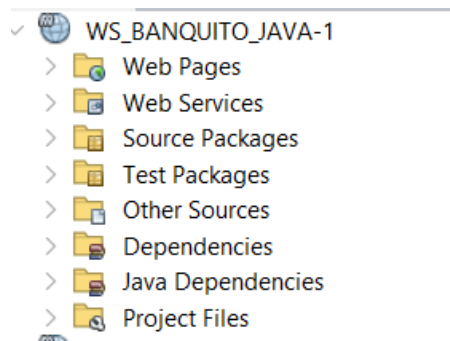


Figura 26 Página principal del proyecto

MODELO MVC

Como siguiente paso, se crean los paquetes necesarios para la aplicación basándose en MVC, los cuales son db, modelo, vista, controlador, prueba, servicio y ws.

Para la creación de un paquete se debe realizar lo siguiente, después todos los paquetes que se quieran crear siguen la misma lógica, iniciando por la selección de la carpeta Source Packages y presionando clic derecho para luego escoger la opción New > Java Package.

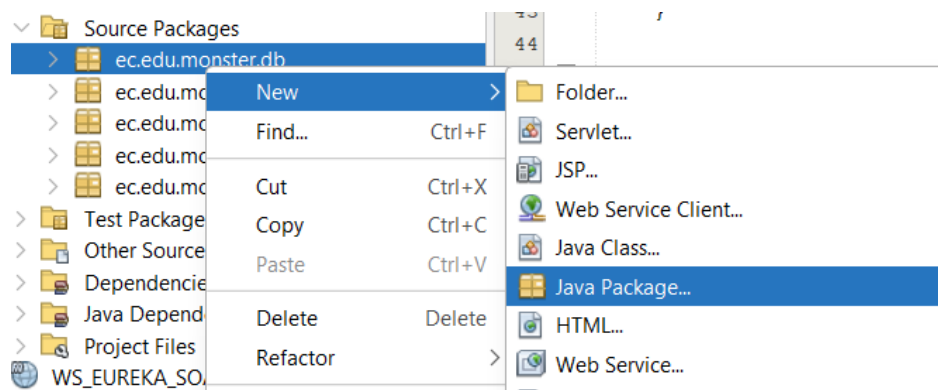


Figura 27 Creación de los paquetes

Dentro de la ventana de configuración se procede a configurar el nombre del paquete a crear que para efectos de la práctica seguirá el estándar ec.edu.monster.nombre.

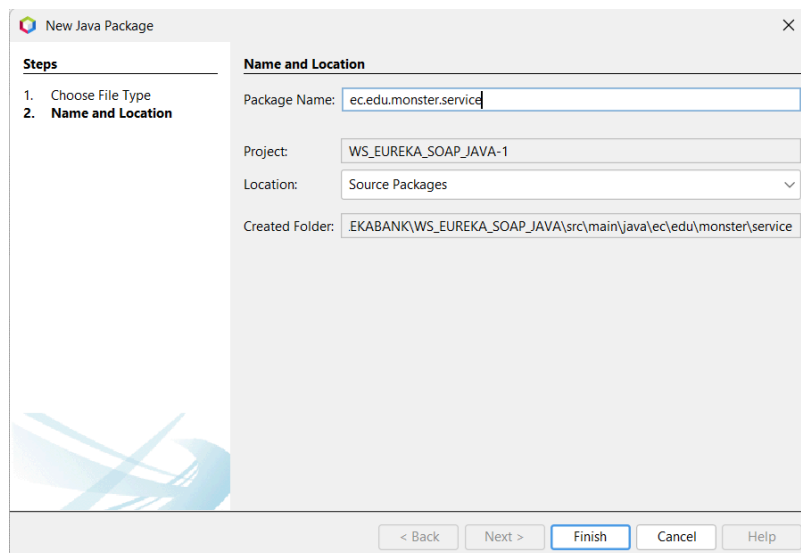


Figura 28 Denominación de los paquetes

Se verifica la creación del nuevo paquete observando la estructura del proyecto.

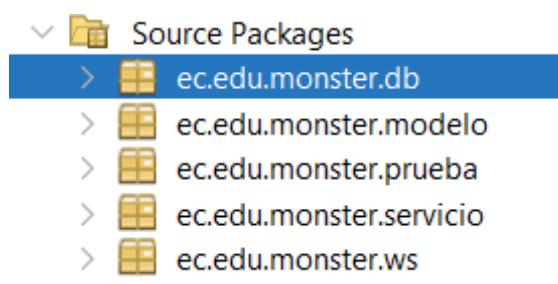


Figura 29 Verificación de la creación de paquetes

Para los demás paquetes se realiza lo mismo y al final debe quedar la siguiente estructura de paquetes.

CONEXIÓN A BASE DE DATOS

Una vez creado los paquetes necesarios, se procede a crear la conexión a la base de datos de la siguiente manera.

Primero, se selecciona el paquete db para luego dar clic derecho sobre el mismo y seleccionar New > Java Class.

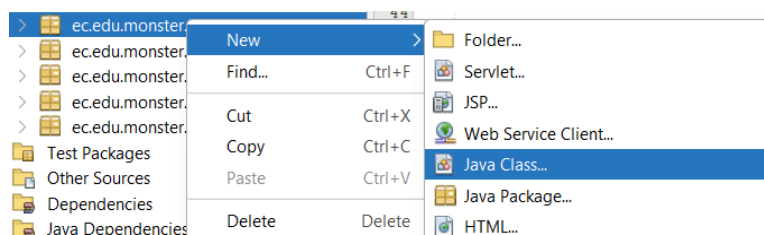


Figura 30 Creación de la clase AccesoDB

Dentro del menú de configuración de la nueva clase se coloca el nombre de AccesoDB para luego finalizar la creación.

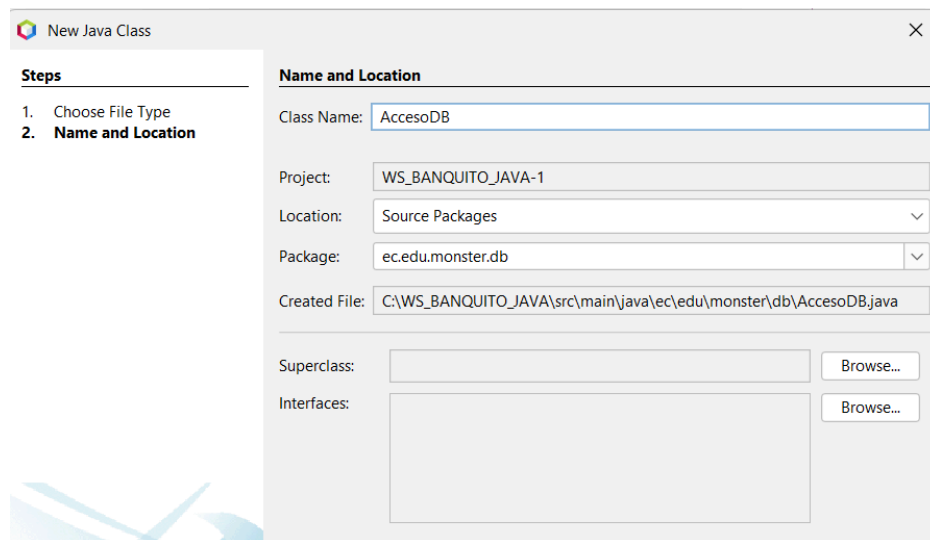


Figura 31 Configuración de la clase

Creada la clase se procede a codificar.

```
package ec.edu.monster.db;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

/**
 *
 * @author johnny
 */
public class AccesoDB {

    public static Connection getConnection() throws SQLException {
        Connection cn = null;
        try {
            //Datos MySQL
            Class.forName("com.mysql.cj.jdbc.Driver");
            cn = DriverManager.getConnection("jdbc:mysql://localhost:3306/banquito?useSSL=false", "root", "Jl0sd3512");
        } catch (SQLException e) {
            throw e;
        } catch (ClassNotFoundException e) {
            throw new SQLException("Error, no se encuentra el driver");
        } catch (Exception e) {
            throw new SQLException("Error, no se tiene acceso al servidor");
        }
        return cn;
    }
}
```

Figura 32 Codificación de la clase

Tabla 2 Clase Acceso Base de Datos

```
package ec.edu.monster.db;
```

```
import java.sql.Connection;
```

```

import java.sql.DriverManager;
import java.sql.SQLException;

/**
 *
 * @author johnny
 */
public class AccesoDB {

    public static Connection getConnection() throws SQLException {
        Connection cn = null;
        try {
            //Datos MySQL
            Class.forName("com.mysql.cj.jdbc.Driver");

            cn =
DriverManager.getConnection("jdbc:mysql://localhost:3306/banquito?useSSL=false", "root",
"Jlosd3512");
        } catch (SQLException e) {
            throw e;
        }
        catch(ClassNotFoundException e){
            throw new SQLException("Error, no se encuentra el driver");
        }
        catch(Exception e){
            throw new SQLException("Error, no se tiene acceso al servidor");
        }
        return cn;
    }
}

```

PRUEBA DE CONEXIÓN A BASE DE DATOS

A continuación, se procede a crear una nueva clase en el paquete de pruebas que se llama PruebaConexionBD.

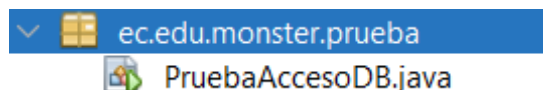


Figura 33 Creación de la clase de pruebas para la conexión

Luego se codifica la clase como si fuera una clase principal.

```

package ec.edu.monster.prueba;

import ec.edu.monster.db.AccesoDB;
import java.sql.Connection;

/**
 *
 * @author johnny
 */
public class PruebaAccesoDB {
    public static void main(String[] args) {
        try{
            try (Connection cn = AccesoDB.getConnection()) {
                System.out.println("Ok");
                cn.close();
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}

```

Figura 34 Codificación clase de prueba

Tabla 3 Clase de prueba de conexión

```

package ec.edu.monster.prueba;

import ec.edu.monster.db.AccesoDB;
import java.sql.Connection;

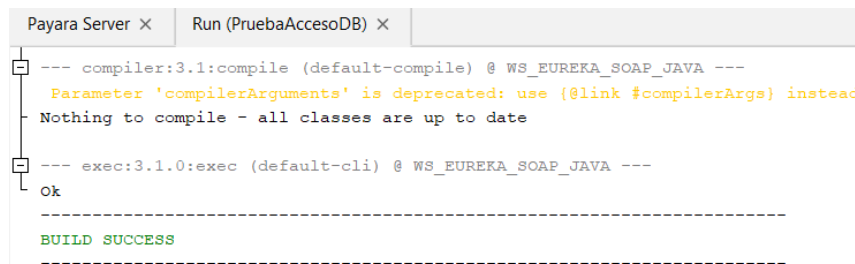
/**
 *
 * @author johnny
 */
public class PruebaAccesoDB {
    public static void main(String[] args){
        try{
            try (Connection cn = AccesoDB.getConnection()) {
                System.out.println("Ok");
                cn.close();
            }
        } catch (Exception e){
            e.printStackTrace();
        }
    }
}

```

}

Para comprobar la funcionalidad de la conexión a la base de datos se procede a ejecutar el archivo PruebaConexionBD.java, en caso de estar todo correcto se espera el mensaje Prueba de conexión exitosa, caso contrario la conexión ha fallado.

Una vez importado el conector de mysql, se vuelve a ejecutar la prueba de conexión con resultado diferente a la primera vez, obteniendo la respuesta mensaje Prueba de conexión exitosa.



```
--- compiler:3.1:compile (default-compile) @ WS_EUREKA_SOAP_JAVA ---
Parameter 'compilerArguments' is deprecated: use {@link #compilerArgs} instead.
Nothing to compile - all classes are up to date
--- exec:3.1.0:exec (default-cli) @ WS_EUREKA_SOAP_JAVA ---
Ok
BUILD SUCCESS
```

Figura 35 Resultado de la prueba de conexión a la base

CODIFICACIÓN DE CLASES

Habiendo obtenido la conexión se crean diferentes clases dentro del paquete modelo como Cliente, Crédito, Cuenta, Cuota, Movimiento y AmortizacionDTO.

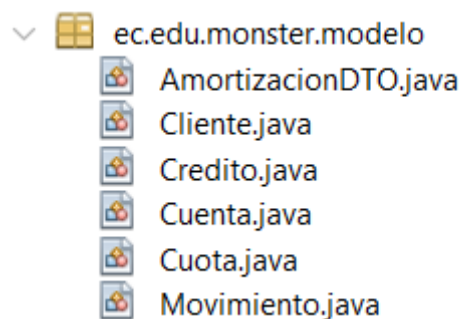


Figura 36 Creación de modelos

Se procede a codificar las nuevas clases éstas se pueden ver en la Tabla 5, creadas las clases, se procede a implementar los servicios, para ello se crea una nueva clase llamada BanQuitoService dentro del paquete servicio.

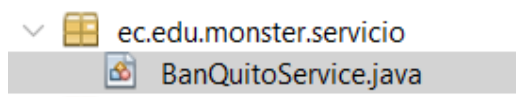


Figura 37 Creación de la clase BanQuitoService

Tabla 5 Codificación clase Movimiento, Cliente, Crédito, Cuenta, Cuota

```
//CLIENTE

package ec.edu.monster.modelo;

import java.util.Date;

/**
 *
 * @author jhonn
 */

public class Cliente {

    private int codCliente;

    private Integer codCredito;

    private String cedula;

    private String nombre;

    private String genero;

    private Date fechaNacimiento;

    // Getters y Setters

    public int getCodCliente() {

        return codCliente;

    }

}
```

```
public void setCodCliente(int codCliente) {  
  
    this.codCliente = codCliente;  
  
}
```

```
public Integer getCodCredito() {  
  
    return codCredito;  
  
}
```

```
public void setCodCredito(Integer codCredito) {  
  
    this.codCredito = codCredito;  
  
}
```

```
public String getCedula() {  
  
    return cedula;  
  
}
```

```
public void setCedula(String cedula) {  
  
    this.cedula = cedula;  
  
}
```

```
public String getNombre() {  
  
    return nombre;  
  
}
```

```
}
```

```
public void setNombre(String nombre) {
```

```
    this.nombre = nombre;
```

```
}
```

```
public String getGenero() {
```

```
    return genero;
```

```
}
```

```
public void setGenero(String genero) {
```

```
    this.genero = genero;
```

```
}
```

```
public Date getFechaNacimiento() {
```

```
    return fechaNacimiento;
```

```
}
```

```
public void setFechaNacimiento(Date fechaNacimiento) {
```

```
    this.fechaNacimiento = fechaNacimiento;
```

```
}
```

```
}
```

```
//CUOTA

package ec.edu.monster.modelo;

import java.util.Date;

/**
 *
 * @author jhonn
 */
public class Cuota {

    private int codCuota;

    private Integer codCredito;

    private Double cuotaValor;

    private Date fecha;

    // Getters y Setters

    public int getCodCuota() {

        return codCuota;

    }

    public void setCodCuota(int codCuota) {

        this.codCuota = codCuota;

    }

}
```

```
}
```

```
public Integer getCodCredito() {
```

```
    return codCredito;
```

```
}
```

```
public void setCodCredito(Integer codCredito) {
```

```
    this.codCredito = codCredito;
```

```
}
```

```
public Double getCuotaValor() {
```

```
    return cuotaValor;
```

```
}
```

```
public void setCuotaValor(Double cuotaValor) {
```

```
    this.cuotaValor = cuotaValor;
```

```
}
```

```
public Date getFecha() {
```

```
    return fecha;
```

```
}
```

```

        public void setFecha(Date fecha) {

            this.fecha = fecha;

        }

    }

//CUENTA

package ec.edu.monster.modelo;

/**
 *
 * @author jhonn
 */

public class Cuenta {

    private String numCuenta;

    private int codCliente;

    private Double saldo;

    // Getters y Setters

    public String getNumCuenta() {

        return numCuenta;

    }

    public void setNumCuenta(String numCuenta) {

        this.numCuenta = numCuenta;
    }

```

```

    }

    public int getCodCliente() {

        return codCliente;

    }

    public void setCodCliente(int codCliente) {

        this.codCliente = codCliente;

    }

    public Double getSaldo() {

        return saldo;

    }

    public void setSaldo(Double saldo) {

        this.saldo = saldo;

    }

}

//CREDITO

package ec.edu.monster.modelo;

import java.util.Date;

/**
 *
 * @author jhonn
 */

public class Credito {

```

```
private int codCredito;

private int codCliente;

private Double saldo;

private Date fecha;

private Integer numCuotas;


// Getters y Setters

public int getCodCredito() {

    return codCredito;

}


public void setCodCredito(int codCredito) {

    this.codCredito = codCredito;

}


public int getCodCliente() {

    return codCliente;

}


public void setCodCliente(int codCliente) {

    this.codCliente = codCliente;

}
```



```
}
```

```
public Double getSaldo() {
```

```
    return saldo;
```

```
}
```

```
public void setSaldo(Double saldo) {
```

```
    this.saldo = saldo;
```

```
}
```

```
public Date getFecha() {
```

```
    return fecha;
```

```
}
```

```
public void setFecha(Date fecha) {
```

```
    this.fecha = fecha;
```

```
}
```

```
public Integer getNumCuotas() {
```

```
    return numCuotas;
```

```
}
```

```

        public void setNumCuotas(Integer numCuotas) {

            this.numCuotas = numCuotas;

        }

    }

//MOVIMIENTO

package ec.edu.monster.modelo;

import java.util.Date;

/**
 *
 * @author jhonn
 */

public class Movimiento {

    private int codMovimiento;

    private String numCuenta;

    private String tipo;

    private Double valor;

    private Date fecha;

    // Getters y Setters

    public int getCodMovimiento() {

        return codMovimiento;

    }

    public void setCodMovimiento(int codMovimiento) {

```

```
        this.codMovimiento = codMovimiento;

    }

    public String getNumCuenta() {

        return numCuenta;

    }

    public void setNumCuenta(String numCuenta) {

        this.numCuenta = numCuenta;

    }

    public String getTipo() {

        return tipo;

    }

    public void setTipo(String tipo) {

        this.tipo = tipo;

    }

    public Double getValor() {

        return valor;

    }

    public void setValor(Double valor) {

        this.valor = valor;

    }

    public Date getFecha() {

        return fecha;

    }

}
```

```

    }

    public void setFecha(Date fecha) {

        this.fecha = fecha;

    }

}

```

Tabla 5 Codificación Clase BanQuitoService

```

package ec.edu.monster.servicio;

import ec.edu.monster.db.AccesoDB;

import ec.edu.monster.modelo.AmortizacionDTO;

import ec.edu.monster.modelo.Movimiento;

import java.sql.Connection;

import java.sql.PreparedStatement;

import java.sql.ResultSet;

import java.sql.SQLException;

import java.util.ArrayList;

import java.util.List;

/**
 * Servicio centralizado para gestionar operaciones de BanQuito.
 */

public class BanQuitoService {

```

```

// ----- Métodos relacionados con Amortización -----

public List<AmortizacionDTO> obtenerAmortizacion(int codCredito) {

    Connection cn = null;

    List<AmortizacionDTO> lista = new ArrayList<>();

    String sql = "SELECT NUMERO_CUOTA, VALOR_CUOTA, INTERES_PAGADO, CAPITAL_PAGADO,
SALDO_RESTANTE "

        + "FROM CUOTA WHERE COD_CREDITO = ? ORDER BY NUMERO_CUOTA";

    try {

        cn = AccesoDB.getConnection();

        PreparedStatement pstmt = cn.prepareStatement(sql);

        pstmt.setInt(1, codCredito);

        ResultSet rs = pstmt.executeQuery();

        while (rs.next()) {

            AmortizacionDTO dto = new AmortizacionDTO();

            dto.setNumeroCuota(rs.getInt("NUMERO_CUOTA"));

            dto.setValorCuota(rs.getDouble("VALOR_CUOTA"));

            dto.setInteresPagado(rs.getDouble("INTERES_PAGADO"));

            dto.setCapitalPagado(rs.getDouble("CAPITAL_PAGADO"));

            dto.setSaldo(rs.getDouble("SALDO_RESTANTE"));

            lista.add(dto);

        }

        rs.close();
    }
}

```

```

    } catch (SQLException e) {

        throw new RuntimeException("Error en la consulta de amortización: " + e.getMessage(), e);

    } finally {

        try {

            if (cn != null) {

                cn.close();

            }

        } catch (Exception e) {

            throw new RuntimeException("Error al cerrar la conexión: " + e.getMessage(), e);

        }

    }

    return lista;

}

```

// ----- Métodos relacionados con Créditos -----

```

public boolean validarCredito(String cedula) {

    Connection cn = null;

    boolean esValido = false;

    String query = "SELECT COUNT(*) FROM CLIENTE WHERE CEDULA = ? AND NOT EXISTS ("

        + "SELECT 1 FROM CREDITO WHERE COD_CLIENTE = CLIENTE.COD_CLIENTE)";

    try {

        cn = AccesoDB.getConnection();
    }

```

```

PreparedStatement cmd = cn.prepareStatement(query);

cmd.setString(1, cedula);

ResultSet rs = cmd.executeQuery();


if (rs.next()) {

    esValido = rs.getInt(1) > 0;

}


rs.close();

cmd.close();

} catch (SQLException e) {

    throw new RuntimeException("Error validando crédito: " + e.getMessage(), e);

} finally {

    try {

        if (cn != null) {

            cn.close();

        }

    } catch (Exception e) {

        throw new RuntimeException("Error al cerrar la conexión: " + e.getMessage(), e);

    }

}

return esValido;

}

```

```

public void otorgarCredito(int codCliente, double valorTelefono, int numeroCuotas) {

    Connection cn = null;

    if (numeroCuotas < 3 || numeroCuotas > 18) {

        throw new IllegalArgumentException("El número de cuotas debe estar entre 3 y 18.");

    }

    try {

        cn = AccesoDB.getConnection();

        cn.setAutoCommit(false);

        // Calcular la cuota fija

        double tasaAnual = 16.5 / 100; // Tasa anual

        double tasaPeriodo = tasaAnual / 12; // Tasa mensual

        double cuota = (valorTelefono * tasaPeriodo) /

            (1 - Math.pow(1 + tasaPeriodo, -numeroCuotas));

        // Insertar crédito

        String insertCredito = "INSERT INTO CREDITO (COD_CLIENTE, SALDO, FECHA, NUM_CUOTAS) "

            + "VALUES (?, ?, NOW(), ?)";

        PreparedStatement cmdCredito = cn.prepareStatement(insertCredito,
PreparedStatement.RETURN_GENERATED_KEYS);

```



```

cmdCredito.setInt(1, codCliente);

cmdCredito.setDouble(2, valorTelefono);

cmdCredito.setInt(3, numeroCuotas);

cmdCredito.executeUpdate();


ResultSet rs = cmdCredito.getGeneratedKeys();

int codCredito = 0;

if (rs.next()) {

    codCredito = rs.getInt(1);

}


// Generar tabla de amortización

generarTablaAmortizacion(codCredito, valorTelefono, cuota, numeroCuotas, tasaPeriodo, cn);


cn.commit();

} catch (SQLException e) {

    try {

        if (cn != null) {

            cn.rollback();

        }

    } catch (SQLException ex) {

        throw new RuntimeException("Error haciendo rollback: " + ex.getMessage(), ex);

    }

    throw new RuntimeException("Error otorgando crédito: " + e.getMessage(), e);

```

```

    } finally {

        try {

            if (cn != null) {

                cn.close();

            }

        } catch (SQLException e) {

            throw new RuntimeException("Error al cerrar la conexión: " + e.getMessage(), e);

        }

    }

}

```

```

    private void generarTablaAmortizacion(int codCredito, double saldoInicial, double cuota, int
numeroCuotas, double tasaPeriodo, Connection cn) throws SQLException {

```

```

        double saldo = saldoInicial;

```

```

        for (int numeroCuota = 1; numeroCuota <= numeroCuotas; numeroCuota++) {

```

```

            double interes = saldo * tasaPeriodo;

```

```

            double capital = cuota - interes;

```

```

            saldo -= capital;

```

```

            String insertCuota = "INSERT INTO CUOTA (COD_CREDITO, NUMERO_CUOTA, VALOR_CUOTA,
INTERES_PAGADO, CAPITAL_PAGADO, SALDO_REstante, FECHA) "

```

```

                + "VALUES (?, ?, ?, ?, ?, DATE_ADD(NOW(), INTERVAL ? MONTH))";

```

```

            PreparedStatement cmdCuota = cn.prepareStatement(insertCuota);

```

```

        cmdCuota.setInt(1, codCredito);

        cmdCuota.setInt(2, numeroCuota);

        cmdCuota.setDouble(3, cuota);

        cmdCuota.setDouble(4, interes);

        cmdCuota.setDouble(5, capital);

        cmdCuota.setDouble(6, Math.max(0, saldo));

        cmdCuota.setInt(7, numeroCuota);

        cmdCuota.executeUpdate();

    }

}

// ----- Métodos relacionados con Monto Máximo -----

public double calcularMaxCredito(int codCliente) {

    Connection cn = null;

    double montoMaximo = 0;

    try {

        cn = AccesoDB.getConnection();

        double promedioDepositos = obtenerPromedioMovimientos(codCliente, "DEPOSITO", cn);

        double promedioRetiros = obtenerPromedioMovimientos(codCliente, "RETIRO", cn);

        montoMaximo = ((promedioDepositos - promedioRetiros) * 0.35) * 6;
    }
}

```

```

        if (montoMaximo <= 0) {

            throw new RuntimeException("El cliente no tiene movimientos suficientes para calcular el
crédito.");

        }

    } catch (SQLException e) {

        throw new RuntimeException("Error calculando el monto máximo de crédito: " + e.getMessage(),
e);

    } finally {

        try {

            if (cn != null) {

                cn.close();

            }

        } catch (SQLException e) {

            throw new RuntimeException("Error al cerrar la conexión: " + e.getMessage(), e);

        }

    }

    return montoMaximo;

}

private double obtenerPromedioMovimientos(int codCliente, String tipoMovimiento, Connection cn)
throws SQLException {

    double promedio = 0;

    String query = "SELECT AVG(VAOR) FROM MOVIMIENTO WHERE NUM_CUENTA IN ("

        + "SELECT NUM_CUENTA FROM CUENTA WHERE COD_CLIENTE = ?) AND TIPO = ? AND
FECHA > DATE_SUB(NOW(), INTERVAL 3 MONTH)";

    PreparedStatement cmd = cn.prepareStatement(query);

    cmd.setInt(1, codCliente);

```

```

cmd.setString(2, tipoMovimiento);

ResultSet rs = cmd.executeQuery();

if (rs.next()) {

    promedio = rs.getDouble(1);

}

rs.close();

cmd.close();

return promedio;

}

}

```

CREACIÓN DEL WEB SERVICE

Una vez creados los servicios y modelos se procede con la creación del Web Service el cual contiene las operaciones que estarán expuestas y se consumen por otros servidores o clientes.

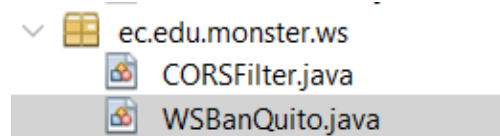


Figura 38 Creación de la clase WSBanQuito.

Se agrega un nuevo web service.

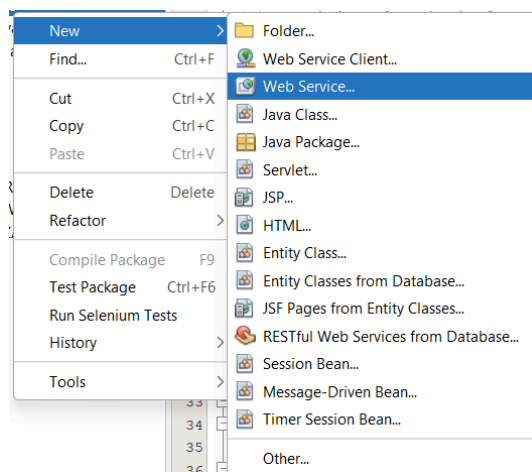


Figura 38 Creación de la clase WSBanQuito.

Se agregan las operaciones correspondientes con los requerimientos establecidos en el servicio bancario.

```
package ec.edu.monster.ws;

import ec.edu.monster.modelo.AmortizacionDTO;
import ec.edu.monster.servicio.BanQuitoService;
import java.util.List;
import javax.jws.WebMethod;
import javax.jws.WebParam;
import javax.jws.WebResult;
import javax.jws.WebService;

/**
 * Servicio web principal para BanQuito
 * Proporciona operaciones relacionadas con créditos y amortización.
 */
@WebService(serviceName = "WSBanQuito")
public class WSBanQuito {

    private final BanQuitoService service = new BanQuitoService();

    /**
     * Operación para obtener la tabla de amortización.
     * @param codCredito Código del crédito
     * @return Lista de amortización
     */
    @WebMethod(operationName = "obtenerAmortizacion")
    @WebResult(name = "amortizacion")
    public List<AmortizacionDTO> obtenerAmortizacion(@WebParam(name = "codCredito") int codCredito) {
        try {
            return service.obtenerAmortizacion(codCredito);
        } catch (Exception e) {
            throw new RuntimeException("Error al obtener la tabla de amortización: " + e.getMessage(), e);
        }
    }
}
```

Figura 39 WSBanQuito codificación

Tabla 6 Codificación clase WSBanQuito

```
package ec.edu.monster.ws;

import ec.edu.monster.modelo.AmortizacionDTO;

import ec.edu.monster.servicio.BanQuitoService;

import java.util.List;

import javax.jws.WebMethod;

import javax.jws.WebParam;

import javax.jws.WebResult;

import javax.jws.WebService;

/**

 * Servicio web principal para BanQuito

 * Proporciona operaciones relacionadas con créditos y amortización.
```

```

*/

@WebService(serviceName = "WSBanQuito")

public class WSBanQuito {

    private final BanQuitoService service = new BanQuitoService();

    /**

    * Operación para obtener la tabla de amortización.

    * @param codCredito Código del crédito

    * @return Lista de amortización

    */

    @WebMethod(operationName = "obtenerAmortizacion")

    @WebResult(name = "amortizacion")

    public List<AmortizacionDTO> obtenerAmortizacion(@WebParam(name = "codCredito") int
codCredito) {

        try {

            return service.obtenerAmortizacion(codCredito);

        } catch (Exception e) {

            throw new RuntimeException("Error al obtener la tabla de amortización: " + e.getMessage(), e);

        }

    }

    /**

    * Operación para validar si un cliente es sujeto de crédito.

    * @param cedula Cédula del cliente

    * @return Estado de validación (true si es sujeto de crédito, false en caso contrario)

```

```

*/

@WebMethod(operationName = "validarCredito")

@WebResult(name = "esSujetoCredito")

public boolean validarCredito(@WebParam(name = "cedula") String cedula) {

    try {

        return service.validarCredito(cedula);

    } catch (Exception e) {

        throw new RuntimeException("Error al validar crédito: " + e.getMessage(), e);

    }

}

/**

* Operación para otorgar un crédito.

* @param codCliente Código del cliente

* @param valorTelefono Valor del crédito

* @param numeroCuotas Número de cuotas

* @return Mensaje de éxito o error

*/

@WebMethod(operationName = "otorgarCredito")

@WebResult(name = "mensaje")

public String otorgarCredito(@WebParam(name = "codCliente") int codCliente,

                             @WebParam(name = "valorTelefono") double valorTelefono,

                             @WebParam(name = "numeroCuotas") int numeroCuotas) {

    try {

        service.otorgarCredito(codCliente, valorTelefono, numeroCuotas);

```



```

        return "Crédito otorgado exitosamente.";

    } catch (Exception e) {

        throw new RuntimeException("Error al otorgar crédito: " + e.getMessage(), e);

    }

}

/**

 * Operación para calcular el monto máximo de crédito de un cliente.

 * @param codCliente Código del cliente

 * @return Monto máximo de crédito

 */

@WebMethod(operationName = "calcularMaxCredito")

@WebResult(name = "montoMaximo")

public double calcularMaxCredito(@WebParam(name = "codCliente") int codCliente) {

    try {

        return service.calcularMaxCredito(codCliente);

    } catch (Exception e) {

        throw new RuntimeException("Error al calcular el monto máximo de crédito: " + e.getMessage(),
e);

    }

}

}

```

Ahora se ejecuta el servidor y verificamos que el wsdl se genere correctamente con el código XML.

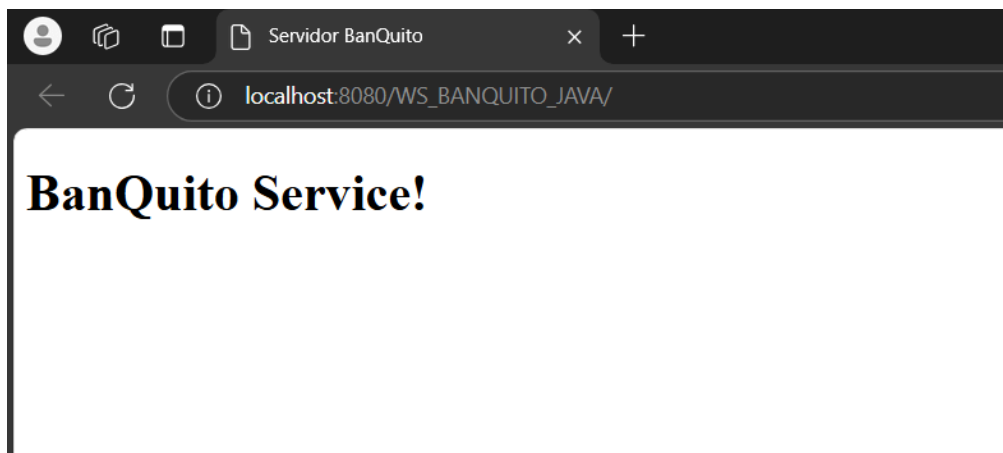


Figura 40 Ejecución del servidor BanQuito

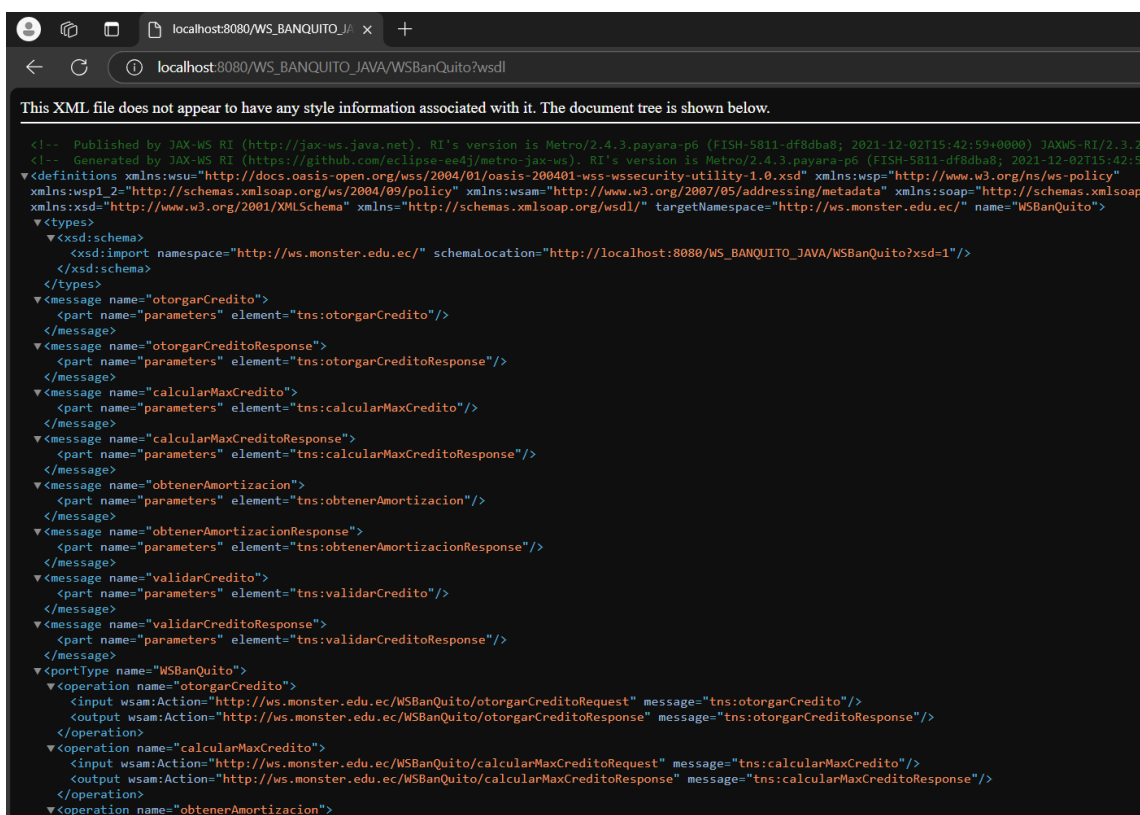


Figura 41 wsdl del servidor BanQuito

SERVIDOR COMERCIALIZADORA

CREACIÓN DEL PROYECTO SERVIDOR

El siguiente paso del proyecto es crear el servidor de la aplicación comercializadora, recordando que este debe ser necesariamente web, además de que debe ser creado y guardado en la carpeta 01. SERVIDOR.

Se crea entonces un proyecto seleccionando la opción File > New Project.

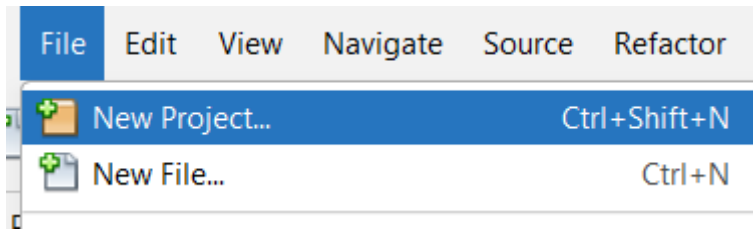


Figura 42 Nuevo proyecto

Posteriormente, seleccionar la opción Java with Maven > Web Application.

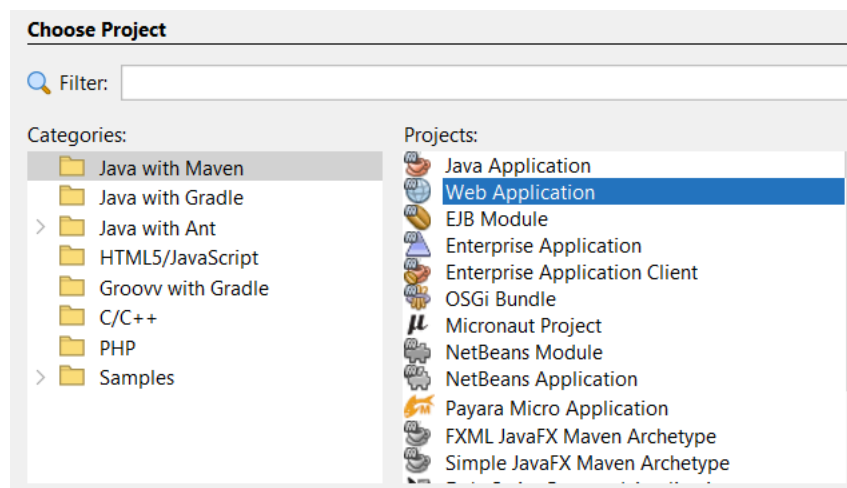


Figura 43 Aplicación tipo web

Se procede a configurar el nombre y la ubicación de la aplicación servidor.

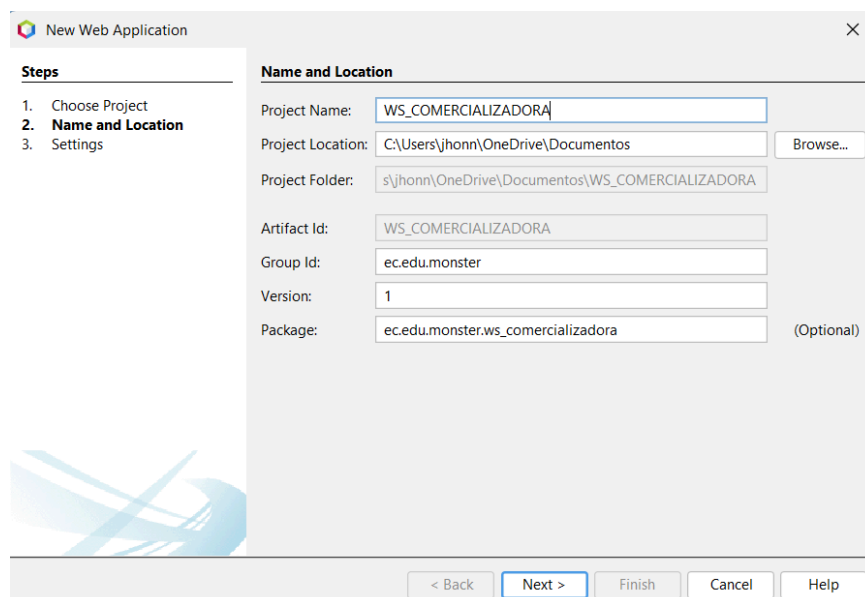


Figura 44 Asignación de nombre y ubicación

Luego ajustar las configuraciones del servidor.

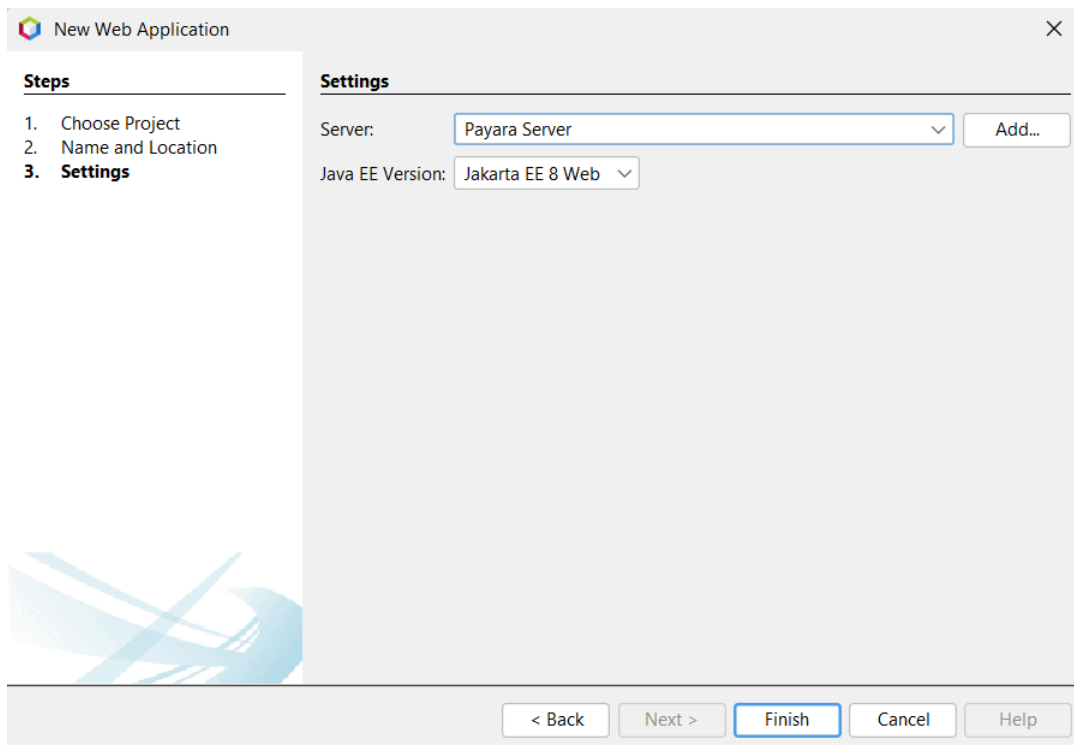


Figura 45 Configuración del servidor

Se verifica que todo salió bien cuando en el IDE se muestra el siguiente archivo.

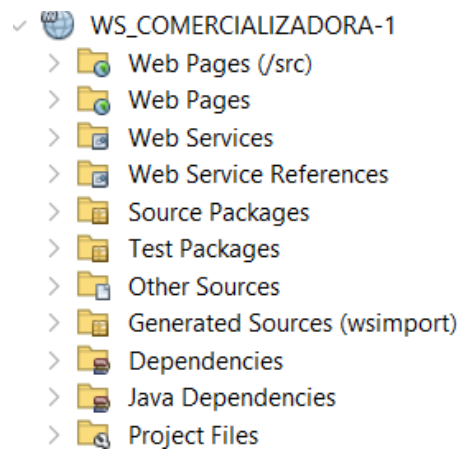


Figura 46 Página principal del proyecto

MODELO MVC

Como siguiente paso, se crean los paquetes necesarios para la aplicación basándose en MVC, los cuales son db, modelo, vista, controlador, prueba, servicio y ws.

Para la creación de un paquete se debe realizar lo siguiente, después todos los paquetes que se quieran crear siguen la misma lógica, iniciando por la selección de la carpeta Source Packages y presionando clic derecho para luego escoger la opción New > Java Package.

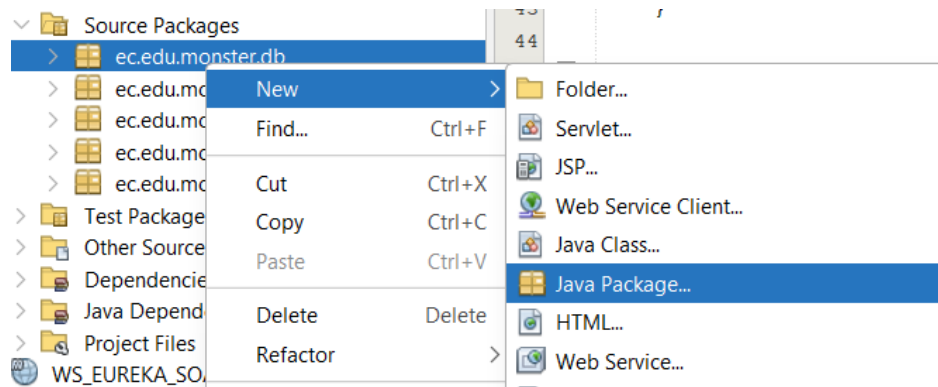


Figura 47 Creación de los paquetes

Dentro de la ventana de configuración se procede a configurar el nombre del paquete a crear que para efectos de la práctica seguirá el estándar ec.edu.monster.nombre.

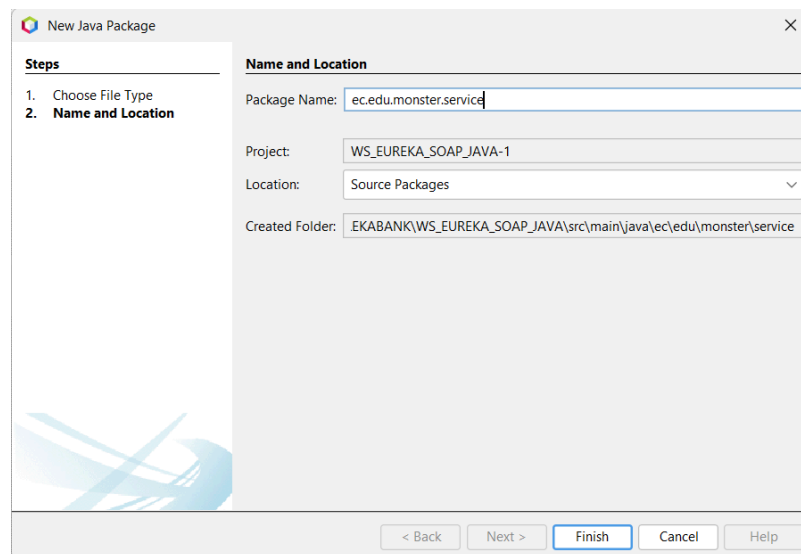


Figura 48 Denominación de los paquetes

Se verifica la creación del nuevo paquete observando la estructura del proyecto.

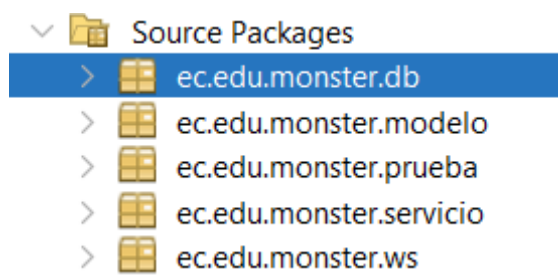


Figura 49 Verificación de la creación de paquetes

Para los demás paquetes se realiza lo mismo y al final debe quedar la siguiente estructura de paquetes.

CONEXIÓN A BASE DE DATOS

Una vez creado los paquetes necesarios, se procede a crear la conexión a la base de datos de la siguiente manera.

Primero, se selecciona el paquete db para luego dar clic derecho sobre el mismo y seleccionar New > Java Class.

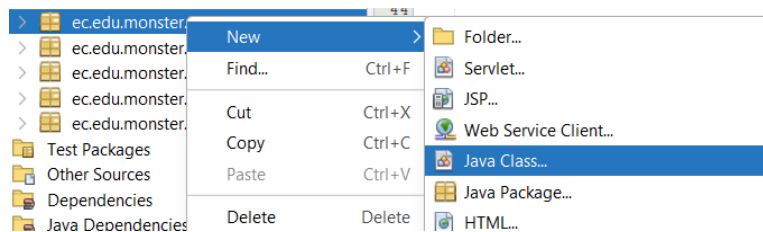


Figura 50 Creación de la clase AccesoDB

Dentro del menú de configuración de la nueva clase se coloca el nombre de AccesoDB para luego finalizar la creación.

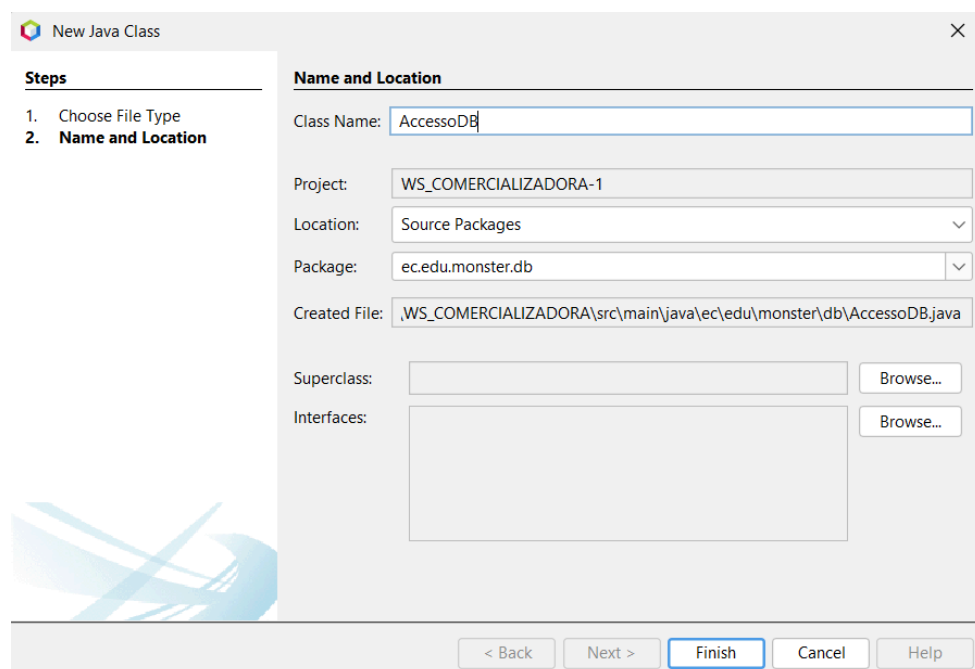


Figura 51 Configuración de la clase

Creada la clase se procede a codificar.

```

package ec.edu.monster.db;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

/**
 *
 * @author johnny
 */
public class AccesoDB {

    public static Connection getConnection() throws SQLException {
        Connection cn = null;
        try {
            //Datos MySQL
            Class.forName("com.mysql.cj.jdbc.Driver");
            cn = DriverManager.getConnection("jdbc:mysql://localhost:3306/banquito?useSSL=false", "root", "Jl0sd3512");
        } catch (SQLException e) {
            throw e;
        }
        catch(ClassNotFoundException e){
            throw new SQLException("Error, no se encuentra el driver");
        }
        catch(Exception e){
            throw new SQLException("Error, no se tiene acceso al servidor");
        }
        return cn;
    }
}

```

Figura 52 Codificación de la clase

Tabla 2 Clase Acceso Base de Datos

```
package ec.edu.monster.db;
```

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
```

```
/**
```

```
*
```

```
* @author johnny
```

```
*/
```

```
public class AccesoDB {
```

```
    public static Connection getConnection() throws SQLException {
```

```
        Connection cn = null;
```

```
        try {
```

```
            //Datos MySQL
```

```
            Class.forName("com.mysql.cj.jdbc.Driver");
```

```

                                                    cn      =
DriverManager.getConnection("jdbc:mysql://localhost:3306/banquito?useSSL=false", "root",
"Jl0sd3512");
```

```
        } catch (SQLException e) {
```

```
            throw e;
```

```
        }
```

```

catch(ClassNotFoundException e){
    throw new SQLException("Error, no se encuentra el driver");
}
catch(Exception e){
    throw new SQLException("Error, no se tiene acceso al servidor");
}
return cn;
}
}

```

PRUEBA DE CONEXIÓN A BASE DE DATOS

A continuación, se procede a crear una nueva clase en el paquete de pruebas que se llama PruebaConexionBD.

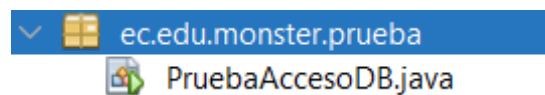


Figura 53 Creación de la clase de pruebas para la conexión

Luego se codifica la clase como si fuera una clase principal.

```

package ec.edu.monster.prueba;

import ec.edu.monster.db.AccesoDB;
import java.sql.Connection;

/**
 *
 * @author johnny
 */
public class PruebaAccesoDB {
    public static void main(String[] args) {
        try{
            try (Connection cn = AccesoDB.getConnection()) {
                System.out.println("Ok");
                cn.close();
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}

```

Figura 54 Codificación clase de prueba

Tabla 3 Clase de prueba de conexión

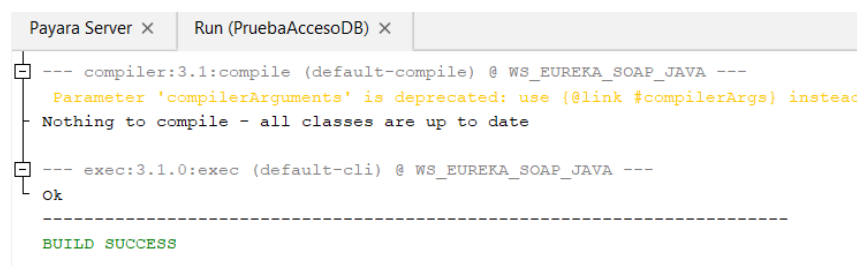
```
package ec.edu.monster.prueba;

import ec.edu.monster.db.AccesoDB;
import java.sql.Connection;

/**
 *
 * @author johnny
 */
public class PruebaAccesoDB {
    public static void main(String[] args){
        try{
            try (Connection cn = AccesoDB.getConnection()) {
                System.out.println("Ok");
                cn.close();
            }
        }catch(Exception e){
            e.printStackTrace();
        }
    }
}
```

Para comprobar la funcionalidad de la conexión a la base de datos se procede a ejecutar el archivo PruebaConexionBD.java, en caso de estar todo correcto se espera el mensaje Prueba de conexión exitosa, caso contrario la conexión ha fallado.

Una vez importado el conector de mysql, se vuelve a ejecutar la prueba de conexión con resultado diferente a la primera vez, obteniendo la respuesta mensaje Prueba de conexión exitosa.



```
Payara Server × Run (PruebaAccesoDB) ×
--- compiler:3.1:compile (default-compile) @ WS_EUREKA_SOAP_JAVA ---
Parameter 'compilerArguments' is deprecated: use {@link #compilerArgs} instead.
Nothing to compile - all classes are up to date
--- exec:3.1.0:exec (default-cli) @ WS_EUREKA_SOAP_JAVA ---
Ok
-----
BUILD SUCCESS
-----
```

Figura 55 Resultado de la prueba de conexión a la base

CODIFICACIÓN DE CLASES

Habiendo obtenido la conexión se crean diferentes clases dentro del paquete modelo como Telefono, Factura, FacturaDTO y AmortizacionDTO.

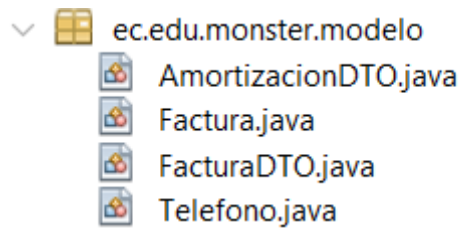


Figura 56 Creación de modelos

Se procede a codificar las nuevas clases éstas se pueden ver en la Tabla 5, creadas las clases, se procede a implementar los servicios, para ello se crea una nueva clase llamada BanQuitoService dentro del paquete servicio.

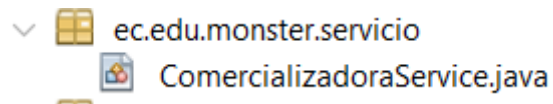


Figura 57 Creación de la clase ComercializadoraService

Tabla 5 Codificación clase Telefono, Factura, FacturaDTO y AmortizacionDTO

```
package ec.edu.monster.modelo;
```

```
import java.util.Date;
```

```
/**
```

```
*
```

```
* @author jhonn
```

```
*/
```

```
public class Factura {
```

```
    private int idFactura;
```

```
    private Date fecha;
```

```
    private String clienteNombre;
```

```
    private String clienteCedula;
```

```
private String formaPago;

private double total;

private double descuento;


// Getters y Setters

public int getIdFactura() {

    return idFactura;

}


public void setIdFactura(int idFactura) {

    this.idFactura = idFactura;

}


public Date getFecha() {

    return fecha;

}


public void setFecha(Date fecha) {

    this.fecha = fecha;

}


public String getClienteNombre() {

    return clienteNombre;

}


public void setClienteNombre(String clienteNombre) {
```

```
        this.clienteNombre = clienteNombre;
    }

    public String getClienteCedula() {

        return clienteCedula;
    }

    public void setClienteCedula(String clienteCedula) {

        this.clienteCedula = clienteCedula;
    }

    public String getFormaPago() {

        return formaPago;
    }

    public void setFormaPago(String formaPago) {

        this.formaPago = formaPago;
    }

    public double getTotal() {

        return total;
    }

    public void setTotal(double total) {

        this.total = total;
    }

    public double getDescuento() {

        return descuento;
    }
}
```

```

    }

    public void setDescuento(double descuento) {

        this.descuento = descuento;

    }

}

```

Tabla 5 Codificación Clase ComercializadoraService

```

package ec.edu.monster.servicio;

import ec.edu.monster.db.AccesoDB;

import ec.edu.monster.modelo.AmortizacionDTO;

import ec.edu.monster.modelo.FacturaDTO;

import java.sql.Connection;

import java.sql.PreparedStatement;

import java.sql.ResultSet;

import java.util.ArrayList;

import java.util.List;

/**
 * Servicio combinado para manejar facturación y compras.
 */

```

```

public class ComercializadoraService {

    /**
     * Registra una factura en efectivo y calcula el descuento.
     *
     * @param clienteNombre Nombre del cliente.
     * @param clienteCedula Cédula del cliente.
     * @param precio Precio del producto.
     * @return Detalles de la factura registrada.
     * @throws Exception En caso de error en la base de datos.
     */

    public FacturaDTO facturarEfectivo(String clienteNombre, String clienteCedula, double precio) throws
    Exception {

        double descuento = precio * 0.42; // 42% de descuento

        double total = precio - descuento;

        FacturaDTO factura = null;

        String queryInsert = "INSERT INTO FACTURA (FECHA, CLIENTE_NOMBRE, CLIENTE_CEDULA,
        FORMA_PAGO, TOTAL, DESCUENTO) "

            + "VALUES (NOW(), ?, ?, 'Efectivo', ?, ?);";

        String querySelect = "SELECT ID_FACTURA, FECHA, CLIENTE_NOMBRE, CLIENTE_CEDULA,
        FORMA_PAGO, TOTAL, DESCUENTO "

            + "FROM FACTURA WHERE ID_FACTURA = ?;";

        try (Connection cn = AccesoDB.getConnection());

```

```

        PreparedStatement psInsert = cn.prepareStatement(queryInsert,
PreparedStatement.RETURN_GENERATED_KEYS) {

    // Insertar la factura

    psInsert.setString(1, clienteNombre);

    psInsert.setString(2, clienteCedula);

    psInsert.setDouble(3, total);

    psInsert.setDouble(4, descuento);

    psInsert.executeUpdate();

    // Obtener el ID generado

    try (ResultSet rs = psInsert.getGeneratedKeys()) {

        if (rs.next()) {

            int facturalId = rs.getInt(1);

            // Recuperar los detalles de la factura

            try (PreparedStatement psSelect = cn.prepareStatement(querySelect)) {

                psSelect.setInt(1, facturalId);

                try (ResultSet rsSelect = psSelect.executeQuery()) {

                    if (rsSelect.next()) {

                        factura = new FacturaDTO();

                        factura.setIdFactura(rsSelect.getInt("ID_FACTURA"));

                        factura.setFecha(rsSelect.getDate("FECHA"));

                        factura.setClienteNombre(rsSelect.getString("CLIENTE_NOMBRE"));

```

```

        factura.setClienteCedula(rsSelect.getString("CLIENTE_CEDULA"));

        factura.setFormaPago(rsSelect.getString("FORMA_PAGO"));

        factura.setTotal(rsSelect.getDouble("TOTAL"));

        factura.setDescuento(rsSelect.getDouble("DESCUENTO"));

    }

}

}

}

}

}

}

return factura;

}

/**
 * Registra una factura para una compra con crédito.
 *
 * @param clienteNombre Nombre del cliente.
 * @param clienteCedula Cédula del cliente.
 * @param total      Total de la compra.
 * @return ID de la factura generada.
 * @throws Exception En caso de error en la base de datos.
 */
public int registrarFactura(String clienteNombre, String clienteCedula, double total) throws Exception {

```



```
String query = "INSERT INTO FACTURA (FECHA, CLIENTE_NOMBRE, CLIENTE_CEDULA,
FORMA_PAGO, TOTAL, DESCUENTO) "
```

```
+ "VALUES (NOW(), ?, ?, 'Crédito', ?, 0);";
```

```
try (Connection cn = AccesoDB.getConnection();
```

```
PreparedStatement ps = cn.prepareStatement(query,
PreparedStatement.RETURN_GENERATED_KEYS)) {
```

```
    ps.setString(1, clienteNombre);
```

```
    ps.setString(2, clienteCedula);
```

```
    ps.setDouble(3, total);
```

```
    ps.executeUpdate();
```

```
    try (ResultSet rs = ps.getGeneratedKeys()) {
```

```
        if (rs.next()) {
```

```
            return rs.getInt(1);
```

```
        } else {
```

```
            throw new Exception("No se pudo registrar la factura.");
```

```
        }
```

```
    }
```

```
}
```

```
}
```

```
/**
```

```
 * Consulta los detalles de una factura registrada.
```

```

*

* @param facturald ID de la factura.

* @return Detalles de la factura.

* @throws Exception En caso de error en la base de datos.

*/

public String consultarFactura(int facturald) throws Exception {

    String query = "SELECT ID_FACTURA, FECHA, CLIENTE_NOMBRE, CLIENTE_CEDULA, FORMA_PAGO,
TOTAL, DESCUENTO "

        + "FROM FACTURA WHERE ID_FACTURA = ?;";

    try (Connection cn = AccesoDB.getConnection();

        PreparedStatement ps = cn.prepareStatement(query)) {

        ps.setInt(1, facturald);

        try (ResultSet rs = ps.executeQuery()) {

            if (rs.next()) {

                return String.format("Factura ID: %d, Fecha: %s, Cliente: %s, Total: %.2f",

                    rs.getInt("ID_FACTURA"),

                    rs.getDate("FECHA"),

                    rs.getString("CLIENTE_NOMBRE"),

                    rs.getDouble("TOTAL"));

            } else {

                throw new Exception("Factura no encontrada.");

            }

        }

    }

}

```

```

    }

    }

}

/**
 * Consulta la tabla de amortización para un crédito específico.
 *
 * @param codCredito Código del crédito.
 * @return Lista de amortización.
 * @throws Exception En caso de error en la base de datos.
 */
public List<AmortizacionDTO> obtenerAmortizacion(int codCredito) throws Exception {

    List<AmortizacionDTO> amortizacion = new ArrayList<>();

    String query = "SELECT NUMERO_CUOTA, VALOR_CUOTA, INTERES_PAGADO, CAPITAL_PAGADO,
SALDO_RESTANTE "

        + "FROM CUOTA WHERE COD_CREDITO = ? ORDER BY NUMERO_CUOTA;";

    try (Connection cn = AccesoDB.getConnection();

        PreparedStatement ps = cn.prepareStatement(query)) {

        ps.setInt(1, codCredito);

        try (ResultSet rs = ps.executeQuery()) {

            while (rs.next()) {

                AmortizacionDTO cuota = new AmortizacionDTO();

```

```

        cuota.setNumeroCuota(rs.getInt("NUMERO_CUOTA"));

        cuota.setValorCuota(rs.getDouble("VALOR_CUOTA"));

        cuota.setInteresPagado(rs.getDouble("INTERES_PAGADO"));

        cuota.setCapitalPagado(rs.getDouble("CAPITAL_PAGADO"));

        cuota.setSaldo(rs.getDouble("SALDO_RESTANTE"));

        amortizacion.add(cuota);

    }

}

}

return amortizacion;

}

}

```

CREACIÓN DEL WEB SERVICE

Una vez creados los servicios y modelos se procede con la creación del Web Service el cual contiene las operaciones que estarán expuestas y se consumen por otros servidores o clientes.

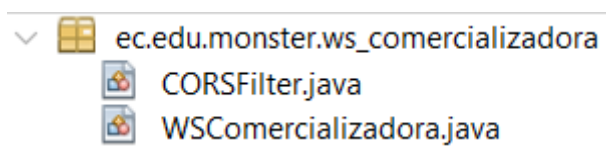


Figura 38 Creación de la clase WSBanQuito.

Se agrega un nuevo web service.

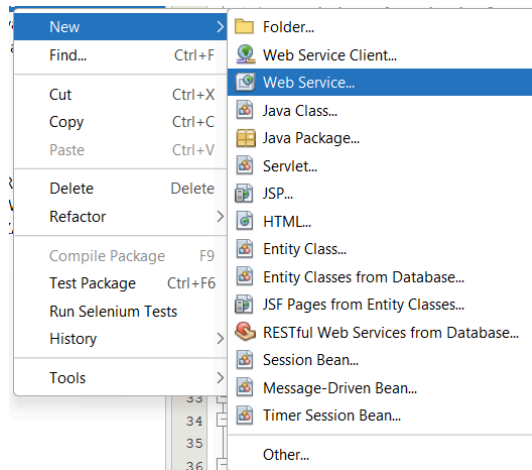


Figura 38 Creación de la clase WSComercializadora.

Se agregan las operaciones correspondientes con los requerimientos establecidos para la comercializadora de teléfonos.

```

1 package ec.edu.monster.ws_comercializadora;
2
3
4 import ec.edu.monster.modelo.AmortizacionDTO;
5 import ec.edu.monster.modelo.FacturaDTO;
6 import ec.edu.monster.servicio.ComercializadoraService;
7 import ec.edu.monster.ws.WSBanQuito;
8 import ec.edu.monster.ws.WSBanQuito_Service;
9
10 import javax.jws.WebMethod;
11 import javax.jws.WebParam;
12 import javax.jws.WebResult;
13 import javax.jws.WebService;
14 import java.util.List;
15
16 @WebService(serviceName = "WSComercializadora")
17 public class WSComercializadora {
18
19     private final ComercializadoraService comercializadoraService = new ComercializadoraService();
20     private final WSBanQuito banQuitoService;
21
22     public WSComercializadora() {
23         // Inicializar el cliente del servicio BanQuito
24         WSBanQuito_Service service = new WSBanQuito_Service();
25         this.banQuitoService = service.getWSBanQuitoPort();
26     }
27
28     @WebMethod(operationName = "compraConCredito")
29     @WebResult(name = "resultadoCompra")
30     public String compraConCredito(
31         @WebParam(name = "cedula") String cedula,
32         @WebParam(name = "codCliente") int codCliente,
33         @WebParam(name = "valor") double valor,
34         @WebParam(name = "numeroCuotas") int numeroCuotas,
35         @WebParam(name = "nombreCliente") String nombreCliente) {
36
37         try {
38             // Validar crédito

```

Figura 39 WSComercializadora codificación

Tabla 6 Codificación clase WSComercializadora

```
package ec.edu.monster.ws_comercializadora;
```

```

import ec.edu.monster.modelo.AmortizacionDTO;

import ec.edu.monster.modelo.FacturaDTO;

import ec.edu.monster.servicio.ComercializadoraService;

import ec.edu.monster.ws.WSBanQuito;

import ec.edu.monster.ws.WSBanQuito_Service;


import javax.jws.WebMethod;

import javax.jws.WebParam;

import javax.jws.WebResult;

import javax.jws.WebService;

import java.util.List;


@WebService(serviceName = "WSComercializadora")

public class WSComercializadora {


    private final ComercializadoraService comercializadoraService = new ComercializadoraService();

    private final WSBanQuito banQuitoService;


    public WSComercializadora() {

        // Inicializar el cliente del servicio BanQuito

        WSBanQuito_Service service = new WSBanQuito_Service();

        this.banQuitoService = service.getWSBanQuitoPort();

    }

```

```

@WebMethod(operationName = "compraConCredito")

@WebResult(name = "resultadoCompra")

public String compraConCredito(

    @WebParam(name = "cedula") String cedula,

    @WebParam(name = "codCliente") int codCliente,

    @WebParam(name = "valor") double valor,

    @WebParam(name = "numeroCuotas") int numeroCuotas,

    @WebParam(name = "nombreCliente") String nombreCliente) {

    try {

        // Validar crédito

        boolean esSujetoCredito = banQuitoService.validarCredito(cedula);

        if (!esSujetoCredito) {

            return "El cliente no es sujeto de crédito.";

        }

        // Consultar monto máximo

        double montoMaximo = banQuitoService.calcularMaxCredito(codCliente);

        if (valor > montoMaximo) {

            return "El valor excede el monto máximo aprobado.";

        }

        // Otorgar crédito

        banQuitoService.otorgarCredito(codCliente, valor, numeroCuotas);
    }
}

```

```

        // Obtener tabla de amortización

        List<ec.edu.monster.ws.AmortizacionDTO> tablaAmortizacion =
banquitoService.obtenerAmortizacion(codCliente);

        // Registrar factura

        int facturald = comercializadoraService.registrarFactura(nombreCliente, cedula, valor);

        // Consultar detalles de la factura

        String facturaDetalles = comercializadoraService.consultarFactura(facturald);

        return String.format("Compra realizada con éxito. Factura: %s, Amortización: %s",

            facturaDetalles, tablaAmortizacion.toString());

    } catch (Exception e) {

        throw new RuntimeException("Error: " + e.getMessage(), e);

    }

}

@WebMethod(operationName = "facturarEfectivo")

@WebResult(name = "factura")

public FacturaDTO facturarEfectivo(

    @WebParam(name = "clienteNombre") String clienteNombre,

    @WebParam(name = "clienteCedula") String clienteCedula,

    @WebParam(name = "precio") double precio) {

```



```

try {

    return comercializadoraService.facturarEfectivo(clienteNombre, clienteCedula, precio);

} catch (Exception e) {

    throw new RuntimeException("Error al facturar en efectivo: " + e.getMessage(), e);

}

}

}

```

Ahora se ejecuta el servidor y verificamos que el wsdl se genere correctamente con el código XML.

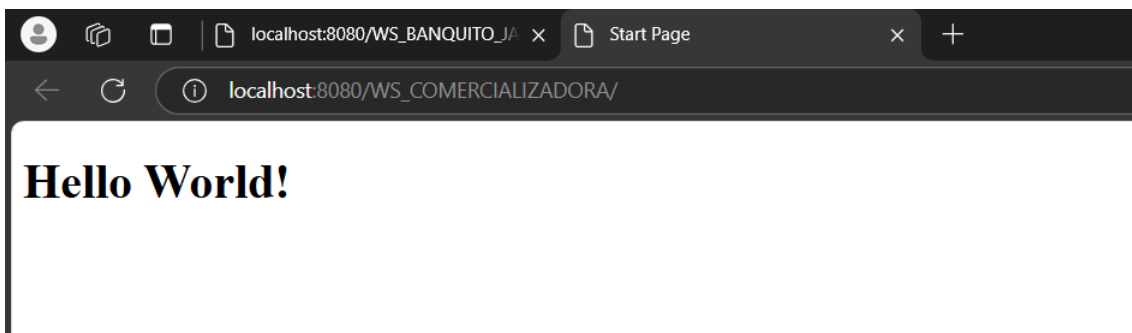


Figura 40 Ejecución del servidor Comercializadora

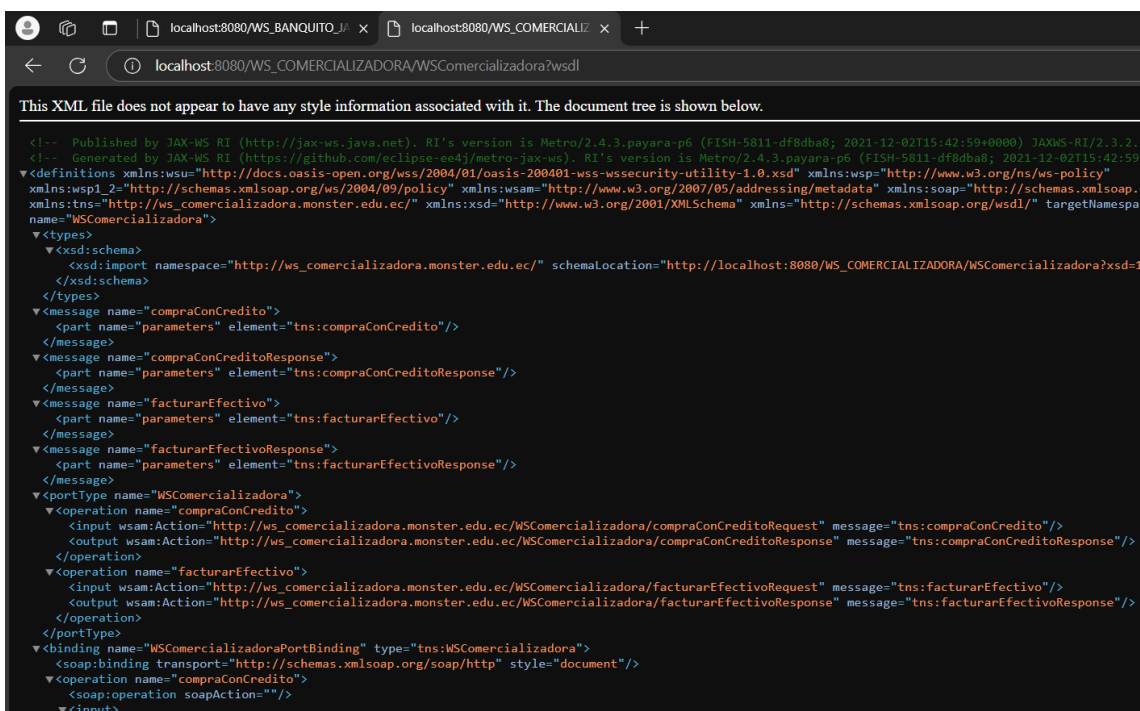


Figura 61 wsdl del servidor BanQuito

APLICACIÓN CLIENTE

CREACIÓN DEL PROYECTO CLIENTE ESCRITORIO

Realizado el servidor se avanza a la aplicación cliente, para ello se crea un nuevo proyecto de tipo Windows Forms.NET Framework llamado 02.CLIESC.

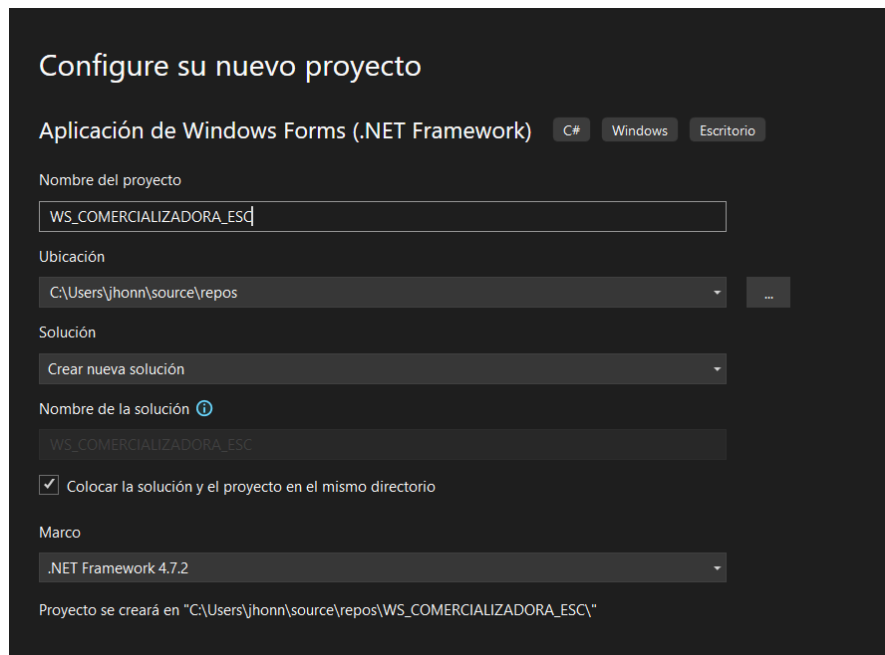


Figura 66 Creación del proyecto cliente

Si todo va bien se muestra la siguiente estructura donde agregamos nuestros directorios MVC.

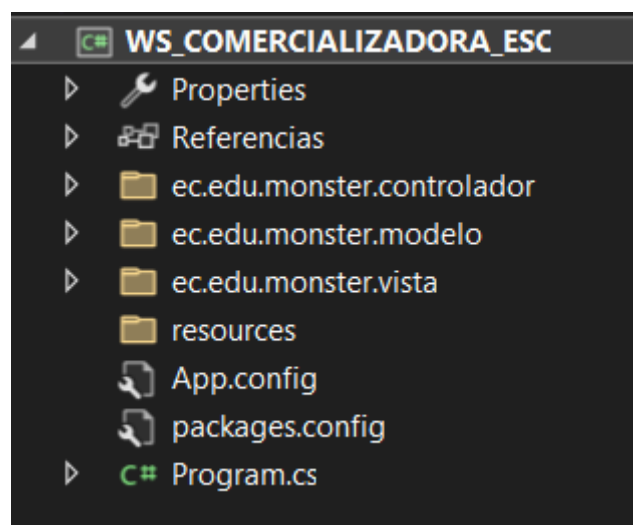


Figura 69 Estructura inicial del proyecto

CREACIÓN DEL MODELO

Se crea la capa de modelo para mapear las entidades de nuestra base.

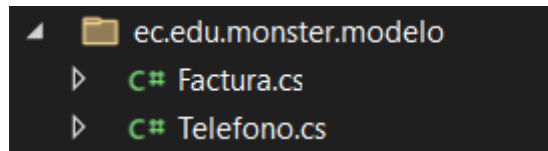


Figura 70 Creación de la clase Factura y Teléfono

Una vez creadas las clases se procede a su codificación.

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace WS_COMERCIALIZADORA_ESC.ec.edu.monster.modelo
{
    public class Telefono
    {
        public int IdTelefono { get; set; }
        1 referencia
        public string Modelo { get; set; }
        1 referencia
        public string Marca { get; set; }
        1 referencia
        public decimal Precio { get; set; }
        0 referencias
        public string Descripcion { get; set; }
    }
}
```

Figura 70 Creación de la clase Telefono

Tabla 17 Clase llamada Telefono

using System;

using System.Collections.Generic;

using System.Linq;

using System.Text;

using System.Threading.Tasks;

namespace WS_COMERCIALIZADORA_ESC.ec.edu.monster.modelo

{

public class Telefono

{

public int IdTelefono { get; set; }

public string Modelo { get; set; }

```

        public string Marca { get; set; }

        public decimal Precio { get; set; }

        public string Descripcion { get; set; }

    }

}

```

CREACIÓN DEL CONTROLADOR

Para crear el controlador, es necesario crear un paquete y crear una nueva clase.

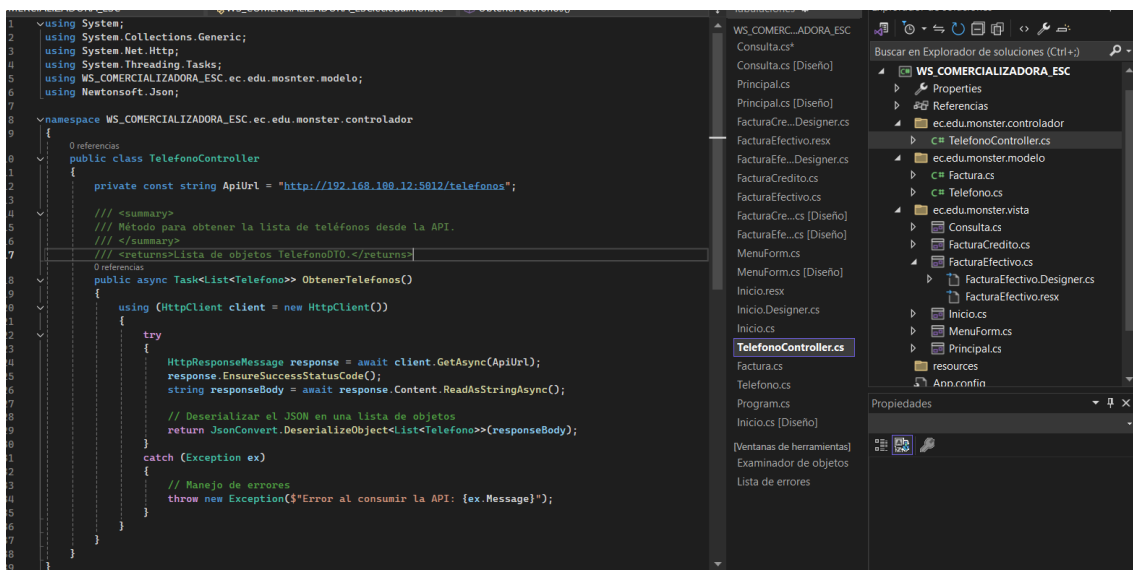


Figura 76 Creación del controlador

Después se codifica la clase de la siguiente manera.

Tabla 12 Codificación controlador

```
using System;

using System.Collections.Generic;

using System.Net.Http;

using System.Threading.Tasks;

using WS_COMERCIALIZADORA_ESC.ec.edu.monster.modelo;

using Newtonsoft.Json;

namespace WS_COMERCIALIZADORA_ESC.ec.edu.monster.controlador

{

    public class TelefonoController

    {

        private const string ApiUrl = "http://192.168.100.12:5012/telefonos";
```

```
/// <summary>

/// Método para obtener la lista de teléfonos desde la API.

/// </summary>

/// <returns>Lista de objetos TelefonoDTO.</returns>

public async Task<List<Telefono>> ObtenerTelefonos()

{

    using (HttpClient client = new HttpClient())

    {

        try

        {

            HttpResponseMessage response = await client.GetAsync(ApiUrl);

            response.EnsureSuccessStatusCode();

            string responseBody = await response.Content.ReadAsStringAsync();
```

```
        // Deserializar el JSON en una lista de objetos

        return JsonConvert.DeserializeObject<List<Telefono>>(responseBody);

    }

    catch (Exception ex)

    {

        // Manejo de errores

        throw new Exception($"Error al consumir la API: {ex.Message}");

    }

}

}

}

}
```

CREACIÓN DE LA VISTA

Para la creación de la vista es necesario realizar 3 formularios.

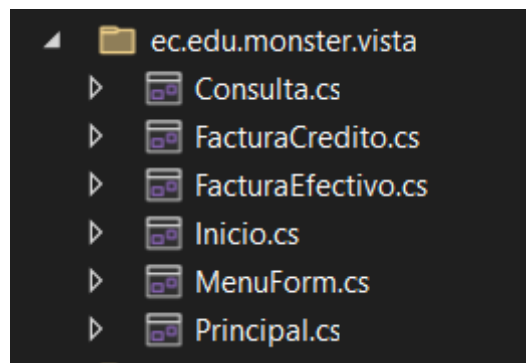


Figura 77 Creación de las vistas

Se los diseña de la manera que cada persona desee, empezando por la vista principal, Depósito y Consulta.



Figura 78 Prueba a la interfaz gráfica principal

Consulta de Amortización

Credito: 2

	Cuota	Fecha Pago	Valor Cuota	Interés	Capital	Saldo
1		2026-01-16	68.05	10.00	58.05	691.95
2		2026-02-16	68.05	9.23	58.82	633.13
3		2026-03-16	68.05	8.44	59.61	573.52
4		2026-04-16	68.05	7.65	60.40	513.11
5		2026-05-16	68.05	6.84	61.21	451.91
6		2026-06-16	68.05	6.03	62.02	389.88
7		2026-07-16	68.05	5.20	62.85	327.03
8		2026-08-16	68.05	4.36	63.69	263.34
9		2026-09-16	68.05	3.51	64.54	198.80
10		2026-10-16	68.05	2.65	65.40	133.40
11		2026-11-16	68.05	1.78	66.27	67.13
12		2026-12-16	68.05	0.90	67.15	0.00

Figura 78 Prueba a la interfaz gráfica ver amortización

Seleccionar Producto del Catálogo



PlayStation 5 - \$800.00

ID	Nombre	Precio	Descripción
1	Refrigeradora	750.00	Refrigeradora d...
2	Cocina de Induc...	500.00	4 hornillas
3	Lavadora	450.00	Carga frontal 20...
4	Microondas	150.00	Digital plateado
5	PlayStation 5	800.00	Consola Sony 82...
6	Laptop HP	650.00	Core i5 8GB RAM
7	Licuada Oster	80.00	3 Velocidades
8	Smart TV Sony 5...	450.00	Televisor Ultra H...
9	PlayStation 4	250.00	Play Station 4 Sli...
10	Xbox One	450.00	Consola de Vide...

Figura 79 Prueba a la interfaz gráfica de catalogo

Nueva Venta

Cliente: 1718972597 Ednan Merino

Producto: (Seleccione un producto)

Cantidad: 1

Pago: Forma: Cuotas: 0

ID	Nombre	Cant.	Precio U.	Subtotal
5	PlayStation 5	1	800.00	800.00

Figura 80 Prueba a la interfaz gráfica opción de pago.

Confirmar Compra

*** CONFIRMAR COMPRA ***

Cliente: Ednan Merino
Cédula: 1718972597

Producto	Cant.	Subtotal
PlayStation 5	1	800,00
SUBTOTAL:		800,00
DESCUENTO (33%):		-264,00
TOTAL A PAGAR:		536,00

Forma de Pago: Efectivo.

Aceptar

Cancelar

Figura 80 Prueba a la interfaz gráfica opción de pago en efectivo.

Cantidad:

1

Añadir

Quitar

Pago:

Forma:

Credito

Cuotas:

25

ID	Nombre
5	PlayStation 5

Figura 80 Prueba a la interfaz gráfica opción de pago a credito.

Una vez desarrollada la interfaz gráfica, se procede a programar los distintos métodos para cada clase respectivamente.

Tabla 13 LoginVista

```
using System;

using System.Net.Http;

using System.Text;

using System.Threading.Tasks;

using System.Windows.Forms;

using Newtonsoft.Json;

namespace WS_COMERCIALIZADORA_ESC.ec.edu.monster.vista

{

    public partial class Login : Form

    {

        private const string ApiUrl = "http://192.168.137.1:5012/login"; // URL del API

        public Login()

        {
```

```
InitializeComponent();

this.StartPosition = FormStartPosition.CenterScreen;

txtContrasena.PasswordChar = '*';

}

private async void btnInicio_Click(object sender, EventArgs e)
{
    // Capturar los datos ingresados por el usuario

    string username = txtUsuario.Text.Trim();

    string password = txtContrasena.Text.Trim();

    // Validar que los campos no estén vacíos

    if (string.IsNullOrEmpty(username))
    {
        MessageBox.Show("El campo Usuario es obligatorio.", "Error", MessageBoxButtons.OK, MessageBoxIcon.Warning);
    }
}
```

```
        return;
    }

    if (string.IsNullOrEmpty(password))
    {
        MessageBox.Show("El campo Contraseña es obligatorio.", "Error", MessageBoxButtons.OK, MessageBoxIcon.Warning);

        return;
    }

    // Realizar la autenticación

    bool autenticado = await AutenticarUsuarioAsync(username, password);

    if (autenticado)
    {
        // Si la autenticación es exitosa, abre la ventana principal
    }
}
```

```
        MessageBox.Show("¡Inicio de sesión exitoso!", "Éxito", MessageBoxButtons.OK, MessageBoxIcon.Information);

        this.Hide(); // Oculta el formulario de login

    }

    // Abre la ventana principal

    Principal principalForm = new Principal();

    principalForm.ShowDialog();

    this.Hide(); // Cierra el formulario de login

}

else

{

    MessageBox.Show("Credenciales incorrectas. Intente nuevamente.", "Error", MessageBoxButtons.OK, MessageBoxIcon.Error);

}

}

private async Task<bool> AutenticarUsuarioAsync(string username, string password)
```

```
{  
  
    try  
  
    {  
  
        using (HttpClient client = new HttpClient())  
  
        {  
  
            // Crear el objeto de solicitud  
  
            var loginRequest = new  
  
            {  
  
                Username = username,  
  
                Password = password  
  
            };  
  
  
  
            // Serializar el objeto a JSON  
  
            string jsonRequest = JsonConvert.SerializeObject(loginRequest);
```

```
// Configurar la solicitud HTTP

var content = new StringContent(jsonRequest, Encoding.UTF8, "application/json");

HttpResponseMessage response = await client.PostAsync(ApiUrl, content);

if (response.IsSuccessStatusCode)

{

    string result = await response.Content.ReadAsStringAsync();

    return result == "1"; // Verificar si el servidor devuelve "1" para éxito

}

}

}

catch (Exception ex)

{

    MessageBox.Show($"Error al autenticar: {ex.Message}", "Error", MessageBoxButtons.OK, MessageBoxIcon.Error);

}
```



```
        return false; // Retorna false si ocurre un error o si no es exitoso  
    }  
}  
}  
}
```

```
using Newtonsoft.Json;  
  
using Newtonsoft.Json.Linq;  
  
using System;  
  
using System.Net.Http;  
  
using System.Text;  
  
using System.Threading.Tasks;  
  
using System.Windows.Forms;
```

```
namespace WS_COMERCIALIZADORA_ESC.ec.edu.monster.vista

{

    public partial class FacturaCredito : Form

    {

        private readonly string _telefonoSeleccionado;

        private readonly decimal _precio;

        private JObject facturaGenerada;

        private JArray tablaAmortizacion;


        public FacturaCredito(string telefonoSeleccionado, decimal precio)

        {

            InitializeComponent();

            _telefonoSeleccionado = telefonoSeleccionado;

        }

    }

}
```

```
_precio = precio;

this.StartPosition = FormStartPosition.CenterScreen;

}

private async void button1_Click(object sender, EventArgs e)

{

    if (string.IsNullOrEmpty(txtCedula.Text) ||

        string.IsNullOrEmpty(txtCodCliente.Text) ||

        string.IsNullOrEmpty(txtNumeroCuotas.Text))

    {

        MessageBox.Show("Por favor, complete todos los campos.", "Advertencia", MessageBoxButtons.OK, MessageBoxIcon.Warning);

        return;

    }

}
```

```
try
{
    var compraRequest = new
    {
        Cedula = txtCedula.Text,
        CodCliente = int.Parse(txtCodCliente.Text),
        Valor = _precio,
        NumeroCuotas = int.Parse(txtNumeroCuotas.Text),
        Nombre = txtClienteNombre.Text
    };

    var jsonContent = JsonConvert.SerializeObject(compraRequest);

    var content = new StringContent(jsonContent, Encoding.UTF8, "application/json");
```

```
using (var client = new HttpClient())

{

    string url = "http://192.168.137.1:5012/compras/credito";

    var response = await client.PostAsync(url, content);


    if (response.IsSuccessStatusCode)

    {

        // Leer la respuesta exitosa

        var responseBody = await response.Content.ReadAsStringAsync();


        // Deserializar la respuesta a un JObject

        var responseData = JObject.Parse(responseBody);


        // Extraer los datos necesarios
```

```
string mensaje = responseData["mensaje"]?.ToString();

facturaGenerada = (JsonObject)responseData["factura"];

tablaAmortizacion = (JArray)responseData["tablaAmortizacion"];

MessageBox.Show($"¡Compra con crédito realizada exitosamente!\n\nMensaje: {mensaje}",
    "Éxito", MessageBoxButtons.OK, MessageBoxIcon.Information);

btnMostrarFactura.Visible = true; // Hacer visible el botón para mostrar la factura
}

else
{

var errorResponse = await response.Content.ReadAsStringAsync();

if (string.IsNullOrEmpty(errorResponse))
```

```
{  
  
    MessageBox.Show("El servidor no devolvió ningún contenido.", "Error en la Respuesta", MessageBoxButtons.OK, MessageBoxIcon.Error);  
  
    return;  
  
}  
  
try  
  
{  
  
    var errorData = JObject.Parse(errorResponse);  
  
    string mensaje = errorData["mensaje"]?.ToString() ?? "Error desconocido.";  
  
    MessageBox.Show($"No se pudo realizar el crédito:\n\nMensaje: {mensaje}",  
  
        "Crédito Rechazado",  
  
        MessageBoxButtons.OK,  
  
        MessageBoxIcon.Warning);  
  
}
```

```
        catch (Exception ex)

        {

            MessageBox.Show($"Error al procesar la respuesta del servidor:\n{errorResponse}\n\nDetalle: {ex.Message}", "Error", MessageBoxButtons.OK,
MessageBoxIcon.Error);

        }

    }

}

}

catch (Exception ex)

{

    MessageBox.Show($"Ocurrió un error inesperado:\n{ex.Message}", "Error", MessageBoxButtons.OK, MessageBoxIcon.Error);

}

}
```



```
private void btnMostrarFactura_Click(object sender, EventArgs e)

{

    if (facturaGenerada == null || tablaAmortizacion == null)

    {

        MessageBox.Show("No hay datos para mostrar.", "Información", MessageBoxButtons.OK, MessageBoxIcon.Information);

        return;

    }

    try

    {

        StringBuilder facturaDetalle = new StringBuilder();

        facturaDetalle.AppendLine("Factura:");

        facturaDetalle.AppendLine($"ID: {facturaGenerada["idFactura"]}");

        facturaDetalle.AppendLine($"Fecha: {facturaGenerada["fecha"]}");

    }

    catch { }

}
```

```

facturaDetalle.AppendLine($"Cliente: {facturaGenerada["clienteNombre"]}");

facturaDetalle.AppendLine($"Total: ${facturaGenerada["total"]}");

facturaDetalle.AppendLine("\nTabla de Amortización:");

foreach (var cuota in tablaAmortizacion)
{
    facturaDetalle.AppendLine($"Cuota {cuota["numeroCuota"]}: Valor Cuota ${cuota["valorCuota"]}, Capital ${cuota["capitalPagado"]}, Interés
    ${cuota["interesPagado"]}, Saldo ${cuota["saldo"]}");

}

MessageBox.Show(facturaDetalle.ToString(), "Factura y Tabla de Amortización", MessageBoxButtons.OK, MessageBoxIcon.Information);

}

catch (Exception ex)

```

```
{  
  
    MessageBox.Show($"Error al procesar los detalles de la factura:\n{ex.Message}", "Error", MessageBoxButtons.OK, MessageBoxIcon.Error);  
  
}  
  
}
```

```
private void FacturaCredito_Load(object sender, EventArgs e)
```

```
{  
  
    lblTelefonoSeleccionado.Text = $"Teléfono Seleccionado: {_telefonoSeleccionado}";  
  
    lblPrecio.Text = $"Precio: {_precio:C}";  
  
    btnMostrarFactura.Visible = false;  
  
}
```

```
private void btnRegresar_Click(object sender, EventArgs e)
```

```
{
```

```
        var inicio = new Inicio();  
  
        inicio.Show();  
  
        this.Close();  
  
    }  
  
}  
  
}
```

```
using System;  
  
using System.Net.Http;  
  
using System.Text;  
  
using System.Threading.Tasks;  
  
using System.Windows.Forms;  
  
using Newtonsoft.Json;
```

```
namespace WS_COMERCIALIZADORA_ESC.ec.edu.monster.vista

{

    public partial class FacturaEfectivo : Form

    {

        private readonly string _telefonoSeleccionado;

        private readonly decimal _precio;


        public FacturaEfectivo(string telefonoSeleccionado, decimal precio)

        {

            InitializeComponent();

            _telefonoSeleccionado = telefonoSeleccionado;

            _precio = precio;

            this.StartPosition = FormStartPosition.CenterScreen;

        }

    }

}
```

```
private async void btnGenerarFactura_Click_1(object sender, EventArgs e)

{

    // Validar campos requeridos

    if (string.IsNullOrEmpty(txtClienteNombre.Text) || string.IsNullOrEmpty(txtClienteCedula.Text))

    {

        MessageBox.Show("Por favor, complete todos los campos.", "Advertencia", MessageBoxButtons.OK, MessageBoxIcon.Warning);

        return;

    }

    try

    {

        // Crear el objeto FacturaRequest con descuento y total calculados

        var descuento = _precio * 0.42m; // Calcula el 42% de descuento
```

```
var total = _precio - descuento;

var facturaRequest = new

{

    ClienteNombre = txtClienteNombre.Text,

    ClienteCedula = txtClienteCedula.Text,

    Precio = _precio

};

// Serializar el objeto a JSON usando Newtonsoft.Json

var jsonContent = JsonConvert.SerializeObject(facturaRequest);

var content = new StringContent(jsonContent, Encoding.UTF8, "application/json");

// Enviar la solicitud POST
```

```

using (var client = new HttpClient())

{

    string url = "http://192.168.137.1:5012/facturas/efectivo"; // URL del servicio

    var response = await client.PostAsync(url, content);


    if (response.IsSuccessStatusCode)

    {

        // Leer y mostrar la respuesta

        var responseBody = await response.Content.ReadAsStringAsync();


        // Mostrar mensaje con los detalles de la factura generada

        MessageBox.Show($"Factura generada exitosamente:\n\n" +

            $"Cliente: {txtClienteNombre.Text}\n" +

            $"Cédula: {txtClienteCedula.Text}\n" +

```



```
        $"Precio Original: {_precio:C}\n" +  
  
        $"Descuento: {descuento:C}\n" +  
  
        $"Total a Pagar: {total:C}",  
  
        "Éxito", MessageBoxButtons.OK, MessageBoxIcon.Information);  
  
// Abrir el formulario Inicio  
  
var inicio = new Inicio(); // Ajusta el constructor según sea necesario  
  
inicio.Show();  
  
// Cierra este formulario  
  
this.Close();  
  
}  
  
else  
  
{
```

```
        // Leer y mostrar el mensaje de error devuelto por la API

        var errorResponse = await response.Content.ReadAsStringAsync();

        MessageBox.Show($"Error al generar la factura:\n{errorResponse}", "Error", MessageBoxButtons.OK, MessageBoxIcon.Error);

    }

}

}

catch (Exception ex)

{

    // Manejo de errores generales

    MessageBox.Show($"Ocurrió un error:\n{ex.Message}", "Error", MessageBoxButtons.OK, MessageBoxIcon.Error);

}

}

private void FacturaEfectivo_Load(object sender, EventArgs e)
```

```
{  
  
    // Mostrar información inicial en los Labels  
  
    lblTelefonoSeleccionado.Text = $"Teléfono Seleccionado: {_telefonoSeleccionado}";  
  
    // lblPrecio.Text = $"Precio Original: {_precio:C}"; // Mostrar el precio con formato de moneda  
  
}
```

```
private void label2_Click(object sender, EventArgs e)
```

```
{
```

```
}
```

```
private void btnRegresar_Click(object sender, EventArgs e)
```

```
{
```

```
    var inicio = new Inicio();
```

```
        inicio.ShowDialog();  
  
        this.Close();  
    }  
}  
}
```

EJECUCIÓN

Para realizar la ejecución, se debe tener desplegado el servidor, como siguiente paso es necesario en la aplicación cliente dar clic derecho y mandar a ejecutar la vista principal.

Una vez ejecutada la vista principal se puede presionar en cualquiera de las 3 opciones y se abren ventanas similares a las siguientes:

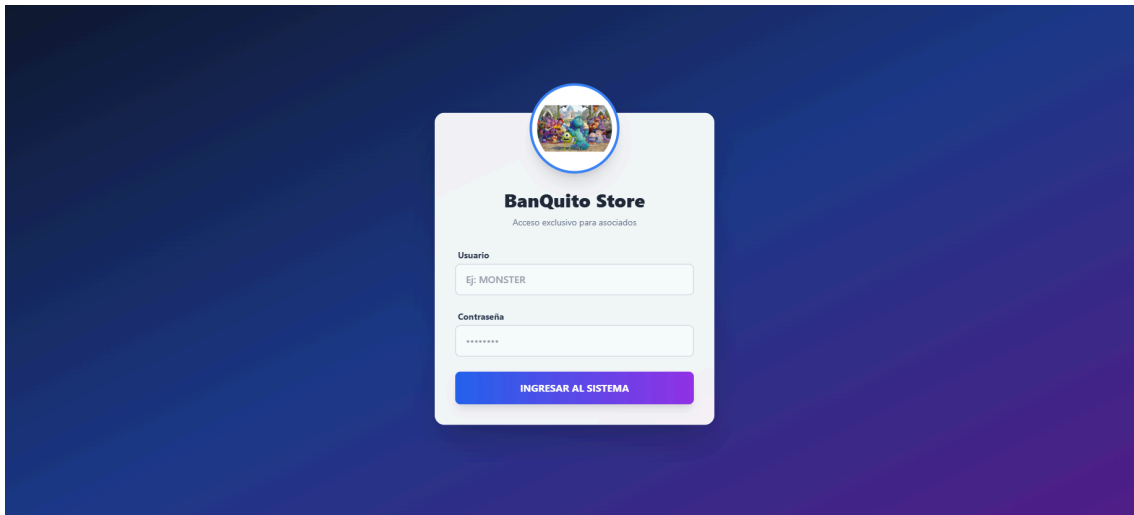


Figura 76 Inicio de sesión

Nuevo Producto

Nombre

Precio

Stock

URL Imagen

https://...

Descripción

CREAR PRODUCTO

Producto

\$0

Figura 77 CRUD de teléfonos

Nuevo Producto

Nombre

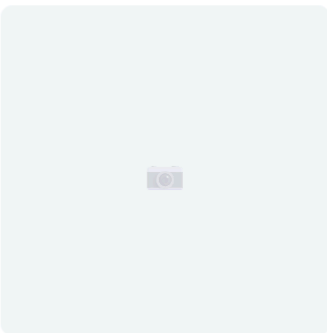
Precio

Stock

URL Imagen

Descripción

CREAR PRODUCTO



Producto

\$0

Figura 78 Agregar Teléfono


Comercializadora BanQuito

Catálogo
Nueva Venta
Consultas
Inventario
Cerrar Sesión

Bienvenido, Asociado

Gestiona el inventario y realiza ventas en la plataforma oficial de Comercializadora BanQuito.



Catálogo Disponible



Stock: 9

Refrigeradora

Refrigeradora doble puerta



Stock: 19

Cocina de Inducción

4 hornillas



Stock: 9

Lavadora

Carga frontal 20kg



Stock: 29

Microondas

Digital plateado

Figura 78 Prueba a la interfaz gráfica principal

125

Centro de Consultas

[Consultar Factura](#)[Tabla de Amortización](#)Buscar

Plan de Pagos (Crédito)

Cuota	Fecha Pago	Valor Cuota	Interés	Saldo Capital
1	16/1/2026	\$68.05	\$10	\$691.95
2	16/2/2026	\$68.05	\$9.23	\$633.13
3	16/3/2026	\$68.05	\$8.44	\$573.52
4	16/4/2026	\$68.05	\$7.65	\$513.11
5	16/5/2026	\$68.05	\$6.84	\$451.91
6	16/6/2026	\$68.05	\$6.03	\$389.88
7	16/7/2026	\$68.05	\$5.2	\$327.03
8	16/8/2026	\$68.05	\$4.36	\$263.34

Figura 78 Prueba a la interfaz gráfica ver amortización

Catálogo Disponible



Refrigeradora
Refrigeradora doble puerta

PRECIO
\$750



Cocina de Inducción
4 hornillas

PRECIO
\$500



Lavadora
Carga frontal 20kg

PRECIO
\$450



Microondas
Digital plateado

PRECIO
\$150



PlayStation 5
Consola Sony 825GB



Laptop HP
Core i5 8GB RAM



Licuadora Oster
3 Velocidades



Smart TV Sony 55 4K
Televisor Ultra HD con Android TV

Figura 79 Interfaz gráfica de catálogo

Datos del Cliente

☒ Ednan Merino

Agregar Productos

PRODUCTO

Laptop HP - \$650 (Stock: 8)

CANTIDAD

1

Carrito de Compra 2 items

Cocina de Inducción

1 x \$500

\$500.00

Laptop HP

1 x \$650

\$650.00

FORMA DE PAGO

☒ Efectivo (33% Descuento)

Total Estimado

\$1150.00

Figura 80 Carrito de productos

Carrito de Compra

2 items

Cocina de Inducción

1 x \$500

\$500.00

Laptop HP

1 x \$650

\$650.00

FORMA DE PAGO

☒ Efectivo (33% Descuento)

☒ Efectivo (33% Descuento)
 ☐ Crédito Diferido

APLICACIÓN CLIENTE CREACIÓN DEL PROYECTO CONSOLA

Creamos un nuevo proyecto aplicación de consola .NET Framework

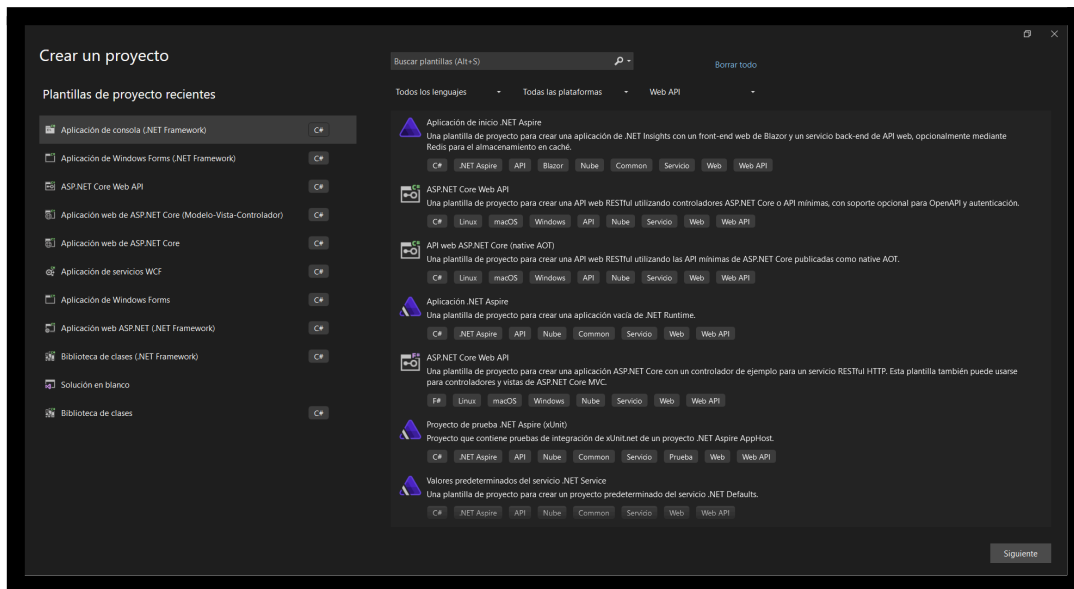


Figura 83 Creación Proyecto Aplicación Consola

Realizado el servidor se avanza a la aplicación cliente, para ello se crea un nuevo proyecto denominado `WS_COMERCIALIZADORA_CON`.

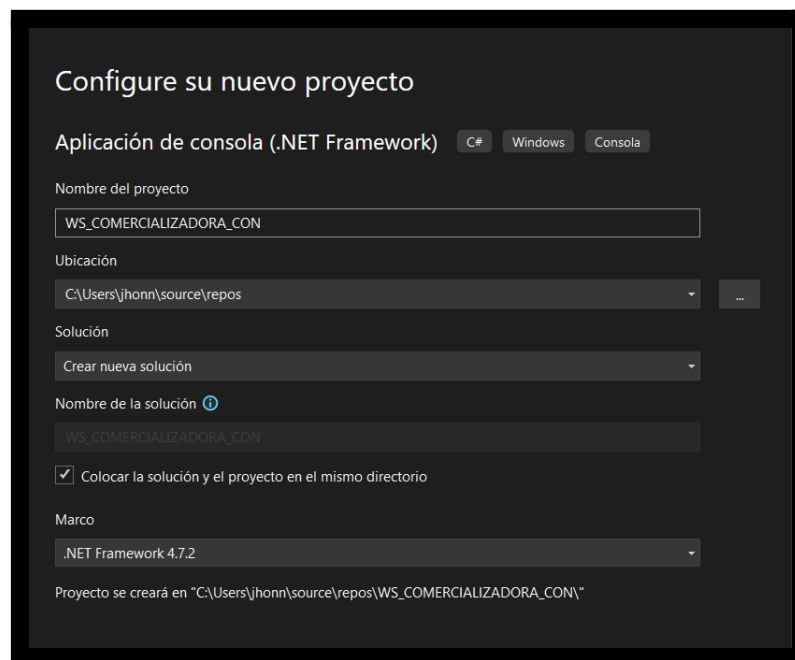


Figura 84 Creación proyecto estructura general

Si todo va bien se muestra la siguiente estructura, a la que agregamos los siguientes paquetes.

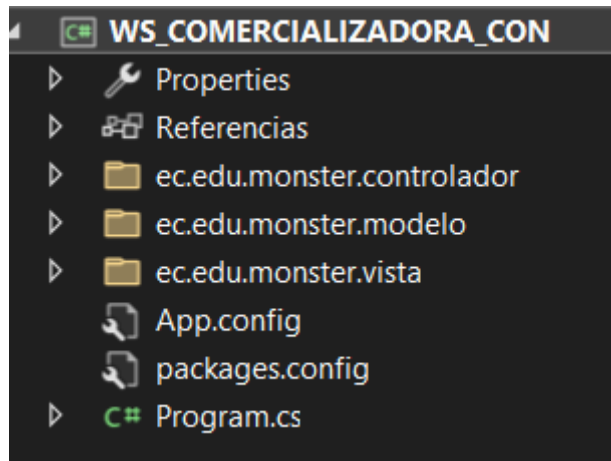


Figura 88 90 Estructura general del proyecto

CREACIÓN DEL MODELO

Se crea la capa de modelo para mapear las entidades de nuestra base.

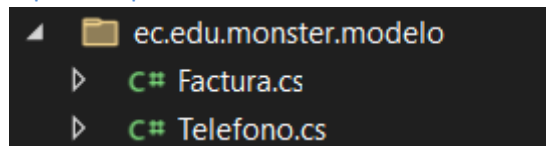


Figura 70 Creación de la clase Factura y Teléfono.

Una vez creadas las clases se procede a su codificación.

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace WS_COMERCIALIZADORA_ESC.ec.edu.monster.modelo
{
    public class Telefono
    {
        public int IdTelefono { get; set; }
        public string Modelo { get; set; }
        public string Marca { get; set; }
        public decimal Precio { get; set; }
        public string Descripcion { get; set; }
    }
}
```

Figura 70 Creación de la clase Telefono

Tabla 17 Clase llamada Telefono

```
using System;
```

```
using System.Collections.Generic;
```

```
using System.Linq;
```

```

using System.Text;

using System.Threading.Tasks;

namespace WS_COMERCIALIZADORA_ESC.ec.edu.monster.modelo
{

    public class Telefono

    {

        public int IdTelefono { get; set; }

        public string Modelo { get; set; }

        public string Marca { get; set; }

        public decimal Precio { get; set; }

        public string Descripcion { get; set; }

    }

}

```

CREACIÓN DEL CONTROLADOR

Para crear el controlador, es necesario crear un paquete y crear una nueva clase.

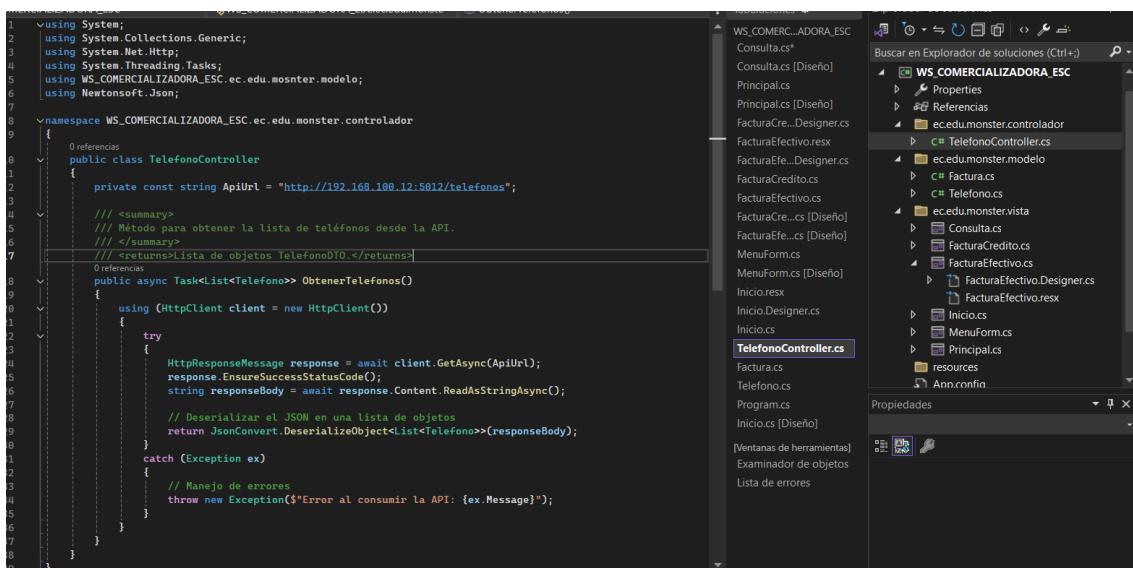


Figura 76 Creación del controlador

Después se codifica la clase de la siguiente manera.

CREACIÓN DE LA VISTA

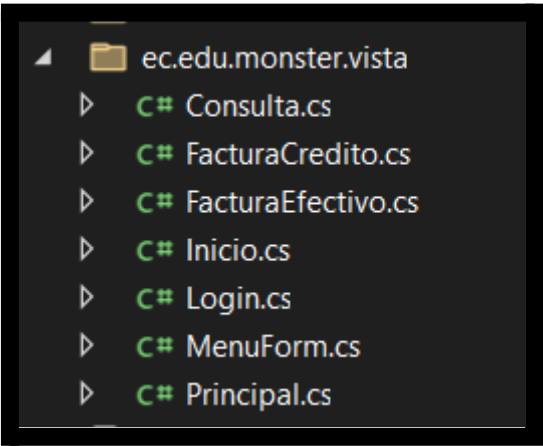


Figura 89 Estructura general de las clases

CREACIÓN DE LOGIN

```
using System;
using System.Net.Http;
using System.Text;
using System.Threading.Tasks;
using Newtonsoft.Json;

namespace WS_COMERCIALIZADORA_CON.ec.edu.monster.vista
{
    1 referencia
    internal class Login
    {
        private const string ApiUrl = "http://192.168.137.1:5012/login";

        1 referencia
        public async Task IniciarAsync()
        {
            Console.WriteLine("=== Bienvenido al Sistema de Comercializadora ===");
            Console.Write("Usuario: ");
            string username = Console.ReadLine();

            Console.Write("Contraseña: ");
            string password = LeerContraseña(); // Método para ocultar la contraseña

            // Llama al método para autenticar al usuario
            bool autenticado = await AutenticarUsuarioAsync(username, password);

            if (autenticado)
            {
                Console.WriteLine("\nInicio de sesión exitoso!");
                Console.WriteLine("Redirigiendo al menú principal...");
                await Task.Delay(1000); // Espera un segundo antes de redirigir

                // Llama al menú principal
                var principal = new Principal();
                principal.MostrarMenuPrincipal();
            }
            else
            {
                Console.WriteLine("\nCredenciales incorrectas. Intente nuevamente.");
                Console.WriteLine("\nPresione cualquier tecla para salir...");
                Console.ReadKey();
            }
        }
    }
}
```

Figura 90 Creación de la clase login que contendrá el código de la vista

Tabla 28 Clase llamada Login

using System;

using System.Net.Http;

```

using System.Text;

using System.Threading.Tasks;

using Newtonsoft.Json;

namespace WS_COMERCIALIZADORA_CON.ec.edu.monster.vista
{
    internal class Login
    {
        private const string ApiUrl = "http://192.168.137.1:5012/login";

        public async Task IniciarAsync()
        {
            Console.WriteLine("=== Bienvenido al Sistema de Comercializadora ===");

            Console.Write("Usuario: ");

            string username = Console.ReadLine();

            Console.Write("Contraseña: ");

            string password = LeerContraseña(); // Método para ocultar la contraseña

            // Llama al método para autenticar al usuario

            bool autenticado = await AutenticarUsuarioAsync(username, password);

            if (autenticado)
            {
                Console.WriteLine("\n¡Inicio de sesión exitoso!");
            }
        }
    }
}

```

```

        Console.WriteLine("Redirigiendo al menú principal...");

        await Task.Delay(1000); // Espera un segundo antes de redirigir

        // Llama al menú principal

        var principal = new Principal();

        principal.MostrarMenuPrincipal();

    }

    else

    {

        Console.WriteLine("\nCredenciales incorrectas. Intente nuevamente.");

        Console.WriteLine("\nPresione cualquier tecla para salir...");

        Console.ReadKey();

    }

}

private string LeerContraseña()

{

    StringBuilder password = new StringBuilder();

    ConsoleKeyInfo key;

    // Lee caracteres uno por uno hasta que se presione Enter

    do

    {

        key = Console.ReadKey(intercept: true);

```

```

        if (key.Key == ConsoleKey.Backspace && password.Length > 0)
        {
            password.Remove(password.Length - 1, 1);

            Console.Write("\b \b"); // Borra el último carácter en la consola
        }

        else if (!char.IsControl(key.KeyChar))
        {
            password.Append(key.KeyChar);

            Console.Write("*"); // Muestra un asterisco en lugar del carácter real
        }
    } while (key.Key != ConsoleKey.Enter);

    Console.WriteLine(); // Salta a la siguiente línea después de Enter

    return password.ToString();
}

private async Task<bool> AutenticarUsuarioAsync(string username, string password)
{
    try
    {
        using (HttpClient client = new HttpClient())
        {
            // Crea el objeto de solicitud

            var loginRequest = new
            {

```

```

        Username = username,

        Password = password

    };

    // Serializa el objeto a JSON

    string jsonRequest = JsonConvert.SerializeObject(loginRequest);

    // Configura la solicitud HTTP

    var content = new StringContent(jsonRequest, Encoding.UTF8, "application/json");

    HttpResponseMessage response = await client.PostAsync(ApiUrl, content);

    if (response.IsSuccessStatusCode)
    {
        string result = await response.Content.ReadAsStringAsync();

        return result == "1"; // Verifica si el servidor devuelve "1"
    }
}

catch (Exception ex)
{
    Console.WriteLine($"Error al autenticar: {ex.Message}");
}

return false; // Devuelve false si hubo un error o no fue exitoso
}

```



```
}
```

```
}
```

CODIFICACIÓN DE CLASES

Se procede a implementar los servicios, para ello se crea una nueva clase llamada Service dentro del paquete servicio.

```
using System;
using System.Net.Http;
using System.Text;
using System.Threading.Tasks;
using Newtonsoft.Json;
using Newtonsoft.Json.Linq;
using WS_COMERCIALIZADORA_CON.ec.edu.monster.modelo;

namespace WS_COMERCIALIZADORA_CON.ec.edu.monster.vista
{
    internal class FacturaCredito
    {
        private readonly Telefono telefonoSeleccionado;
        private JObject facturaGenerada;
        private JArray tablaAmortizacion;

        1 referencia
        public FacturaCredito(Telefono telefono)
        {
            telefonoSeleccionado = telefono;
        }

        1 referencia
        public async Task GenerarFactura()
        {
            Console.Clear();
            Console.WriteLine("Factura - Pago a Crédito");
            Console.WriteLine($"Teléfono Seleccionado: {telefonoSeleccionado.Modelo}, Marca: {telefonoSeleccionado.Marca}");
            Console.WriteLine($"Precio: {telefonoSeleccionado.Precio:C}\n");

            // Solicitar datos al usuario
            Console.Write("Ingrese el nombre del cliente: ");
            string clienteNombre = Console.ReadLine();

            Console.Write("Ingrese la cédula del cliente: ");
            string cedula = Console.ReadLine();

            Console.Write("Ingrese el código del cliente: ");
            string codClienteInput = Console.ReadLine();
        }
    }
}
```

Figura 95 Creación clase Factura Crédito

Tabla 29 Clase llamada Factura Crédito

```
using System;

using System.Net.Http;

using System.Text;

using System.Threading.Tasks;

using Newtonsoft.Json;

using Newtonsoft.Json.Linq;

using WS_COMERCIALIZADORA_CON.ec.edu.monster.modelo;
```

```

namespace WS_COMERCIALIZADORA_CON.ec.edu.monster.vista
{
    internal class FacturaCredito
    {
        private readonly Telefono telefonoSeleccionado;

        private JObject facturaGenerada;

        private JArray tablaAmortizacion;

        public FacturaCredito(Telefono telefono)
        {
            telefonoSeleccionado = telefono;
        }

        public async Task GenerarFactura()
        {
            Console.Clear();

            Console.WriteLine("Factura - Pago a Crédito");

            Console.WriteLine($"Teléfono Seleccionado: {telefonoSeleccionado.Modelo}, Marca:
{telefonoSeleccionado.Marca}");

            Console.WriteLine($"Precio: {telefonoSeleccionado.Precio:C}\n");

            // Solicitar datos al usuario

            Console.Write("Ingrese el nombre del cliente: ");

            string clienteNombre = Console.ReadLine();

```

```

Console.Write("Ingrese la cédula del cliente: ");

string cedula = Console.ReadLine();


Console.Write("Ingrese el código del cliente: ");

string codClienteInput = Console.ReadLine();


Console.Write("Ingrese el número de cuotas: ");

string numeroCuotasInput = Console.ReadLine();


// Validar entradas

if (string.IsNullOrEmpty(clienteNombre) || string.IsNullOrEmpty(cedula) ||

    string.IsNullOrEmpty(codClienteInput) || string.IsNullOrEmpty(numeroCuotasInput)

||

    !int.TryParse(codClienteInput, out int codCliente) || !int.TryParse(numeroCuotasInput, out int

numeroCuotas) ||

    codCliente <= 0 || numeroCuotas <= 0)

{

    Console.WriteLine("\nError: Todos los campos son obligatorios y deben tener valores válidos.");

    Console.WriteLine("Presione cualquier tecla para regresar...");

    Console.ReadKey();

    return;

}


try

{

    // Crear el objeto de la solicitud

```

```

var compraRequest = new

{

    Cedula = cedula,

    CodCliente = codCliente,

    Valor = telefonoSeleccionado.Precio,

    NumeroCuotas = numeroCuotas,

    Nombre = clienteNombre

};


// Serializar el objeto a JSON

var jsonContent = JsonConvert.SerializeObject(compraRequest);

var content = new StringContent(jsonContent, Encoding.UTF8, "application/json");


// Enviar la solicitud POST

using (var client = new HttpClient())

{

    string url = "http://192.168.137.1:5012/compras/credito"; // URL de la API

    var response = await client.PostAsync(url, content);

    if (response.IsSuccessStatusCode)

    {

        // Leer y procesar la respuesta exitosa

        var responseBody = await response.Content.ReadAsStringAsync();

        var responseData = JObject.Parse(responseBody);
    }
}

```

```

        // Extraer datos de la respuesta

        string mensaje = responseData["mensaje"]?.ToString();

        facturaGenerada = (JsonObject)responseData["factura"];

        tablaAmortizacion = (JArray)responseData["tablaAmortizacion"];

        Console.WriteLine("\n¡Compra con crédito realizada exitosamente!");

        Console.WriteLine($"Mensaje: {mensaje}");

        Console.WriteLine("\nPresione cualquier tecla para ver la factura y la tabla de
amortización...");

        Console.ReadKey();

        MostrarFacturaYTabla();

    }

    else

    {

        // Manejo de errores de la API

        var errorResponse = await response.Content.ReadAsStringAsync();

        Console.WriteLine($"Error al realizar la compra:\n{errorResponse}");

    }

}

}

}

catch (Exception ex)

{

    Console.WriteLine($"Ocurrió un error inesperado:\n{ex.Message}");

}

```

```

        Console.WriteLine("\nPresione cualquier tecla para regresar...");

        Console.ReadKey();

    }

    private void MostrarFacturaYTabla()

    {

        if (facturaGenerada == null || tablaAmortizacion == null)

        {

            Console.WriteLine("No hay datos para mostrar.");

            return;

        }

        try

        {

            // Mostrar detalles de la factura

            Console.Clear();

            Console.WriteLine("Factura:");

            Console.WriteLine($"ID: {facturaGenerada["idFactura"]}");

            Console.WriteLine($"Fecha: {facturaGenerada["fecha"]}");

            Console.WriteLine($"Cliente: {facturaGenerada["clienteNombre"]}");

            Console.WriteLine($"Total: {facturaGenerada["total"]:C}");

            // Mostrar tabla de amortización

            Console.WriteLine("\nTabla de Amortización:");

            foreach (var cuota in tablaAmortizacion)

```

```

    {

        Console.WriteLine(

            $"Cuota {cuota["numeroCuota"]}: Valor Cuota: {cuota["valorCuota"]:C}, " +

            $"Capital: {cuota["capitalPagado"]:C}, Interés: {cuota["interesPagado"]:C}, " +

            $"Saldo: {cuota["saldo"]:C}");

    }

}

catch (Exception ex)

{

    Console.WriteLine($"Error al procesar los detalles de la factura y la tabla:\n{ex.Message}");

}

}

}

}

```

EJECUCIÓN

Para realizar la ejecución, se debe tener desplegado el servidor, como siguiente paso es necesario en la aplicación cliente y dar click en run

```

+-----+
--- Cliente de Consola (02CLICON) ---
Por favor, inicie sesión para continuar.
Usuario: MONSTER
Contraseña: MONSTER9

¡Acceso concedido! Bienvenido a la Comercializadora.

```

Figura 95 Primera Vista Inicio de Sesión

```
===== MENÚ PRINCIPAL =====  
1. Ver Catálogo de Electrodomésticos  
2. Registrar Cliente en Tienda  
3. Realizar una Compra  
4. Consultar Tabla de Amortización  
5. Buscar Factura por ID (NUEVO)  
0. Salir
```

Figura 95 Vista menú principal

```
Seleccione una opción: 1  
  
--- Obteniendo Catálogo de Productos ---  
ID: 1 | Producto: Refrigeradora | Precio: $750.00 | Stock: 9  
ID: 2 | Producto: Cocina de Inducción | Precio: $500.00 | Stock: 19  
ID: 3 | Producto: Lavadora | Precio: $450.00 | Stock: 9  
ID: 4 | Producto: Microondas | Precio: $150.00 | Stock: 30  
ID: 5 | Producto: PlayStation 5 | Precio: $800.00 | Stock: 9  
ID: 6 | Producto: Laptop HP | Precio: $650.00 | Stock: 4  
ID: 7 | Producto: Licuadora Oster | Precio: $80.00 | Stock: 19  
ID: 8 | Producto: Smart TV 55 4K | Precio: $450.00 | Stock: 10  
ID: 9 | Producto: PlayStation 4 | Precio: $250.00 | Stock: 16  
  
(Presione Enter para volver al menú...)
```

Figura 95 Catálogo de teléfonos


```
--- Nueva Compra ---  
Ingrese Cédula del cliente: 1718972597  
  
¡Bienvenido, Ednan!  
(Cargando catálogo actualizado...)  
ID del Electrodomestico (0 para finalizar): 1  
-> Producto: Refrigeradora (Precio: $750.00) - Cantidad: 1  
    [Añadido al carrito]  
ID del Electrodomestico (0 para finalizar): 0  
  
Forma de Pago (Efectivo / Credito):  
Procesando factura con el servidor...  
  
✓ VENTA EXITOSA - Factura #14  
Total a Pagar: $750.00
```

Figura 101 Menú opciones de compra

```
--- Nueva Compra ---  
Ingrese Cédula del cliente: 1718972597  
  
¡Bienvenido, Ednan!  
(Cargando catálogo actualizado...)  
ID del Electrodomestico (0 para finalizar): 1  
-> Producto: Refrigeradora (Precio: $750.00) - Cantidad: 1  
    [Añadido al carrito]  
ID del Electrodomestico (0 para finalizar): 0  
  
Forma de Pago (Efectivo / Credito):  
Procesando factura con el servidor...  
  
✓ VENTA EXITOSA - Factura #14  
Total a Pagar: $750.00
```

Figura 104 Opción pago en efectivo

```

--- Nueva Compra ---
Ingrese Cédula del cliente: 1718972597

¡Bienvenido, Ednan!
(Cargando catálogo actualizado...)
ID del Electrodomestico (0 para finalizar): 1
-> Producto: Refrigeradora (Precio: $750.00) - Cantidad: 1
    [Añadido al carrito]
ID del Electrodomestico (0 para finalizar): 0

Forma de Pago (Efectivo / Credito):
Procesando factura con el servidor...

✓ VENTA EXITOSA - Factura #14
Total a Pagar: $750.00

```

Figura 105 Opción pago transferencia exitoso

```

--- Consultar Tabla de Amortización ---
ID Crédito: 2

```

Cuota	Fecha	Valor	Interés	Saldo
1	2026-01-16	\$68,05	\$10,00	\$691,95
2	2026-02-16	\$68,05	\$9,23	\$633,13
3	2026-03-16	\$68,05	\$8,44	\$573,52
4	2026-04-16	\$68,05	\$7,65	\$513,11
5	2026-05-16	\$68,05	\$6,84	\$451,91
6	2026-06-16	\$68,05	\$6,03	\$389,88
7	2026-07-16	\$68,05	\$5,20	\$327,03
8	2026-08-16	\$68,05	\$4,36	\$263,34
9	2026-09-16	\$68,05	\$3,51	\$198,80
10	2026-10-16	\$68,05	\$2,65	\$133,40
11	2026-11-16	\$68,05	\$1,78	\$67,13
12	2026-12-16	\$68,05	\$0,90	\$0,00

Figura 102 Factura y Tabla de amortización

APLICACIÓN CLIENTE

CREACIÓN DEL PROYECTO WEB

Como tercer paso, se crea el proyecto web que permite un funcionamiento multiplataforma.

Se deben tener en cuenta los siguientes pasos para desarrollar el proyecto.

Figura 102 Creación directorio 02.CLIENTE WEB

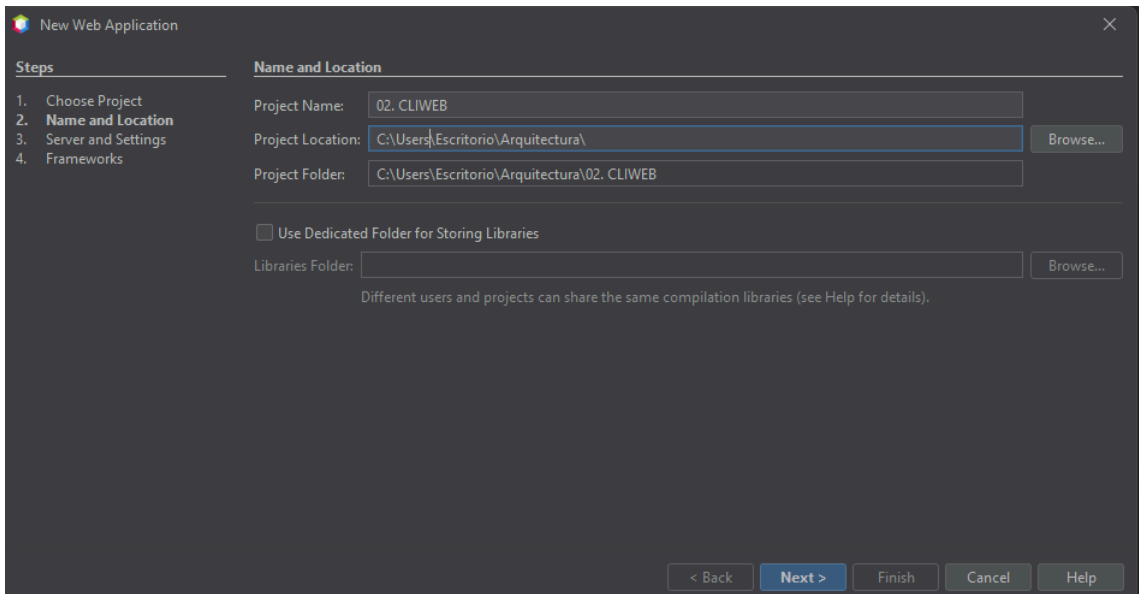


Figura 103 Creación directorio WSTelefonoClient

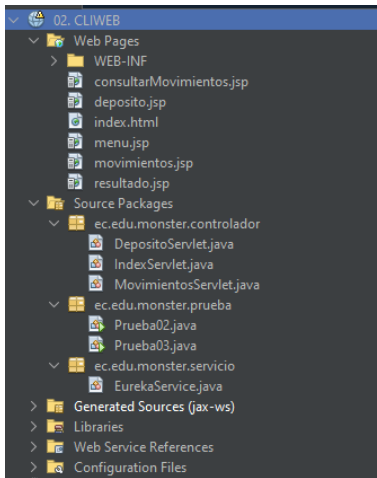


Figura 104 Creación estructura proyecto Web

Tabla 18 Codificación Vista Principal

```
<!DOCTYPE html>
<html lang="es">
```

```

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Tu Aplicación - Administrador de Movimientos</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      margin: 0;
      padding: 0;
      background-image:
url('https://d23.com/app/uploads/2013/04/1180w-600h_a-to-z-monsters-inc-1180x600.jpg'
); /* Reemplaza 'ruta/de/la/imagen.jpg' con la URL o la ruta de tu imagen */
      background-size: cover;
      background-position: center;
      background-repeat: no-repeat;
      height: 100vh;
    }

    header {
      background-color: #333;
      color: white;
      padding: 10px 0;
      text-align: center;
    }

    nav {
      background-color: #666;
      padding: 10px 0;
      text-align: center;
    }

    nav ul {
      list-style-type: none;
      margin: 0;
      padding: 0;
    }

    nav ul li {
      display: inline;
      margin-right: 20px;
    }

    nav ul li a {
      color: white;
      text-decoration: none;
    }

    nav ul li a:hover {
      color: #FFA500;
    }
  </style>

```

```

</head>
<body>
  <header>
    <h1> BANQUITO </h1>
  </header>
  <nav>
    <ul>
      <li><a
href="${pageContext.request.contextPath}/consultarMovimientos.jsp">Consultar
Movimientos</a></li>
      <li><a href="${pageContext.request.contextPath}/deposito.jsp">Registrar
Depósito</a></li>
      <!-- Puedes agregar más enlaces aquí según sea necesario -->
    </ul>
  </nav>
  <!-- Aquí puedes agregar el contenido principal de tu página -->
</body>
</html>

```

Tabla 19 Codificación consultarMovimientos

```

<%--
  Document   : consultarMovimientos
  Created on : 9 jun 2024, 15:04:09
  Author    : andre
--%>

<%@ page language="java" contentType="text/html; charset=UTF-8"
pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <title>Consultar Movimientos</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      margin: 0;
      padding: 0;
      background-image:
url('https://d23.com/app/uploads/2013/04/1180w-600h_a-to-z-monsters-inc-1180x600.jpg'
);
      background-size: cover;
      background-position: center;
      background-repeat: no-repeat;
      height: 100vh;
    }

    header {
      background-color: #333;
      color: white;

```

```

padding: 10px 0;
text-align: center;
}

nav {
background-color: #666;
padding: 10px 0;
text-align: center;
}

nav ul {
list-style-type: none;
margin: 0;
padding: 0;
}

nav ul li {
display: inline;
margin-right: 20px;
}

nav ul li a {
color: white;
text-decoration: none;
}

nav ul li a:hover {
color: #FFA500;
}

.container {
display: flex;
flex-direction: column;
align-items: center;
justify-content: flex-start;
height: 100%;
}

form {
background-color: rgba(255, 255, 255, 0.8);
padding: 20px;
border-radius: 10px;
text-align: center;
}

form label {
display: block;
margin-bottom: 10px;
}

form input[type="text"] {

```

```

        width: 300px;
        padding: 10px;
        margin-bottom: 10px;
        border: 1px solid #ccc;
        border-radius: 5px;
    }

    form input[type="submit"] {
        padding: 10px 20px;
        background-color: #333;
        color: white;
        border: none;
        border-radius: 5px;
        cursor: pointer;
    }

    form input[type="submit"]:hover {
        background-color: #111;
    }
</style>
</head>
<body>
    <header>
        <h1>Consultar Movimientos</h1>
    </header>
    <nav>
        <ul>
            <li><a
href="\${pageContext.request.contextPath}/consultarMovimientos.jsp">Consultar
Movimientos</a></li>
            <li><a href="\${pageContext.request.contextPath}/deposito.jsp">Registrar
Depósito</a></li>
            <!-- Puedes agregar más enlaces aquí según sea necesario -->
        </ul>
    </nav>
    <div class="container">
        <br></br><br></br><br></br><br></br>
        <form action="MovimientosServlet" method="post">
            <label for="cuenta">Número de Cuenta:</label>
            <input type="text" id="cuenta" name="cuenta" required><br><br>
            <input type="submit" value="Consultar Movimientos">
        </form>
    </div>
</body>
</html>

```

Tabla 20 Codificación deposito

```

<%--
Document : deposito
Created on : 8 jun 2024, 20:22:29

```

```

    Author    : andre
--%>

<%@    page        language="java"        contentType="text/html;        charset=UTF-8"
pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
    <meta charset="UTF-8">
    <title>Registrar Depósito</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            background-image:
url('https://d23.com/app/uploads/2013/04/1180w-600h_a-to-z-monsters-inc-1180x600.jpg'
); /* Cambia esta URL por la de tu imagen */
            background-size: cover;
            background-position: center;
            background-repeat: no-repeat;
            margin: 0;
            color: #333;
            display: flex;
            flex-direction: column;
            min-height: 100vh;
        }

        header {
            background-color: #333;
            color: white;
            padding: 10px 0;
            text-align: center;
        }

        nav {
            background-color: #666;
            padding: 10px 0;
            text-align: center;
        }

        nav ul {
            list-style-type: none;
            margin: 0;
            padding: 0;
        }

        nav ul li {
            display: inline;
            margin-right: 20px;
        }

        nav ul li a {

```



```

    color: white;
    text-decoration: none;
}

nav ul li a: hover {
    color: #FFA500;
}

.content {
    flex: 1;
    display: flex;
    justify-content: center;
    align-items: center;
    padding: 20px;
}

form {
    background-color: rgba(255, 255, 255, 0.9);
    padding: 20px;
    border-radius: 10px;
    box-shadow: 0 0 15px rgba(0, 0, 0, 0.5);
    max-width: 400px;
    width: 100%;
    margin: auto;
}

label {
    display: block;
    margin-bottom: 10px;
    font-weight: bold;
}

input[type="text"],
input[type="number"] {
    width: calc(100% - 22px);
    padding: 10px;
    margin-bottom: 20px;
    border: 1px solid #ccc;
    border-radius: 5px;
    display: block;
}

input[type="submit"] {
    width: 100%;
    padding: 10px;
    background-color: #4CAF50;
    color: white;
    border: none;
    border-radius: 5px;
    font-size: 16px;
    cursor: pointer;
}

```

```

        transition: background-color 0.3s ease;
    }

    input[type="submit"]:hover {
        background-color: #45a049;
    }

    .back-link {
        display: block;
        text-align: center;
        margin-top: 20px;
        color: white;
        text-decoration: none;
        font-weight: bold;
        padding: 10px 20px;
        background-color: #333;
        border-radius: 5px;
        transition: background-color 0.3s;
    }

    .back-link:hover {
        background-color: #111;
    }

    .no-spinners::-webkit-outer-spin-button,
    .no-spinners::-webkit-inner-spin-button {
        -webkit-appearance: none;
        margin: 0;
    }
</style>
</head>
<body>
<header>
    <h1>Realizar Depósito</h1>
</header>
<nav>
    <ul>
        <li><a href="{pageContext.request.contextPath}/consultarMovimientos.jsp">Consultar
Movimientos</a></li>
        <li><a href="{pageContext.request.contextPath}/deposito.jsp">Registrar
Depósito</a></li>
        <!-- Puedes agregar más enlaces aquí según sea necesario -->
    </ul>
</nav>
<div class="content">
    <form action="DepositoServlet" method="post">
        <label for="cuenta">Número de Cuenta:</label>
        <input type="text" id="cuenta" name="cuenta" required>

        <label for="importe">Importe:</label>

```

```

        <input type="number" step="0.01" id="importe" name="importe" required
class="no-spinners">

        <input type="submit" value="Registrar Depósito">
        <br/>
    </form>
</div>
</body>
</html>

```

Tabla 21 Codificación movimientos

```

<%--
    Document   : movimientos
    Created on : 8 jun 2024, 20:23:54
    Author    : andre
--%>

<%@ page language="java" contentType="text/html; charset=UTF-8"
pageEncoding="UTF-8"%>
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<!DOCTYPE html>
<html>
<head>
    <meta charset="UTF-8">
    <title>Movimientos de la Cuenta</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            margin: 0;
            padding: 0;
            background-image:
url('https://d23.com/app/uploads/2013/04/1180w-600h_a-to-z-monsters-inc-1180x600.jpg'
); /* Reemplaza 'ruta/de/la/imagen.jpg' con la URL o la ruta de tu imagen de fondo */
            background-size: cover;
            background-position: center;
            background-repeat: no-repeat;
            color: #333;
        }
        h1 {
            text-align: center;
            margin-top: 20px;
        }
        table {
            width: 80%;
            margin: 20px auto;
            border-collapse: collapse;
            background-color: #fff;
            box-shadow: 0 0 20px rgba(0, 0, 0, 0.1);
        }
        th, td {

```

```

padding: 12px 15px;
border: 1px solid #ddd;
text-align: left;
}
th {
background-color: #333;
color: #fff;
}
tr:nth-child(even) {
background-color: #f2f2f2;
}
.error-message {
color: red;
text-align: center;
margin-top: 20px;
}
.empty-message {
text-align: center;
margin-top: 20px;
}
.link {
display: inline-block;
text-align: center;
margin-top: 20px;
padding: 10px 20px;
background-color: #333;
color: #fff;
text-decoration: none;
border-radius: 5px;
transition: background-color 0.3s;
}
.link:hover {
background-color: #111;
}

header {
background-color: #333;
color: white;
padding: 10px 0;
text-align: center;
}
</style>
</head>
<body>
<header> <h2> Movimientos de la Cuenta </h2></header>

<c:if test="\${not empty error}">
  <p class="error-message">\${error}</p>
</c:if>

<c:if test="\${not empty movimientos}">

```

```

<table>
  <thead>
    <tr>
      <th>Cuenta</th>
      <th>NroMov</th>
      <th>Fecha</th>
      <th>Tipo</th>
      <th>Accion</th>
      <th>Importe</th>
    </tr>
  </thead>
  <tbody>
    <c:forEach var="movimiento" items="{movimientos}">
      <tr>
        <td>${movimiento.cuenta}</td>
        <td>${movimiento.nroMov}</td>
        <td>${movimiento.fecha}</td>
        <td>${movimiento.tipo}</td>
        <td>${movimiento.accion}</td>
        <td>${movimiento.importe}</td>
      </tr>
    </c:forEach>
  </tbody>
</table>
</c:if>

<c:if test="{empty movimientos}">
  <p class="empty-message">No se encontraron movimientos para la cuenta
proporcionada.</p>
</c:if>

<a href="{pageContext.request.contextPath}/consultarMovimientos.jsp" class="link">Consultar otra
cuenta</a>
</body>
</html>

```

Tabla 22 Codificación resultado

```

<%--
  Document   : resultado
  Created on : 9 jun 2024, 15:10:09
  Author    : andre
--%>

<%@ page language="java" contentType="text/html; charset=UTF-8"
pageEncoding="UTF-8"%>
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>
<!DOCTYPE html>
<html>
<head>

```

```

<meta charset="UTF-8">
<title>Resultado del Depósito</title>
<style>
  body {
    font-family: Arial, sans-serif;
    background-image:
url('https://d23.com/app/uploads/2013/04/1180w-600h_a-to-z-monsters-inc-1180x600.jpg'
); /* Cambia esta URL por la de tu imagen */
    background-size: cover; /* La imagen se ajusta al tamaño de la pantalla */
    background-position: center;
    background-repeat: no-repeat;
    background-attachment: fixed; /* Fija la imagen de fondo mientras se desplaza */
    margin: 0;
    padding: 0;
  }

```

```

.container {
  max-width: 600px;
  margin: 50px auto;
  padding: 20px;
  background-color: rgba(255, 255, 255, 0.9);
  border-radius: 5px;
  box-shadow: 0 0 10px rgba(0, 0, 0, 0.1);
}

```

```

h1 {
  text-align: center;
  color: #333;
}

```

```

p {
  text-align: center;
  color: #333;
  margin-bottom: 20px;
}

```

```

a {
  display: block;
  text-align: center;
  color: #fff;
  background-color: #007bff;
  padding: 10px 20px;
  text-decoration: none;
  border-radius: 5px;
  transition: background-color 0.3s;
  width: fit-content;
  margin: 0 auto;
}

```

```

a:hover {

```

```

        background-color: #0056b3;
    }
</style>
</head>
<body>
    <div class="container">
        <h1>Resultado del Depósito</h1>
        <c:choose>
            <c:when test="{resultado == 1}">
                <p>El depósito se ha realizado con éxito.</p>
            </c:when>
            <c:otherwise>
                <p>Hubo un error al realizar el depósito.</p>
            </c:otherwise>
        </c:choose>
        <a href="deposito.jsp">Realizar otro depósito</a>
    </div>
</body>
</html>

```

APLICACIÓN CLIENTE

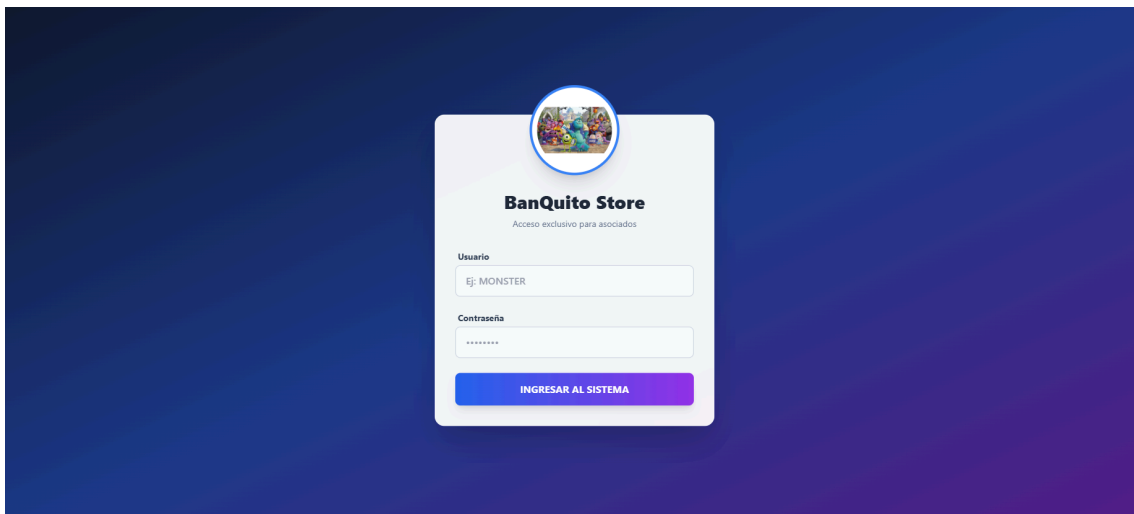


Figura 108 Inicio de sesión

Nuevo Producto

Nombre

Precio

Stock

URL Imagen

https://...

Descripción

CREAR PRODUCTO

Producto
\$0

Inventario Actual (10)

ID	PRODUCTO	PRECIO	STOCK	ACCIONES
#1	Refrigeradora	\$750	9 unid.	Editar
#2	Cocina de Inducción	\$500	19 unid.	Editar

Figura 109 CRUD de teléfonos

Nuevo Producto

Nombre

Precio

Stock

URL Imagen

https://...

Descripción

CREAR PRODUCTO

Producto
\$0

Figura 110 Agregar Teléfono

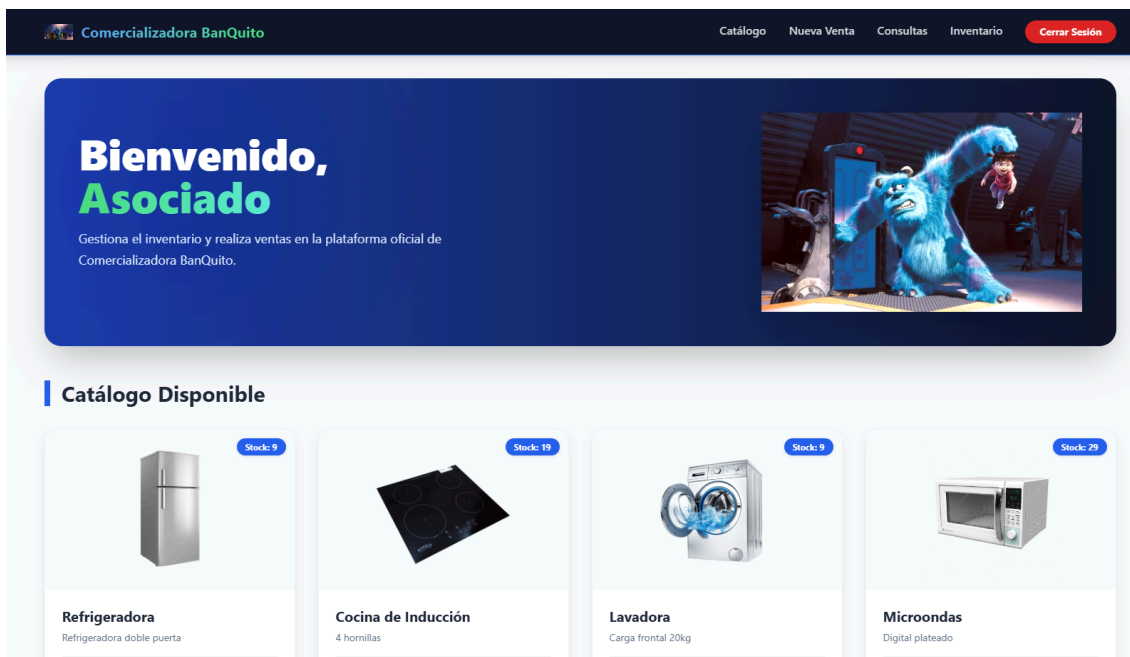


Figura 111 Prueba a la interfaz gráfica principal

Centro de Consultas

[Consultar Factura](#)[Tabla de Amortización](#)Buscar

Plan de Pagos (Crédito)

Cuota	Fecha Pago	Valor Cuota	Interés	Saldo Capital
1	16/1/2026	\$68.05	\$10	\$691.95
2	16/2/2026	\$68.05	\$9.23	\$633.13
3	16/3/2026	\$68.05	\$8.44	\$573.52
4	16/4/2026	\$68.05	\$7.65	\$513.11
5	16/5/2026	\$68.05	\$6.84	\$451.91
6	16/6/2026	\$68.05	\$6.03	\$389.88
7	16/7/2026	\$68.05	\$5.2	\$327.03
8	16/8/2026	\$68.05	\$4.36	\$263.34

Figura 112 Prueba a la interfaz gráfica ver amortización

Catálogo Disponible



Stock: 9

Refrigeradora
Refrigeradora doble puerta

PRECIO
\$750



Stock: 19

Cocina de Inducción
4 hornillas

PRECIO
\$500



Stock: 9

Lavadora
Carga frontal 20kg

PRECIO
\$450



Stock: 29

Microondas
Digital plateado

PRECIO
\$150



Stock: 7

PlayStation 5
Consola Sony 825GB



Stock: 8

Laptop HP
Core i5 8GB RAM



Stock: 19

Licuadora Oster
3 Velocidades



Stock: 16

Smart TV Sony 55 4K
Televisor Ultra HD con Android TV

Figura 113 Interfaz gráfica de catálogo

Datos del Cliente

Buscar

☒ Ednan Merino

jos@mail.com

Agregar Productos

PRODUCTO

Laptop HP - \$650 (Stock: 8)

CANTIDAD

1

+ Añadir

Carrito de Compra

2 items

Cocina de Inducción

1 x \$500

\$500.00

Laptop HP

1 x \$650

\$650.00

FORMA DE PAGO

☒ Efectivo (33% Descuento)

Total Estimado

\$1150.00

CONFIRMAR VENTA

Figura 114 Carrito de productos

Carrito de Compra

2 items

Cocina de Inducción

1 x \$500

\$500.00

Laptop HP

1 x \$650

\$650.00

FORMA DE PAGO

☒ Efectivo (33% Descuento)

☒ Efectivo (33% Descuento)

☐ Crédito Diferido

CONFIRMAR VENTA

Figura 115 Forma de Compra

CREACIÓN DEL PROYECTO MÓVIL

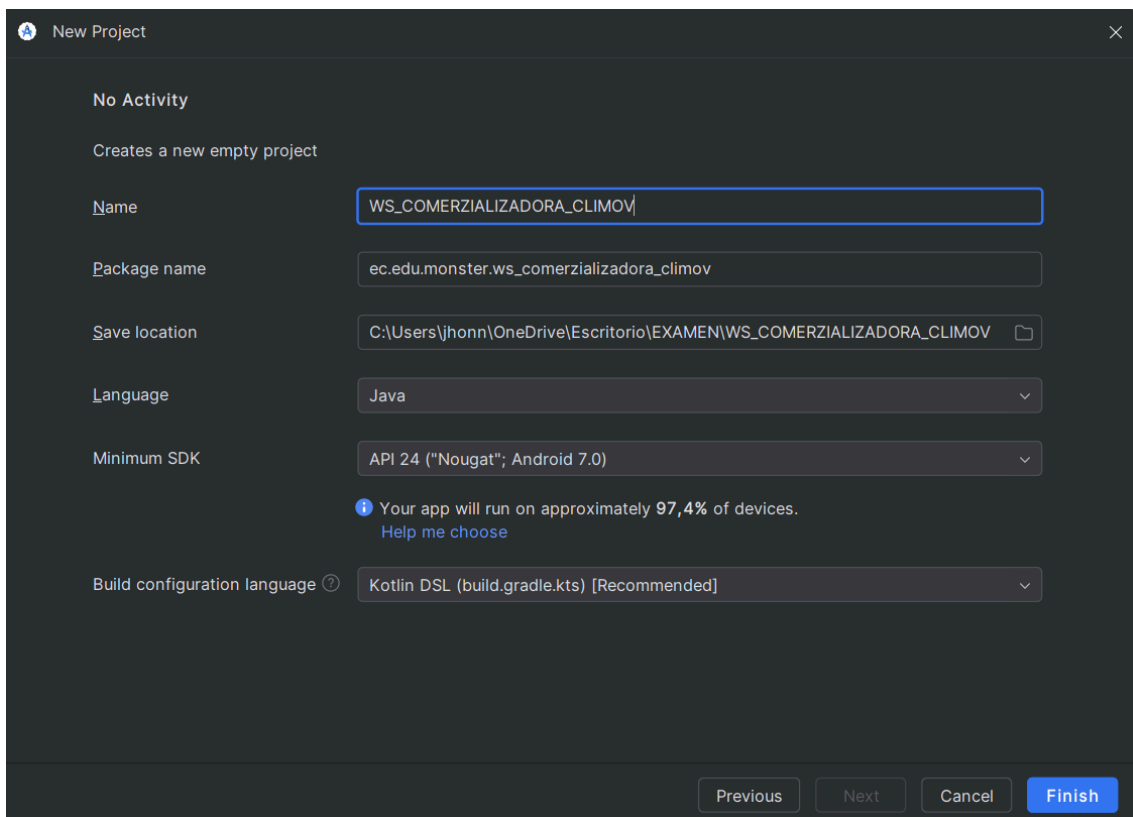


Figura 108 Configuración Nuevo Proyecto

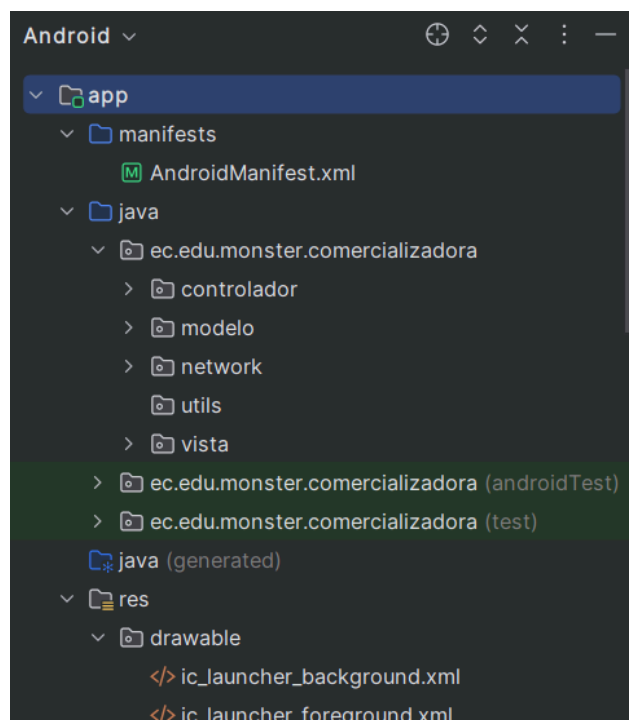


Figura 109 Estructura proyecto

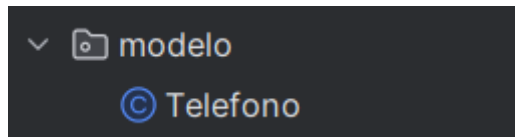


Figura 110 Creación nueva clase en paquete modelo

```
package ec.edu.monster.comercializadora.modelo;

import java.io.Serializable;

public class Telefono implements Serializable { 20 usages
    private int idTelefono; 2 usages
    private String modelo; 2 usages
    private String marca; 2 usages
    private double precio; 2 usages
    private String descripcion; 2 usages

    // Getters y setters
    public int getIdTelefono() { return idTelefono; } no usages
    public void setIdTelefono(int idTelefono) { this.idTelefono = idTelefono; } 1 usage
    public String getModelo() { return modelo; } 3 usages
    public void setModelo(String modelo) { this.modelo = modelo; } 1 usage
    public String getMarca() { return marca; } 1 usage
    public void setMarca(String marca) { this.marca = marca; } 1 usage
    public double getPrecio() { return precio; } 7 usages
    public void setPrecio(double precio) { this.precio = precio; } 1 usage
    public String getDescripcion() { return descripcion; } 1 usage
    public void setDescripcion(String descripcion) { this.descripcion = descripcion; } 1 usage
}
```

Figura 111 Codificación clase Teléfono

Tabla 27 codificación clase Teléfono

```
package ec.edu.monster.comercializadora.modelo;

import java.io.Serializable;

public class Telefono implements Serializable {
    private int idTelefono;
    private String modelo;
    private String marca;
    private double precio;
    private String descripcion;

    // Getters y setters
```

```

public int getIdTelefono() { return idTelefono; }
public void setIdTelefono(int idTelefono) { this.idTelefono = idTelefono; }
public String getModelo() { return modelo; }
public void setModelo(String modelo) { this.modelo = modelo; }
public String getMarca() { return marca; }
public void setMarca(String marca) { this.marca = marca; }
public double getPrecio() { return precio; }
public void setPrecio(double precio) { this.precio = precio; }
public String getDescripcion() { return descripcion; }
public void setDescripcion(String descripcion) { this.descripcion = descripcion; }
}

```

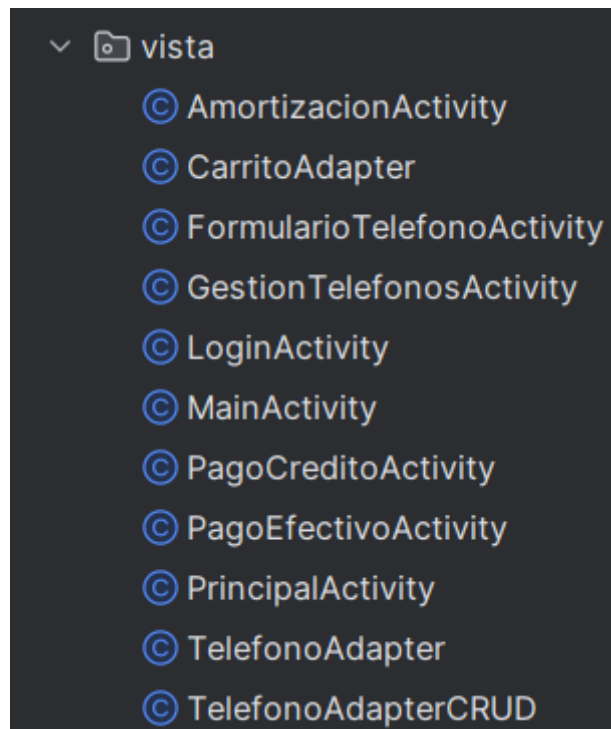


Figura 112 Creación del paquete vista

Tabla 28 Codificación clases

```

package ec.edu.monster.servicio;

import android.os.AsyncTask;
import org.ksoap2.SoapEnvelope;
import org.ksoap2.serialization.SoapObject;
import org.ksoap2.serialization.SoapSerializationEnvelope;
import org.ksoap2.transport.HttpTransportSE;

import java.util.ArrayList;
import java.util.List;

import ec.edu.monster.modelo.Movimiento;

```

```

public class BanquitoServicio {

    private static final String NAMESPACE = "http://ws.monster.edu.ec/";
    private static final String URL =
"http://192.168.19.156:8080/WS_BANQUITO_SOAP_JAVA/WSBanquito";

    // Interface for callback
    public interface ServicioCallback<T> {
        void onSuccess(T result);
        void onFailure(Exception e);
    }

    // AsyncTask for traerMovimientos
    public void traerMovimientosAsync(String cuenta, final ServicioCallback<List<Movimiento>> callback) {
        new TraerMovimientosTask(callback).execute(cuenta);
    }

    // AsyncTask for regDeposito
    public void regDepositoAsync(String cuenta, double importe, final ServicioCallback<Integer> callback) {
        new RegDepositoTask(callback).execute(cuenta, importe);
    }

    // AsyncTask for validarLogin
    public void validarLoginAsync(String cuenta, String contrasena, final ServicioCallback<Integer>
callback) {
        new ValidarLoginTask(callback).execute(cuenta, contrasena);
    }

    // Static AsyncTask for traerMovimientos
    private static class TraerMovimientosTask extends AsyncTask<String, Void, List<Movimiento>> {
        private ServicioCallback<List<Movimiento>> callback;

        public TraerMovimientosTask(ServicioCallback<List<Movimiento>> callback) {
            this.callback = callback;
        }

        @Override
        protected List<Movimiento> doInBackground(String... params) {
            List<Movimiento> movimientos = new ArrayList<>();
            String cuenta = params[0];
            String METHOD_NAME = "traerMovimientos";
            String SOAP_ACTION = NAMESPACE + METHOD_NAME;

            try {
                SoapObject request = new SoapObject(NAMESPACE, METHOD_NAME);
                request.addProperty("cuenta", cuenta);

                SoapSerializationEnvelope envelope = new SoapSerializationEnvelope(SoapEnvelope.VER11);
                envelope.setOutputSoapObject(request);
            }
        }
    }
}

```

```

        HttpTransportSE transport = new HttpTransportSE(URL);
        transport.call(SOAP_ACTION, envelope);

        SoapObject response = (SoapObject) envelope.getResponse();
        for (int i = 0; i < response.getPropertyCount(); i++) {
            SoapObject movimientoSoap = (SoapObject) response.getProperty(i);
            Movimiento movimiento = new Movimiento();
            movimiento.setFecha(movimientoSoap.getProperty("fecha").toString());

            movimiento.setImporte(Double.parseDouble(movimientoSoap.getProperty("importe").toString()));
            movimientos.add(movimiento);
        }
    } catch (Exception e) {
        return null;
    }
    return movimientos;
}

@Override
protected void onPostExecute(List<Movimiento> result) {
    if (result != null) {
        callback.onSuccess(result);
    } else {
        callback.onFailure(new Exception("Error retrieving movements"));
    }
}
}

// Static AsyncTask for regDeposito
private static class RegDepositoTask extends AsyncTask<Object, Void, Integer> {
    private ServicioCallback<Integer> callback;

    public RegDepositoTask(ServicioCallback<Integer> callback) {
        this.callback = callback;
    }

    @Override
    protected Integer doInBackground(Object... params) {
        String cuenta = (String) params[0];
        double importe = (Double) params[1];
        String METHOD_NAME = "regDeposito";
        String SOAP_ACTION = NAMESPACE + METHOD_NAME;

        try {
            SoapObject request = new SoapObject(NAMESPACE, METHOD_NAME);

            request.addProperty("cuenta", String.valueOf(cuenta));
            request.addProperty("importe", Double.toString(importe));

```



```

        SoapSerializationEnvelope envelope = new SoapSerializationEnvelope(SoapEnvelope.VER12);
        envelope.setOutputSoapObject(request);

        HttpTransportSE transport = new HttpTransportSE(URL);
        transport.call(SOAP_ACTION, envelope);

        return Integer.parseInt(envelope.getResponse().toString());
    } catch (Exception e) {
        return -1;
    }
}

@Override
protected void onPostExecute(Integer result) {
    callback.onSuccess(result);
}
}

// Static AsyncTask for validarLogin
private static class ValidarLoginTask extends AsyncTask<Object, Void, Integer> {
    private ServicioCallback<Integer> callback;

    public ValidarLoginTask(ServicioCallback<Integer> callback) {
        this.callback = callback;
    }

    @Override
    protected Integer doInBackground(Object... params) {
        String cuenta = (String) params[0];
        String contrasena = (String) params[1];
        String METHOD_NAME = "validarLogin";
        String SOAP_ACTION = NAMESPACE + METHOD_NAME;

        try {
            SoapObject request = new SoapObject(NAMESPACE, METHOD_NAME);
            request.addProperty("cuenta", cuenta);
            request.addProperty("contrasena", contrasena);

            SoapSerializationEnvelope envelope = new SoapSerializationEnvelope(SoapEnvelope.VER11);
            envelope.setOutputSoapObject(request);

            HttpTransportSE transport = new HttpTransportSE(URL);
            transport.call(SOAP_ACTION, envelope);

            return Integer.parseInt(envelope.getResponse().toString());
        } catch (Exception e) {
            return -1;
        }
    }
}

```

```

@Override
protected void onPostExecute(Integer result) {
    callback.onSuccess(result);
}
}
}

```

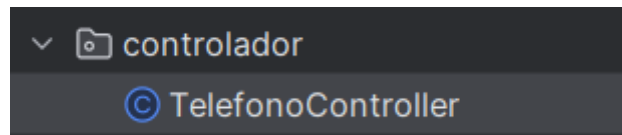


Figura 113 Creacion Controlador

Tabla 29 Codificación clase TelefonoController

```

package ec.edu.monster.comercializadora.controlador;

import android.content.Context;
import android.util.Log;

import ec.edu.monster.comercializadora.network.VolleyHandler;
import ec.edu.monster.comercializadora.vista.MainActivity;
import ec.edu.monster.comercializadora.modelo.Telefono;

import org.json.JSONArray;
import org.json.JSONObject;

import java.util.ArrayList;
import java.util.List;

public class TelefonoController {

    private MainActivity view;
    private Context context;

    public TelefonoController(MainActivity view, Context context) {
        this.view = view;
        this.context = context;
    }

    public void cargarTelefonos() {
        String url = "http://192.168.137.1:5012/telefonos";
        VolleyHandler.getInstance(context).makeGetRequest(url, response -> {
            Log.d("API_RESPONSE", response); // Log de la respuesta
            List<Telefono> telefonos = new ArrayList<>();
            try {
                JSONArray jsonArray = new JSONArray(response);

```

```

        for (int i = 0; i < jsonArray.length(); i++) {
            JSONObject obj = jsonArray.getJSONObject(i);
            Telefono telefono = new Telefono();
            telefono.setIdTelefono(obj.getInt("idTelefono"));
            telefono.setModelo(obj.getString("modelo"));
            telefono.setMarca(obj.getString("marca"));
            telefono.setPrecio(obj.getDouble("precio"));
            telefono.setDescripcion(obj.getString("descripcion"));
            telefonos.add(telefono);
        }
        view.mostrarTelefonos(telefonos);
    } catch (Exception e) {
        Log.e("JSON_ERROR", e.toString()); // Log de errores de parseo
    }
}, error -> {
    Log.e("API_ERROR", error.toString()); // Log de errores de red
});

}
}

```

VISTA

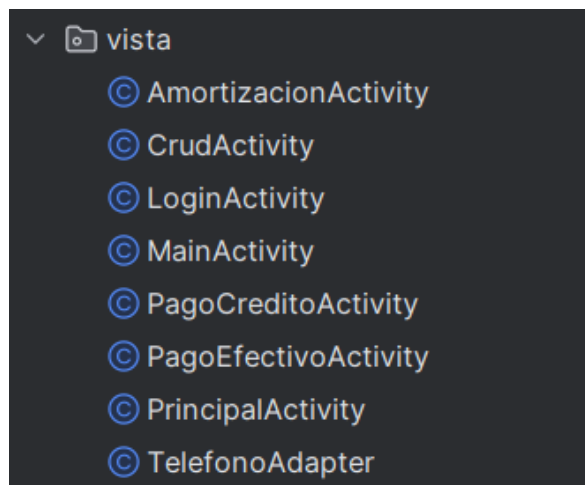


Figura 114 Creación archivo AmortizacionActivity, MainActivity & PagoCreditoActivity, Login

Tabla 30 Codificación archivo PagoCreditoActivity

```

package ec.edu.monster.comercializadora.vista;

import android.app.AlertDialog;
import android.os.Bundle;
import android.util.Log;
import android.view.View;
import android.widget.Button;
import android.widget.EditText;
import android.widget.TextView;
import android.widget.Toast;

```

```

import androidx.appcompat.app.AppCompatActivity;
import com.android.volley.Request;
import com.android.volley.RequestQueue;
import com.android.volley.toolbox.JsonObjectRequest;
import com.android.volley.toolbox.Volley;
import org.json.JSONArray;
import org.json.JSONException;
import org.json.JSONObject;

import ec.edu.monster.comercializadora.R;
import ec.edu.monster.comercializadora.modelo.Telefono;

public class PagoCreditoActivity extends AppCompatActivity {

    private static final String API_URL = "http://192.168.137.1:5012/compras/credito";
    private Button btnMostrarFactura;
    private JSONObject facturaGenerada;
    private JSONArray tablaAmortizacion;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_pago_credito);

        TextView tvResumen = findViewById(R.id.tvResumenCredito);
        EditText etNombreCliente = findViewById(R.id.etNombreClienteCredito);
        EditText etCedulaCliente = findViewById(R.id.etCedulaClienteCredito);
        EditText etCodigoCliente = findViewById(R.id.etCodigoCliente);
        EditText etNumeroCuotas = findViewById(R.id.etNumeroCuotas);
        Button btnConfirmar = findViewById(R.id.btnConfirmarCredito);
        btnMostrarFactura = findViewById(R.id.btnMostrarFactura);

        // Obtener datos del intent
        Telefono telefono = (Telefono) getIntent().getSerializableExtra("telefono");

        // Mostrar resumen del teléfono
        tvResumen.setText(String.format(
            "Modelo: %s\nPrecio: $%.2f",
            telefono.getModelo(), telefono.getPrecio()
        ));

        btnConfirmar.setOnClickListener(v -> {
            String nombreCliente = etNombreCliente.getText().toString().trim();
            String cedulaCliente = etCedulaCliente.getText().toString().trim();
            String codigoClienteStr = etCodigoCliente.getText().toString().trim();
            String numeroCuotasStr = etNumeroCuotas.getText().toString().trim();

            if (nombreCliente.isEmpty()) {
                etNombreCliente.setError("El nombre del cliente es obligatorio.");
                return;
            }
            if (cedulaCliente.isEmpty()) {
                etCedulaCliente.setError("La cédula del cliente es obligatoria.");
                return;
            }
            if (codigoClienteStr.isEmpty()) {
                etCodigoCliente.setError("El código del cliente es obligatorio.");
            }
        });
    }
}

```

```

        return;
    }
    if (numeroCuotasStr.isEmpty()) {
        etNumeroCuotas.setError("El número de cuotas es obligatorio.");
        return;
    }

    int codigoCliente = Integer.parseInt(codigoClienteStr);
    int numeroCuotas = Integer.parseInt(numeroCuotasStr);

    registrarCompraConCredito(telefono, nombreCliente, cedulaCliente, numeroCuotas,
        codigoCliente);
    });

    btnMostrarFactura.setOnClickListener(v -> mostrarFactura());
}

private void registrarCompraConCredito(Telefono telefono, String nombreCliente, String cedulaCliente,
    int numeroCuotas, int codigoCliente) {
    JSONObject compraJson = new JSONObject();
    try {
        compraJson.put("nombre", nombreCliente);
        compraJson.put("cedula", cedulaCliente);
        compraJson.put("valor", telefono.getPrecio());
        compraJson.put("codCliente", codigoCliente);
        compraJson.put("numeroCuotas", numeroCuotas);
    } catch (JSONException e) {
        Log.e("JSON_ERROR", "Error al crear el JSON", e);
        return;
    }

    RequestQueue requestQueue = Volley.newRequestQueue(this);
    JsonObjectRequest request = new JsonObjectRequest(
        Request.Method.POST,
        API_URL,
        compraJson,
        response -> {
            try {
                // Procesar respuesta exitosa
                String mensaje = response.getString("mensaje");
                facturaGenerada = response.getJSONObject("factura");
                tablaAmortizacion = response.getJSONArray("tablaAmortizacion");

                //Toast.makeText(this, mensaje, Toast.LENGTH_LONG).show();
                btnMostrarFactura.setVisibility(View.VISIBLE);

                // Mostrar factura y tabla de amortización
                AlertDialog.Builder builder = new AlertDialog.Builder(this);
                builder.setTitle("Compra Exitosa")
                    .setMessage("¡Compra con crédito realizada exitosamente! Puedes ver los detalles de
la factura.")
                    .setPositiveButton("Aceptar", null)
                    .show();
            } catch (JSONException e) {
                Log.e("JSON_ERROR", "Error al parsear la respuesta del servidor", e);
            }
        },

```

```

error -> {
    String errorMessage = "Error al realizar la compra con crédito";
    if (error.networkResponse != null && error.networkResponse.data != null) {
        try {
            String responseBody = new String(error.networkResponse.data, "UTF-8");
            JSONObject jsonError = new JSONObject(responseBody);

            String mensaje = jsonError.optString("mensaje", "Error desconocido");
            String razon = jsonError.optString("razon", "Razón no especificada");
            errorMessage = mensaje + "\nRazón: " + razon;
        } catch (Exception e) {
            Log.e("ERROR_PARSING", "Error al procesar la respuesta de error", e);
        }
    }

    // Mostrar error en un AlertDialog
    AlertDialog.Builder errorBuilder = new AlertDialog.Builder(this);
    errorBuilder.setTitle("Error en la Compra")
        .setMessage(errorMessage)
        .setPositiveButton("Aceptar", null)
        .show();

    // Mostrar también en un Toast para retroalimentación rápida
    //Toast.makeText(this, errorMessage, Toast.LENGTH_LONG).show();
    Log.e("API_ERROR", errorMessage, error);
}

);

requestQueue.add(request);
}

private void mostrarFactura() {
    if (facturaGenerada == null || tablaAmortizacion == null) {
        Toast.makeText(this, "No hay datos para mostrar.", Toast.LENGTH_SHORT).show();
        return;
    }

    try {
        StringBuilder facturaDetalle = new StringBuilder();
        facturaDetalle.append("Factura:\n")
            .append("ID: ").append(facturaGenerada.getInt("idFactura")).append("\n")
            .append("Fecha: ").append(facturaGenerada.getString("fecha")).append("\n")
            .append("Cliente: ").append(facturaGenerada.getString("clienteNombre")).append("\n")
            .append("Total: $").append(facturaGenerada.getDouble("total")).append("\n");

        facturaDetalle.append("\nTabla de Amortización:\n");
        for (int i = 0; i < tablaAmortizacion.length(); i++) {
            JSONObject cuota = tablaAmortizacion.getJSONObject(i);
            facturaDetalle.append("Cuota ").append(cuota.getInt("numeroCuota"))
                .append(": Valor Cuota $").append(cuota.getDouble("valorCuota")) // Agregamos el valor
de la cuota
                .append(", Capital $").append(cuota.getDouble("capitalPagado"))
                .append(", Interés $").append(cuota.getDouble("interesPagado"))
                .append(", Saldo $").append(cuota.getDouble("saldo"))
                .append("\n");
        }
    }
}

```

```

        AlertDialog.Builder builder = new AlertDialog.Builder(this);
        builder.setTitle("Factura y Tabla de Amortización")
            .setMessage(facturaDetalle.toString())
            .setPositiveButton("Aceptar", null)
            .show();
    } catch (JSONException e) {
        Log.e("JSON_ERROR", "Error al procesar los detalles de la factura", e);
    }
}
}
}

```

Tabla 31 Codificación archivo PagoEfectivoActivity

```

package ec.edu.monster.comercializadora.vista;

import android.app.AlertDialog;
import android.os.Bundle;
import android.util.Log;
import android.view.View;
import android.widget.Button;
import android.widget.EditText;
import android.widget.TextView;
import android.widget.Toast;
import androidx.appcompat.app.AppCompatActivity;
import com.android.volley.Request;
import com.android.volley.RequestQueue;
import com.android.volley.toolbox.JsonObjectRequest;
import com.android.volley.toolbox.Volley;
import org.json.JSONException;
import org.json.JSONObject;

import ec.edu.monster.comercializadora.R;
import ec.edu.monster.comercializadora.modelo.Telefono;

public class PagoEfectivoActivity extends AppCompatActivity {

    private static final String API_URL = "http://192.168.137.1:5012/facturas/efectivo"; // Cambia la IP si
    es necesario

    private Button btnMostrarFactura;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_pago_efectivo);

        TextView tvResumen = findViewById(R.id.tvResumenEfectivo);
    }
}

```

```

EditText etNombreCliente = findViewById(R.id.etNombreCliente);
EditText etCedulaCliente = findViewById(R.id.etCedulaCliente);
Button btnConfirmar = findViewById(R.id.btnConfirmarEfectivo);
btnMostrarFactura = findViewById(R.id.btnMostrarFactura);

// Obtener datos del intent
Telefono telefono = (Telefono) getIntent().getSerializableExtra("telefono");
double descuento = telefono.getPrecio() * 0.42;
double total = telefono.getPrecio() - descuento;

// Mostrar resumen del pago
tvResumen.setText(String.format(
    "Modelo: %s\nPrecio original: $%.2f\nDescuento: $%.2f\nTotal a pagar: $%.2f",
    telefono.getModelo(), telefono.getPrecio(), descuento, total
));

btnConfirmar.setOnClickListener(v -> {
    String nombreCliente = etNombreCliente.getText().toString().trim();
    String cedulaCliente = etCedulaCliente.getText().toString().trim();

    // Validar entrada del usuario
    if (nombreCliente.isEmpty()) {
        etNombreCliente.setError("El nombre del cliente es obligatorio.");
        return;
    }
    if (cedulaCliente.isEmpty()) {
        etCedulaCliente.setError("La cédula del cliente es obligatoria.");
        return;
    }

    // Enviar datos al servidor
    registrarFactura(telefono, nombreCliente, cedulaCliente);
});

btnMostrarFactura.setOnClickListener(v -> {
    Toast.makeText(this, "No hay factura disponible aún.", Toast.LENGTH_SHORT).show();
});

// Botón inicialmente oculto
btnMostrarFactura.setVisibility(View.GONE);
}

private void registrarFactura(Telefono telefono, String nombreCliente, String cedulaCliente) {
    // Crear el objeto JSON para enviar al servidor
    JSONObject facturaJson = new JSONObject();
    try {
        facturaJson.put("clienteNombre", nombreCliente);
        facturaJson.put("clienteCedula", cedulaCliente);
        facturaJson.put("precio", telefono.getPrecio());
    } catch (JSONException e) {

```



```

        Log.e("JSON_ERROR", "Error al crear el JSON", e);
        return;
    }

    // Crear la solicitud POST con Volley
    RequestQueue requestQueue = Volley.newRequestQueue(this);
    JsonObjectRequest request = new JsonObjectRequest(
        Request.Method.POST,
        API_URL,
        facturaJson,
        response -> {
            // Manejo de respuesta exitosa
            try {
                // Extraer los datos de la factura del JSON
                int idFactura = response.getInt("idFactura");
                String fecha = response.getString("fecha");
                String clienteNombre = response.getString("clienteNombre");
                String clienteCedula = response.getString("clienteCedula");
                String formaPago = response.getString("formaPago");
                double total = response.getDouble("total");
                double descuento = response.getDouble("descuento");

                // Mostrar mensaje de éxito
                new AlertDialog.Builder(this)
                    .setTitle("Pago Exitoso")
                    .setMessage("¡El pago se ha realizado exitosamente! La factura ha sido registrada.")
                    .setPositiveButton("Aceptar", (dialog, which) -> {
                        // Mostrar el botón de factura después de cerrar el diálogo
                        btnMostrarFactura.setVisibility(View.VISIBLE);
                        btnMostrarFactura.setOnClickListener(v -> {
                            new AlertDialog.Builder(this)
                                .setTitle("Factura Generada")
                                .setMessage(String.format(
                                    "ID Factura: %d\nFecha: %s\nCliente: %s\nCédula: %s\nForma de Pago: %s\nTotal: $%.2f\nDescuento: $%.2f",
                                    idFactura, fecha, clienteNombre, clienteCedula, formaPago, total,
                                    descuento
                                ))
                                .setPositiveButton("Aceptar", null)
                                .show();
                        });
                    })
                    .show();

            } catch (JSONException e) {
                Log.e("JSON_ERROR", "Error al parsear la respuesta del servidor", e);
            }
        },
        error -> {
            // Manejo de errores

```

```

        Log.e("API_ERROR", "Error al registrar la factura", error);
        Toast.makeText(this, "Error al registrar la factura", Toast.LENGTH_LONG).show();
    }
};

// Añadir la solicitud a la cola
requestQueue.add(request);
}

}

```

Tabla 32 Codificación LoginActivity

```

package ec.edu.monster.comercializadora.vista;

import android.content.Intent;
import android.os.Bundle;
import android.util.Log;
import android.widget.Button;
import android.widget.EditText;
import android.widget.Toast;
import androidx.appcompat.app.AppCompatActivity;
import com.android.volley.Request;
import com.android.volley.RequestQueue;
import com.android.volley.toolbox.JsonObjectRequest;
import com.android.volley.toolbox.StringRequest;
import com.android.volley.toolbox.Volley;
import org.json.JSONException;
import org.json.JSONObject;

import ec.edu.monster.comercializadora.R;

public class LoginActivity extends AppCompatActivity {

    private static final String API_URL = "http://192.168.137.1:5012/login"; // Cambia la IP si es necesario

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_login);

        EditText etUsername = findViewById(R.id.etUsername);
        EditText etPassword = findViewById(R.id.etPassword);
        Button btnLogin = findViewById(R.id.btnLogin);

        btnLogin.setOnClickListener(v -> {
            String username = etUsername.getText().toString().trim();
            String password = etPassword.getText().toString().trim();

            // Validar campos vacíos

```

```

        if (username.isEmpty()) {
            etUsername.setError("El nombre de usuario es obligatorio");
            return;
        }
        if (password.isEmpty()) {
            etPassword.setError("La contraseña es obligatoria");
            return;
        }

        // Realizar la autenticación
        autenticarUsuario(username, password);
    });
}

private void autenticarUsuario(String username, String password) {
    try {
        // Crear la solicitud JSON
        JSONObject loginRequest = new JSONObject();
        loginRequest.put("Username", username);
        loginRequest.put("Password", password);

        // Serializar el objeto JSON a un string
        String jsonRequest = loginRequest.toString();

        // Crear la solicitud POST
        RequestQueue requestQueue = Volley.newRequestQueue(this);
        StringRequest request = new StringRequest(
            Request.Method.POST,
            API_URL,
            response -> {
                try {
                    // Manejo de respuesta como string
                    int resultado = Integer.parseInt(response.trim());

                    if (resultado == 1) {
                        Toast.makeText(this, "Inicio de sesión exitoso", Toast.LENGTH_SHORT).show();

                        // Redirigir a la actividad principal
                        Intent intent = new Intent(LoginActivity.this, PrincipalActivity.class);
                        startActivity(intent);
                        finish();
                    } else {
                        Toast.makeText(this, "Credenciales incorrectas", Toast.LENGTH_SHORT).show();
                    }
                } catch (NumberFormatException e) {
                    Log.e("LOGIN_ERROR", "Error al procesar la respuesta", e);
                    Toast.makeText(this, "Respuesta inesperada del servidor", Toast.LENGTH_SHORT).show();
                }
            },
            error -> {

```

```

        // Manejo de errores
        Log.e("LOGIN_ERROR", "Error de red", error);
        Toast.makeText(this, "Error al conectar con el servidor", Toast.LENGTH_SHORT).show();
    }
}

@Override
public byte[] getBody() {
    // Enviar el cuerpo de la solicitud
    return jsonRequest.getBytes();
}

@Override
public String getBodyContentType() {
    // Configurar el tipo de contenido
    return "application/json";
}
};

// Añadir la solicitud a la cola
requestQueue.add(request);
} catch (JSONException e) {
    Log.e("LOGIN_ERROR", "Error al crear el JSON", e);
}
}
}

```

Tabla 32 Codificación AmortizacionActivity

```

package ec.edu.monster.comercializadora.vista;

import android.os.Bundle;
import android.util.Log;
import android.view.View;
import android.widget.Button;
import android.widget.EditText;
import android.widget.TextView;
import android.widget.Toast;

import androidx.appcompat.app.AppCompatActivity;

import com.android.volley.Request;
import com.android.volley.RequestQueue;
import com.android.volley.Response;
import com.android.volley.VolleyError;
import com.android.volley.toolbox.JsonArrayRequest;
import com.android.volley.toolbox.Volley;

import org.json.JSONArray;
import org.json.JSONException;

```

```

import org.json.JSONObject;

import ec.edu.monster.comercializadora.R;

public class AmortizacionActivity extends AppCompatActivity {

    private static final String BASE_URL = "http://192.168.137.1:5012/amortizacion/";
    private EditText etCodCredito;
    private Button btnConsultarAmortizacion;
    private TextView tvResultadoAmortizacion;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_amortizacion);

        // Referencias a los elementos de la vista
        etCodCredito = findViewById(R.id.etCodCredito);
        btnConsultarAmortizacion = findViewById(R.id.btnConsultarAmortizacion);
        tvResultadoAmortizacion = findViewById(R.id.tvResultadoAmortizacion);

        // Manejo del botón de consulta
        btnConsultarAmortizacion.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                String codCredito = etCodCredito.getText().toString().trim();

                if (codCredito.isEmpty()) {
                    Toast.makeText(AmortizacionActivity.this, "Por favor, ingrese un código de crédito.",
Toast.LENGTH_SHORT).show();
                    return;
                }

                consultarAmortizacion(codCredito);
            }
        });
    }

    private void consultarAmortizacion(String codCredito) {
        String url = BASE_URL + codCredito;

        // Crear una solicitud GET con Volley
        RequestQueue requestQueue = Volley.newRequestQueue(this);
        JsonArrayRequest jsonArrayRequest = new JsonArrayRequest(
            Request.Method.GET,
            url,
            null,
            new Response.Listener<JSONArray>() {
                @Override
                public void onResponse(JSONArray response) {

```

```

try {
    // Procesar la respuesta
    StringBuilder resultado = new StringBuilder();
    resultado.append("Tabla de Amortización:\n");

    for (int i = 0; i < response.length(); i++) {
        JSONObject cuota = response.getJSONObject(i);
        resultado.append("Cuota ").append(cuota.getInt("numeroCuota"))
            .append(": Valor Cuota $").append(cuota.getDouble("valorCuota"))
            .append(", Capital $").append(cuota.getDouble("capitalPagado"))
            .append(", Interés $").append(cuota.getDouble("interesPagado"))
            .append(", Saldo $").append(cuota.getDouble("saldo"))
            .append("\n");
    }

    // Mostrar el resultado en el TextView
    tvResultadoAmortizacion.setText(resultado.toString());
} catch (JSONException e) {
    Toast.makeText(AmortizacionActivity.this, "Error al procesar los datos.",
        Toast.LENGTH_SHORT).show();
    Log.e("JSON_ERROR", "Error al procesar la respuesta JSON", e);
}
},
new Response.ErrorListener() {
    @Override
    public void onErrorResponse(VolleyError error) {
        // Manejar errores
        String mensaje = "Error al consultar la tabla de amortización.";
        if (error.networkResponse != null && error.networkResponse.statusCode == 404) {
            mensaje = "No se encontró información para el crédito especificado.";
        }
        Toast.makeText(AmortizacionActivity.this, mensaje, Toast.LENGTH_SHORT).show();
        Log.e("API_ERROR", "Error en la solicitud: " + error.getMessage(), error);
    }
}
);

// Agregar la solicitud a la cola
requestQueue.add(jsonArrayRequest);
}
}

```

EJECUCIÓN

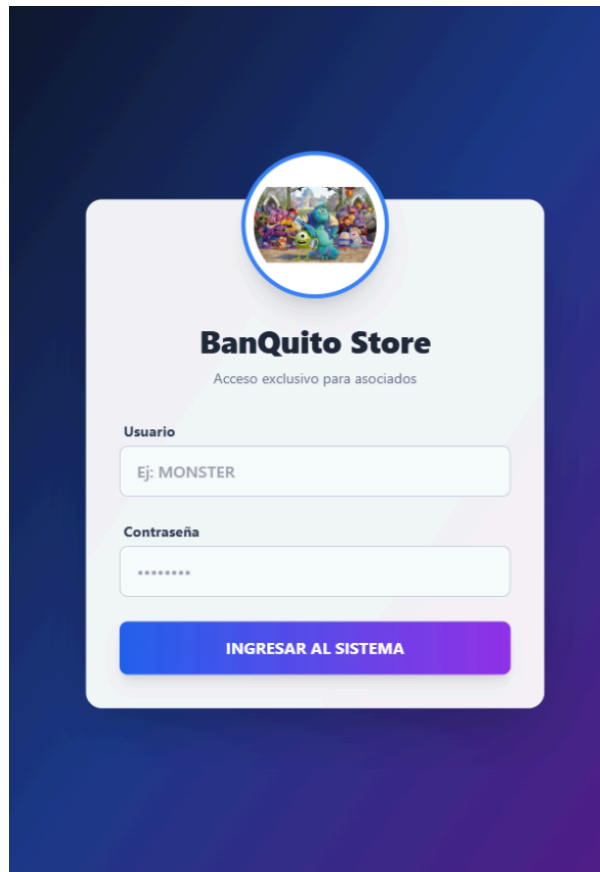


Figura 117 Ejecución Pantalla Inicio de Sesión

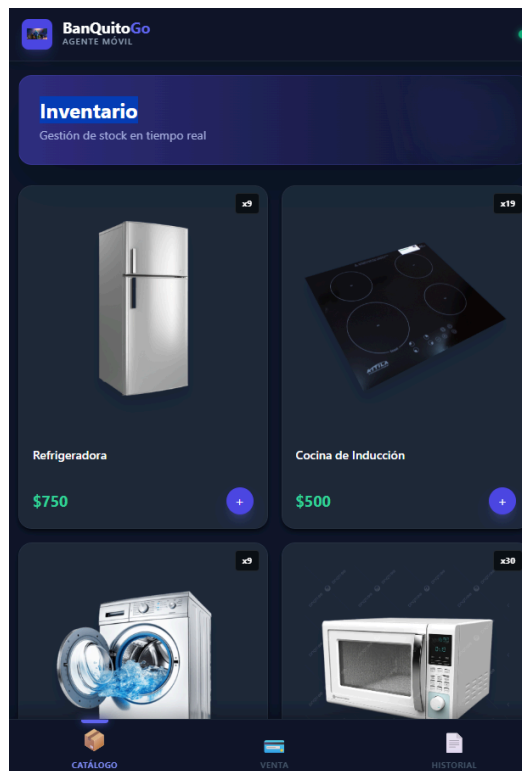


Figura 117 Ejecución Menú Principal

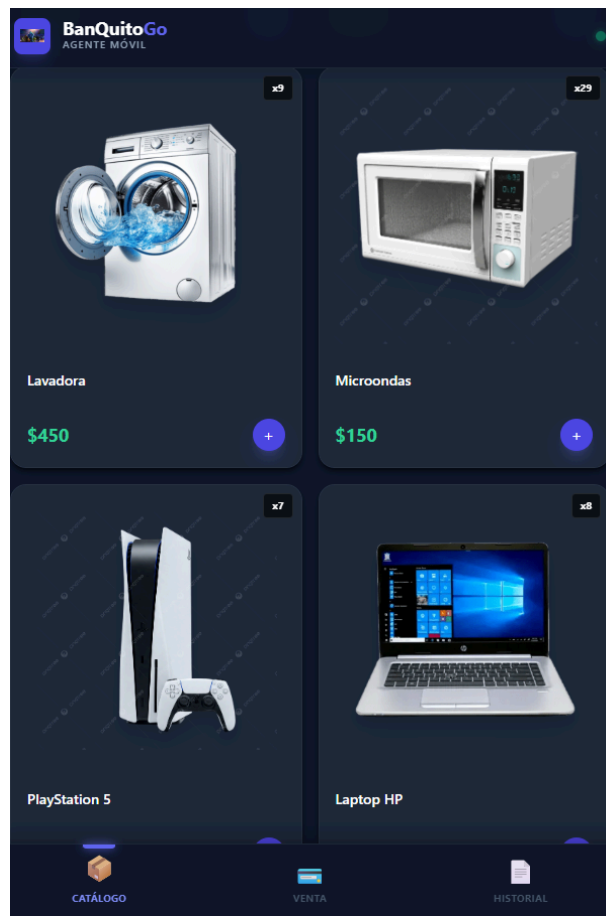


Figura 117 CRUD de teléfonos.

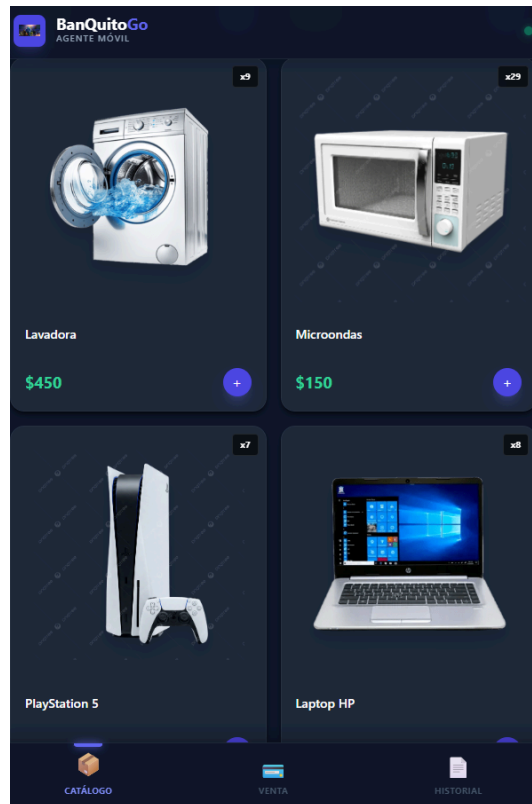


Figura 119 Ver Catálogo

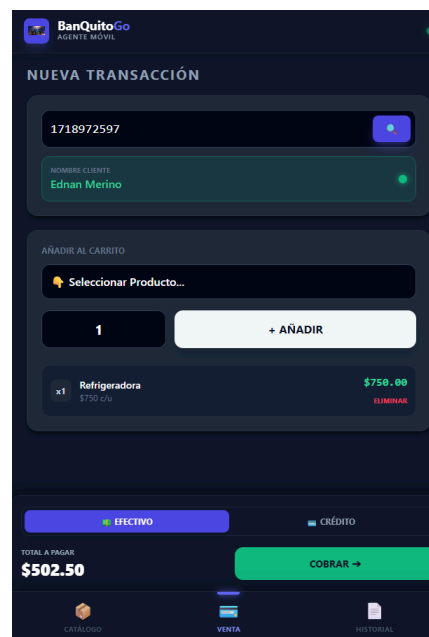


Figura 120 Carrito de compras

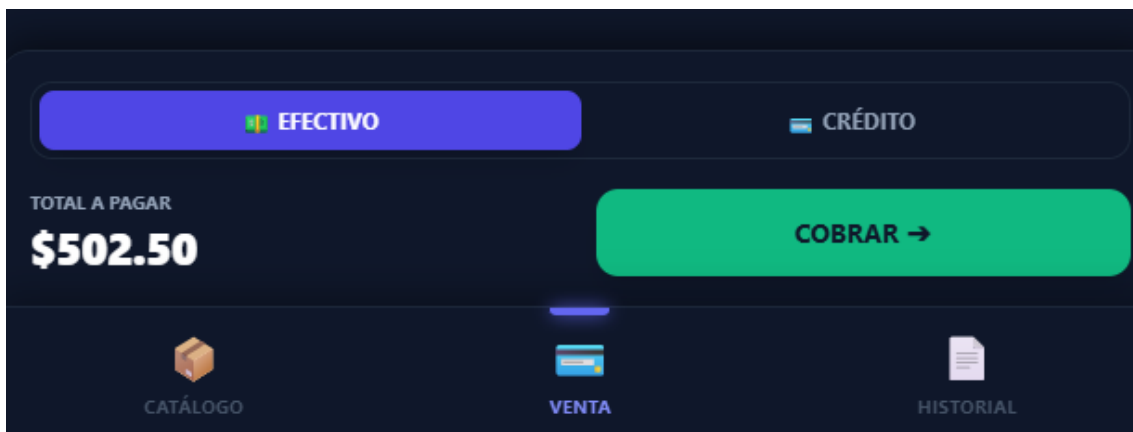


Figura 121 Ejecución Ver Método de pago



Figura 119 Ejecución Pago en efectivo

CENTRO DE CONSULTAS

 FACTURA

 CRÉDITO



FACTURA #15

17/12/2025

CLIENTE

Ednan Merino

1718972597

Refrigeradora

Cant: 1

\$750

TOTAL PAGADO

Efectivo

\$502.5

CATÁLOGO

VENTA

HISTORIAL

Figura 120 Ejecución Factura generada pago en efectivo

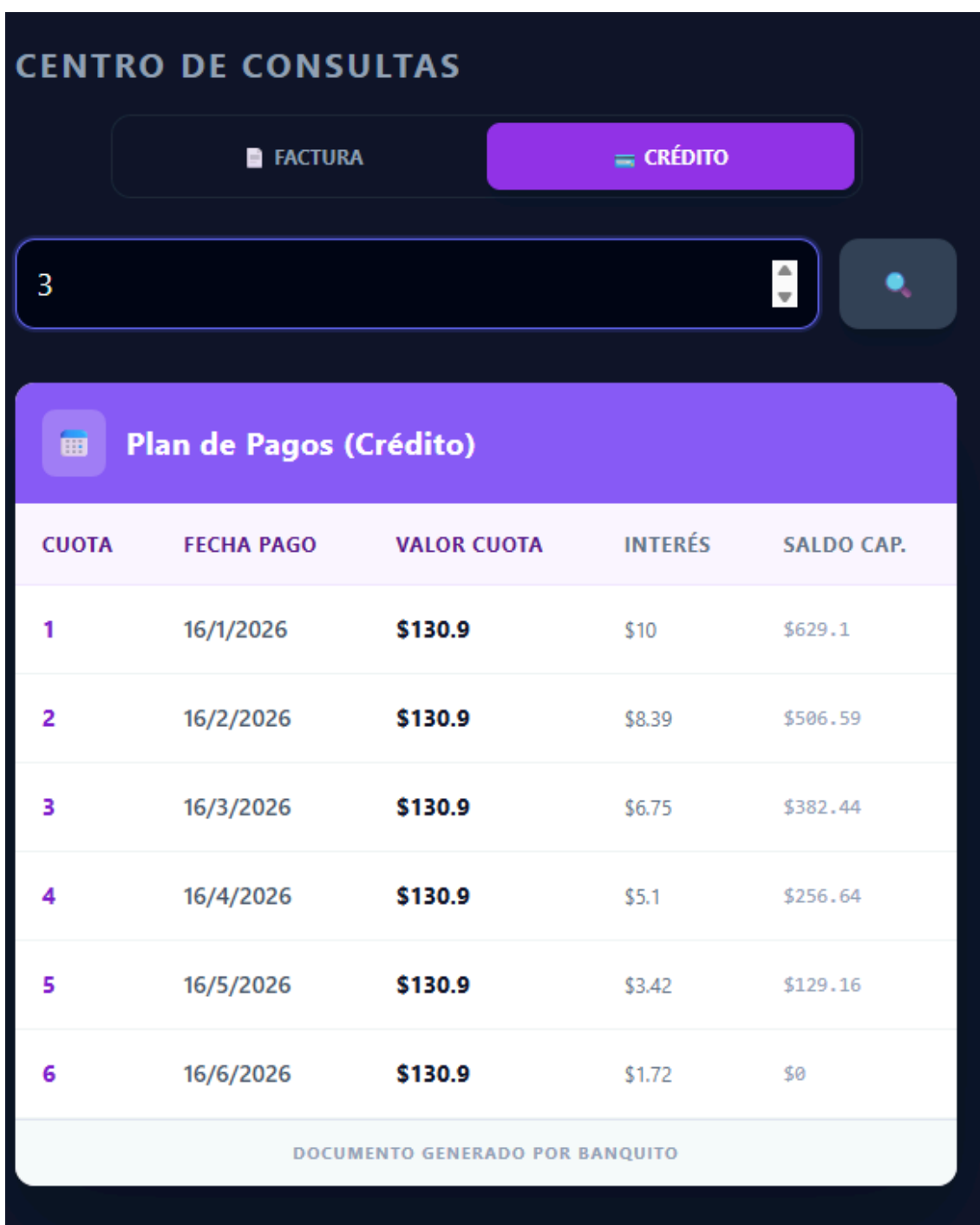


Figura 123 Vista Factura y Tabla de Amortización generada

CONCLUSIONES

- **Integración Eficiente con Servicios SOAP:**

La implementación de servicios web SOAP permite una comunicación estructurada y segura con proveedores, sistemas de facturación y plataformas de pago. Esto garantiza la actualización en tiempo real del stock de teléfonos celulares, la

validación de transacciones y la automatización de procesos clave en la comercialización.

- **Arquitectura Modular y Escalable:**

La separación de responsabilidades en capas (modelo, controlador y vista) facilita el mantenimiento, la depuración y la evolución del sistema. La modularidad permite la incorporación de nuevas funcionalidades sin afectar el núcleo del software, asegurando la adaptabilidad a futuras necesidades del negocio.

- **Manejo Robusto de Errores y Seguridad:**

Se ha integrado un sistema de validaciones y captura de excepciones en la comunicación con los servicios SOAP, evitando interrupciones en los procesos críticos. Además, la seguridad en la transmisión de datos protege la información de clientes y transacciones, garantizando la confiabilidad del sistema.

RECOMENDACIONES

- **Optimización del Rendimiento y Escalabilidad:**

Implementar mecanismos de caché para reducir la carga en los servicios SOAP y optimizar la velocidad de respuesta. Además, considerar la inyección de dependencias para mejorar la flexibilidad del código y facilitar futuras ampliaciones del sistema.

- **Automatización de Pruebas y Monitoreo de Errores:**

Desarrollar pruebas unitarias y de integración para validar la correcta ejecución de las operaciones con los servicios SOAP. Complementar con un sistema de registro de errores en archivos o bases de datos, permitiendo una rápida identificación y resolución de problemas en producción.

- **Mejora en la Experiencia del Usuario:**

Optimizar la interfaz y navegación del sistema, asegurando que los usuarios puedan realizar consultas de productos, gestionar pedidos y emitir facturas de manera intuitiva. Incluir mensajes de confirmación en operaciones críticas y permitir un flujo de navegación flexible para mejorar la interacción con el sistema.